

UNIVERSITÀ DEGLI STUDI DI PADOVA

Dipartimento di Fisica e Astronomia "Galileo Galilei"

Master's Degree in Physics of Data

Machine Learning

Prof. P. Zanuttigh

Lecture Notes

Nicolò Lai

nicolo.lai@studenti.unipd.it

Academic Year 2021/2022

Contents

1 Introduction to Machine Learning **4**
- 1.1 The statistical learning framework . . . 4
- 1.2 Empirical risk minimization . . . 5
  - 1.2.1 Overfitting and inductive bias . . . 5
  - 1.2.2 Finite hypothesis classes . . . 5

2 PAC Learning and its extensions **7**
- 2.1 PAC learning . . . 7
- 2.2 Agnostic PAC learning . . . 8
  - 2.2.1 Releasing the realizability assumption . . . 8
  - 2.2.2 Generalized loss functions . . . 9

3 Learning via Uniform Convergence **11**
- 3.1 Uniform convergence . . . 11
- 3.2 Finite hypothesis classes . . . 12
  - 3.2.1 Discretization trick . . . 13

4 The Bias-Complexity Tradeoff **14**
- 4.1 No free-lunch theorem . . . 14
- 4.2 Error decomposition . . . 15
- 4.3 Training, validation and test sets . . . 16

5 PAC Learnability for infinite-size classes **17**
- 5.1 The VC-dimension . . . 17
- 5.2 Examples of VC-dimension computation . . . 18
  - 5.2.1 Threshold function . . . 18
  - 5.2.2 Intervals . . . 19
  - 5.2.3 Finite classes . . . 19
  - 5.2.4 Number of parameters . . . 19
- 5.3 The fundamental theorem of PAC learning . . . 20

6 Linear Predictors **21**
- 6.1 Affine functions . . . 21
  - 6.1.1 2D geometric interpretation . . . 21
- 6.2 Homogeneous linear functions . . . 22
- 6.3 Halfspaces . . . 22
  - 6.3.1 2D geometric interpretation . . . 22
- 6.4 Perceptron for halfspaces . . . 23
- 6.5 VC-dimension of halfspaces . . . 24
- 6.6 Linear regression . . . 25
  - 6.6.1 Least squares . . . 25
  - 6.6.2 Polynomial regression . . . 26
- 6.7 Logistic regression . . . 26

7 Model Selection and Validation **28**
7.1 Validation . . . 29
7.1.1 Validation set . . . 29
7.1.2 Validation for model selection . . . 29
7.1.3 Model-selection curve . . . 31
7.1.4 Grid-search for multiple parameters . . . 31
7.1.5 Train-validation-test split . . . 32
7.1.6 k-Fold cross validation . . . 32
7.2 What if learning fails . . . 33
7.2.1 Error decomposition using validation . . . 33
7.2.2 Learning curves . . . 34

8 Regularization and Stability **36**
8.1 Regularized loss minimization . . . 36
8.1.1 Ridge regression . . . 36
8.2 Stable rules do not overfit . . . 37
8.3 Tikhonov regularization as a stabilizer . . . 38
8.4 Fitting-stability tradeoff . . . 38

9 Support Vector Machines **39**
9.1 Classification margin . . . 39
9.2 Hard-SVM . . . 40
9.2.1 The homogeneous case . . . 41
9.2.2 Support vectors . . . 41
9.2.3 Duality . . . 42
9.3 Soft-SVM . . . 42

10 Kernel Methods **44**
10.1 Embeddings into feature spaces . . . 44
10.2 The kernel trick . . . 45
10.2.1 Polynomial kernels . . . 47
10.2.2 Radial basis functions . . . 48
10.2.3 The λ parameter . . . 48
10.2.4 Kernels comparison . . . 49

11 Stochastic Gradient Descent **50**
11.1 Gradient descent . . . 50
11.1.1 Accuracy and convergence . . . 51
11.2 Stochastic gradient descent . . . 52
11.2.1 SGD vs GD . . . 52
11.3 Learning with SGD . . . 53
11.3.1 SGD for Risk Minimization . . . 53
11.3.2 SGD for Support Vector Machines . . . 54
11.4 Final considerations on SGD . . . 54
11.4.1 Learning rate . . . 54
11.4.2 Momentum . . . 55
11.4.3 Advanced SGD schemes . . . 55

12 Decision Trees **56**
12.1 Decision tree algorithms . . . 56
12.1.1 Iterative dichotomizer 3 . . . 57
12.1.2 Gain measure . . . 57
12.1.3 Pruning . . . 58
12.2 Random forests . . . 58

12.2.1 Bootstrap aggregation . . . 59
12.2.2 Feature randomness . . . 59

13 Neural Networks **60**
13.1 Feed-forward neural networks . . . 60
13.2 Activation functions . . . 62
13.2.1 Sign and threshold functions . . . 62
13.2.2 Sigmoid function . . . 62
13.2.3 Hyperbolic tangent . . . 63
13.2.4 Rectified linear unit . . . 63
13.3 Forward propagation . . . 64
13.4 Learning neural networks . . . 64
13.4.1 The expressive power of neural networks . . . 64
13.4.2 Conjunction and disjunction . . . 65
13.4.3 Geometric intuition . . . 65
13.5 Sample complexity of neural networks . . . 66
13.6 Runtime of learning neural networks . . . 66
13.7 SGD and backpropagation . . . 67
13.7.1 Backpropagation . . . 68
13.8 Training details . . . 69
13.8.1 Pre-processing and weights initialization . . . 69
13.8.2 Stopping the training process . . . 70
13.8.3 Regularization . . . 70

14 Convolutional Neural Networks **71**
14.1 Learning an image . . . 71
14.2 From neural networks to convolutional neural networks . . . 72
14.2.1 Local connectivity . . . 72
14.2.2 Shared weights . . . 73
14.2.3 Multiple feature maps . . . 73
14.3 The convolution operation . . . 73
14.4 The pooling operation . . . 74
14.5 Overfitting . . . 76
14.5.1 Drop-out . . . 76
14.5.2 Regularization . . . 77
14.5.3 Early stopping . . . 77
14.5.4 Data augmentation . . . 78
14.6 Optimization . . . 78
14.7 Loss function . . . 78
14.8 Generative Adversarial Networks . . . 78
14.8.1 Loss function . . . 79

15 Unsupervised learning **80**
15.1 Clustering . . . 80
15.1.1 A clustering model . . . 80
15.1.2 K-Means clustering algorithm . . . 81
15.1.3 Linkage-based clustering algorithms . . . 82
15.2 Principal Component Analysis (PCA) . . . 83
15.2.1 Mathematical construction . . . 83
15.2.2 Implementation and demonstration . . . 84

Chapter 1

Introduction to Machine Learning

1.1 The statistical learning framework

- **The learner's input** the machine learning algorithm has access to
  - **Domain set $\boldsymbol{\mathcal{X}}$:** the set of objects that we may wish to label, make predictions about. Usually, domain points will be represented by a vector of *features*. we also refer to domain points as *instances* and to $\boldsymbol{\mathcal{X}}$ as *instance space*.
  - **Label set $\boldsymbol{\mathcal{Y}}$:** the set of possible labels. In binary classification, for example, we have $\boldsymbol{\mathcal{Y}} = \{0, 1\}$ (or $\boldsymbol{\mathcal{Y}} = \{-1, +1\}$ as we will see later in the course).
  - **Training data $\boldsymbol{\mathcal{S}}$:** the finite sequence of pairs in $\boldsymbol{\mathcal{X}} \times \boldsymbol{\mathcal{Y}}$, namely a sequence of labeled domain points $\boldsymbol{\mathcal{S}} = ((x_1, y_1) \dots (x_m, y_m))$. This is the input that the learner (machine learning algorithm) has access to. Such labeled examples are often called *training examples*. We also refer to $\boldsymbol{\mathcal{S}}$ as the *training set* (despite it being a sequence, not a set).
- **The learner's output:** the machine learning algorithm is requested to return a *prediction rule* $h : \boldsymbol{\mathcal{X}} \to \boldsymbol{\mathcal{Y}}$. This function is also called *predictor*, *hypothesis* or *classifier*. The predictor is often used to predict the label of new domain points. We use the notation $A(\boldsymbol{\mathcal{S}})$ to denote the prediction rule that a particular learning algorithm A returns upon receiving the training sequence $\boldsymbol{\mathcal{S}}$ (note that it depends on data).
- **Data-generation model:** we assume that the instances are generated by some probability distribution. Let us denote that probability distribution over $\boldsymbol{\mathcal{X}}$ as $\boldsymbol{\mathcal{D}}$. We do not assume that the learner knows anything about this distribution. As to the labels, we assume that there is some "correct" labeling function $f : \boldsymbol{\mathcal{X}} \to \boldsymbol{\mathcal{Y}}$ such that $y_i = f(x_i)\ \forall i$. The labeling function f is also unknown to the learner (it is actually what the learner is trying to figure out).
  In other words, each pair in the training data $\boldsymbol{\mathcal{S}}$ is generated by first sampling a point x_i from $\boldsymbol{\mathcal{X}}$ according to $\boldsymbol{\mathcal{D}}$ and then labeling it by f.
- **Measure of success:** we define the *error of a classifier* to be the probability that it does not predict the correct label on a random data point generated by the aforementioned underlying distribution. The error of h is thus the probability to draw a random instance $x \sim \boldsymbol{\mathcal{D}}$ such that $h(x) \neq f(x) \equiv y$.
  We define the error of a prediction rule $h : \boldsymbol{\mathcal{X}} \to \boldsymbol{\mathcal{Y}}$ to be

$$L_{\boldsymbol{\mathcal{D}}, f}(h) = \mathcal{P}_{x \sim \boldsymbol{\mathcal{D}}}[h(x) \neq f(x)] = \boldsymbol{\mathcal{D}}(\{x : h(x) \neq f(x)\}) \tag{1.1}$$

  and depends on the distribution $\boldsymbol{\mathcal{D}}$ and the *true* labeling function f. This $L_{\boldsymbol{\mathcal{D}}, f}(h)$ is called *generalization error*, *(true) risk*, *(true) error* or also *(true) loss*.

1.2 Empirical risk minimization

As mentioned earlier, a learning algorithm receives as input the training data $\boldsymbol{\mathcal{S}}$ sampled from an *unknown* distribution $\boldsymbol{\mathcal{D}}$ and labeled by some target function f (also *unknown*) and should return a predictor $h_{\boldsymbol{\mathcal{S}}} : \boldsymbol{\mathcal{X}} \to \boldsymbol{\mathcal{Y}}$ (that depends on training data, that is the input of the learner). The goal of the algorithm is to find $h_{\boldsymbol{\mathcal{S}}}$ that minimizes the error with respect to the unknown $\boldsymbol{\mathcal{D}}$ and f.
Since the learner does not know $\boldsymbol{\mathcal{D}}$ and f, the true error is not directly accessible to the learner. What we can compute is the *training error* (i.e., the error the classifier incurs over the training data), which is also called *empirical error* and *empirical risk*:

$$\begin{aligned} L_{\boldsymbol{\mathcal{S}}}(h) &= \frac{|\{i \ : \ h(x_i) \neq y_i, \ i = 1 \dots m\}|}{m} \\ &= \frac{\#\text{ wrong predictions}}{\#\text{ training instances}} \end{aligned} \tag{1.2}$$

Since the training sample is the only information available to the learner, it makes sense to search for a solution that works well on that data. This learning paradigm that is coming up with a predictor h that minimizes $L_{\boldsymbol{\mathcal{S}}}(h)$ is called *Empirical Risk Minimization* (ERM for short.)

1.2.1 Overfitting and inductive bias

Due to its nature, ERM can easily lead to *overfitting*. Overfitting occurs when our hypothesis fits the training data too well, or, from a different point of view, the training set does not represent the real data too well. Thus, the learner returns an optimal $h_{\boldsymbol{\mathcal{S}}}$ that minimizes $L_{\boldsymbol{\mathcal{S}}}(h_{\boldsymbol{\mathcal{S}}})$, but that works badly on different datasets.

We need to search for conditions under which there is a guarantee that ERM does not overfit. Namely, conditions under which the ERM predictor has good performance with respect to the training data and a high probability of performing well over the underlying data distribution.
A standard solution is to apply the ERM learning rule over a restricted search space. In other words, the learner should choose before seeing the data a set of predictors. This set is called *hypothesis class* and is denoted by $\boldsymbol{\mathcal{H}}$. Each $h \in \boldsymbol{\mathcal{H}}$ is a function mapping from $\boldsymbol{\mathcal{X}}$ to $\boldsymbol{\mathcal{Y}}$. For a given class $\boldsymbol{\mathcal{H}}$ and training set $\boldsymbol{\mathcal{S}}$, the $\text{ERM}_{\boldsymbol{\mathcal{H}}}$ learner uses the ERM rule to choose a predictor $h \in \boldsymbol{\mathcal{H}}$ inside the hypothesis class with the lowest possible training error. Formally we have

$$\text{ERM}_{\boldsymbol{\mathcal{H}}}(\boldsymbol{\mathcal{S}}) \in \arg\min_{h \in \boldsymbol{\mathcal{H}}} L_{\boldsymbol{\mathcal{S}}}(h) \tag{1.3}$$

where argmin stantds for the set of hypothesis in $\boldsymbol{\mathcal{H}}$ that achieve the minimum value of $L_{\boldsymbol{\mathcal{S}}}(h)$ over $\boldsymbol{\mathcal{H}}$ (there can be more than one). By restricting the learner to choosing a predictor from $\boldsymbol{\mathcal{H}}$, we bias it toward a particular set of predictors: such restrictions are thus called an *inductive bias*. Since the choice of such a restriction is determined before the learner sees the data, and it should be based on some prior knowledge of the problem at hand.

1.2.2 Finite hypothesis classes

The type of restriction on a class is imposing an upper bound on its size (that is, the number of predictors inside $\boldsymbol{\mathcal{H}}$). We will show that if $\boldsymbol{\mathcal{H}}$ is a finite class, then $\text{ERM}_{\boldsymbol{\mathcal{H}}}$ will not overfit, provided it is based on a sufficiently large training sample.

Let us analyze the performance of the $\text{ERM}_{\boldsymbol{\mathcal{H}}}$ rule assuming that $\boldsymbol{\mathcal{H}}$ is a finite class. For a training sample $\boldsymbol{\mathcal{S}}$ labeled according to some $f : \boldsymbol{\mathcal{X}} \to \boldsymbol{\mathcal{Y}}$, let $h_{\boldsymbol{\mathcal{S}}}$ denote a result of $\text{ERM}_{\boldsymbol{\mathcal{H}}}$ to $\boldsymbol{\mathcal{S}}$. Namely,

$$h_{\boldsymbol{\mathcal{S}}} \in \arg\min_{h \in \boldsymbol{\mathcal{H}}} L_{\boldsymbol{\mathcal{S}}}(h) \tag{1.4}$$

is a predictor h that minimizes the empirical error over $\boldsymbol{\mathcal{S}}$.

We now need two more *key* assumptions:

1. **Realizability assumption:** there exists $h^* \in \boldsymbol{\mathcal{H}}$ such that $L_{\boldsymbol{\mathcal{D}},f}(h^*) = 0$.
   Thus, the assumption ensures the existence of a perfect solution over the whole dataset. Thus, with probability 1 over random samples $\boldsymbol{\mathcal{S}}$ (where instances of $\boldsymbol{\mathcal{S}}$ are sampled according to $\boldsymbol{\mathcal{D}}$ and labeled by f) we have $L_{\boldsymbol{\mathcal{S}}}(h^*) = 0$. In other words, the perfect solution h^* over the whole dataset is perfect over every possible training set $\boldsymbol{\mathcal{S}}$.

2. **Independent identical distribution assumption:** the examples in the training set are independently and identically distributed according to the distribution $\boldsymbol{\mathcal{D}}$. We denote this assumption by $S \sim \boldsymbol{\mathcal{D}}^m$ where m is the size of $\boldsymbol{\mathcal{S}}$ and $\boldsymbol{\mathcal{D}}^m$ denotes the probability over m-tuples induced by applying $\boldsymbol{\mathcal{D}}$ to pick each element of the tuple independently of the other members of the tuple.

The realizability assumption implies that for every ERM hypothesis, we have that $L_{\boldsymbol{\mathcal{S}}}(h_{\boldsymbol{\mathcal{S}}}) = 0$. However, we are interested in the *true* error of $h_{\boldsymbol{\mathcal{S}}}$, that is, $L_{\boldsymbol{\mathcal{D}},f}(h_{\boldsymbol{\mathcal{S}}})$, rather than its empirical risk.

Since $L_{\boldsymbol{\mathcal{D}},f}(h_{\boldsymbol{\mathcal{S}}})$ depends on the training set $\boldsymbol{\mathcal{S}}$, and that training set is picked by a random process, there is randomness in the choice of the predictor $h_{\boldsymbol{\mathcal{S}}}$ and, consequently, in the risk $L_{\boldsymbol{\mathcal{D}},f}(h_{\boldsymbol{\mathcal{S}}})$. Formally, we say that it is a random variable. It is not realistic to expect that with full certainty $\boldsymbol{\mathcal{S}}$ will suffice to direct the learner toward a good classifier (from the point of view of $\boldsymbol{\mathcal{D}}$), as there is always some probability that the sampled training data happens to be very nonrepresentative of the underlying $\boldsymbol{\mathcal{D}}$. We will therefore address the *probability* to sample a training set for which $L_{\boldsymbol{\mathcal{D}},f}(h_{\boldsymbol{\mathcal{S}}})$ is not too large. Usually, we denote the probability of getting a nonrepresentative sample by δ, thus calling $(1-\delta)$ the *confidence parameter* of our prediction. On top of that, since we cannot guarantee perfect label prediction, we introduce another parameter for prediction quality: the *accuracy parameter* ϵ. We interpret the event $L_{\boldsymbol{\mathcal{D}},f}(h_{\boldsymbol{\mathcal{S}}}) > \epsilon$ as a failure of the learner. On the other hand, if $L_{\boldsymbol{\mathcal{D}},f}(h_{\boldsymbol{\mathcal{S}}}) < \epsilon$, we view the output of the algorithm as an approximately correct predictor.

Theorem 1.2.1. *Let $\boldsymbol{\mathcal{H}}$ be a finite hypothesis class. Let $\epsilon \in (0,1)$ and $\delta \in (0,1)$ and let $m \in \mathbb{N}$ be an integer that satisfies*

$$m \geq \frac{\log\left(\frac{|\boldsymbol{\mathcal{H}}|}{\delta}\right)}{\epsilon} \tag{1.5}$$

Then, for any labeling function f and for any distribution $\boldsymbol{\mathcal{D}}$ for which the realizability assumption holds, with a probability of at least $1-\delta$ over the choice of an i.i.d sample $\boldsymbol{\mathcal{S}}$ of size m, we have that for every ERM hypothesis $h_{\boldsymbol{\mathcal{S}}}$ it holds that

$$L_{\boldsymbol{\mathcal{D}},f}(h_{\boldsymbol{\mathcal{S}}}) \leq \epsilon \tag{1.6}$$

This theorem tells us that if $\boldsymbol{\mathcal{H}}$ is a finite hypothesis class, then the $\text{ERM}_{\boldsymbol{\mathcal{H}}}$ rule will be *probably* (with confidence $1-\delta$) *approximately* (up to an error ϵ) correct, provided it is computed on a sufficiently large training set $\boldsymbol{\mathcal{S}}$.
Remember that the theorem requires the realizability assumption: by applying the $\text{ERM}_{\boldsymbol{\mathcal{H}}}$ rule over the training set $\boldsymbol{\mathcal{S}}$, we find the predictor $h_{\boldsymbol{\mathcal{S}}}$ that leads to $L_{\boldsymbol{\mathcal{S}}}(h_{\boldsymbol{\mathcal{S}}}) = 0$ (because of realizability). Moreover, if $\boldsymbol{\mathcal{H}}$ is a finite hypothesis class, then $L_{\boldsymbol{\mathcal{S}}}(h_{\boldsymbol{\mathcal{S}}})$ will probably be close to the true error $L_{\boldsymbol{\mathcal{D}},f}(h_{\boldsymbol{\mathcal{S}}})$ up to an error ϵ, provided it is computed on a sufficiently large training set $\boldsymbol{\mathcal{S}}$.

Chapter 2

PAC Learning and its extensions

2.1 PAC learning

In the previous chapter, we have seen that for a finite hypothesis class under the realizability assumption, if the ERM rule with respect to that class is applied on a sufficiently large training sample, then the output hypothesis will be probably approximately correct.

Definition 2.1.1 (PAC learnability)**.** *A hypothesis class $\boldsymbol{\mathcal{H}}$ is PAC learnable if there exists a function*

$$m_{\boldsymbol{\mathcal{H}}} : (0,1)^2 \to \mathbb{N}$$

and a learning algorithm such that

$$\forall\, \epsilon,\, \delta \in (0,1),\ \forall\, \boldsymbol{\mathcal{D}}\ over\ \boldsymbol{\mathcal{X}},\ \forall f : \boldsymbol{\mathcal{X}} \to \{0,1\},\ \textit{if the realizability assumption holds w.r.t. } \boldsymbol{\mathcal{H}},\, \boldsymbol{\mathcal{D}},\, f$$

then when running the learning algorithm on $m \geq m_{\boldsymbol{\mathcal{H}}}(\epsilon,\, \delta)$ i.i.d examples generated by $\boldsymbol{\mathcal{D}}$ and labeled by f, the algorithm returns a hypothesis h such that $L_{\boldsymbol{\mathcal{D}},f}(h) \leq \epsilon$ with probability of at least $1-\delta$ over the choice of the examples.

The definition of PAC learnability contains two approximation parameters. The accuracy parameter ϵ determines how far the classifier's output can be from the optimal one. The confidence parameter δ indicates how likely the classifier is to meet that accuracy requirement. Note that these approximations are inevitable. Since the training set is randomly generated, there may always be a small chance that it will be noninformative. Furthermore, even when we are lucky enough to get a training sample that faithfully represents $\boldsymbol{\mathcal{D}}$ because it is just a finite sample, there may always be some fine details of $\boldsymbol{\mathcal{D}}$ that it fails to reflect.

The function $m_{\boldsymbol{\mathcal{H}}} : (0,1)^2 \to \mathbb{N}$ determines the *sample complexity* of learning $\boldsymbol{\mathcal{H}}$: that is, how many examples are required to guarantee a probably approximately correct solution. It is a function of the two approximation parameters ϵ and δ. Note that if $\boldsymbol{\mathcal{H}}$ is PAC learnable, then there are many functions $m_{\boldsymbol{\mathcal{H}}}$ that satisfy the requirements given in the definition of PAC learnability. Therefore, to be precise, we will define the sample complexity of learning $\boldsymbol{\mathcal{H}}$ to be the minimal function (i.e., the one that leads to the lowest integer m) that satisfies the requirements of PAC learnability.

The conclusion on the analysis of finite hypothesis classes from the previous chapter can be rephrased as stating:

Corollary 2.1.0.1. *Every finite hypothesis class is PAC learnable with sample complexity*

$$m_{\boldsymbol{\mathcal{H}}}(\epsilon,\, \delta) \leq \left\lceil \frac{\log\left(\frac{|\boldsymbol{\mathcal{H}}|}{\delta}\right)}{\epsilon} \right\rceil \tag{2.1}$$

where we put "less or equal than" because it is a sufficient (but not necessary) condition. If the sample size is bigger than this threshold, then we can directly apply Theorem 1.2.1.

2.2 Agnostic PAC learning

We now generalize the model we have just described in two aspects:

1. **Removing the realizability assumption:**
   We have required that the learning algorithm succeeds on a pair of data distribution $\boldsymbol{\mathcal{D}}$ and a labeling function f provided that the realizability assumption is met. However, for practical learning tasks, this assumption may be too strong.

2. **Learning problems beyond binary classification:**
   The learning task we have been discussing so far is predicting a binary label to a given example. However, many learning tasks take a different form. For example, one may wish to predict a real number or a label picked from a finite set of labels.

2.2.1 Releasing the realizability assumption

A new model for the data-generating distribution Recall that the realizability assumption requires that there exists $h^* \in \boldsymbol{\mathcal{H}}$ such that $\mathbb{P}_{x\sim\boldsymbol{\mathcal{D}}}[h^*(x) = f(x)] = 1$. In many practical problems, thus, the assumption does not hold. Furthermore, it may be more realistic not to assume that the labels are fully determined by the features we measure on input elements. We thus relax the realizability assumption by replacing the target labeling function f with a data-labels generating distribution.
From now on, let $\boldsymbol{\mathcal{D}}$ be a probability distribution over $\boldsymbol{\mathcal{Z}} \equiv \boldsymbol{\mathcal{X}} \times \boldsymbol{\mathcal{Y}}$. That is, $\boldsymbol{\mathcal{D}}$ is a *joint distribution* over domain points and labels. One can see such distribution as composed of two parts:

1. *marginal distribution* $\boldsymbol{\mathcal{D}}_x$: distribution over unlabeled domain points

2. *conditional distribution* $\boldsymbol{\mathcal{D}}((x,\, y)\,|\,x)$: distribution over labels for each domain point

Revision of empirical and true errors For a probability distribution $\boldsymbol{\mathcal{D}}$ over $\boldsymbol{\mathcal{Z}}$, one can measure how likely is h to make an error when labeled points are randomly drawn according to $\boldsymbol{\mathcal{D}}$. We re-define the *true* error of a prediction rule h to be

$$L_{\boldsymbol{\mathcal{D}}}(h) = \mathbb{P}_{(x,\,y)\sim\boldsymbol{\mathcal{D}}}[h(x) \neq y] = \boldsymbol{\mathcal{D}}(\{(x,\, y) : h(x) \neq y\}) \tag{2.2}$$

We would like to find a predictor h for which that error will be minimized. However, the learner does not know the data generating $\boldsymbol{\mathcal{D}}$. What the learner does have access to is the training data $\boldsymbol{\mathcal{S}}$. The definition of the *empirical* error remains the same as before, namely

$$L_{\boldsymbol{\mathcal{S}}}(h) = \frac{|\{i \;:\; h(x_i) \neq y_i,\; i = 1 \ldots m\}|}{m} \tag{2.3}$$

Bayes optimal predictor Given any probability distribution $\boldsymbol{\mathcal{D}}$ over $\boldsymbol{\mathcal{X}} \times \{0,\, 1\}$, the best label predicting function from $\boldsymbol{\mathcal{X}}$ to $\{0,\, 1\}$ will be

$$f_{\boldsymbol{\mathcal{D}}}(x) = \begin{cases} 1 & \text{if } \mathbb{P}[y = 1\,|\,x] \geq \frac{1}{2} \\ 0 & \text{otherwise} \end{cases} \tag{2.4}$$

The Bayes optimal predictor is optimal because no other classifier $g : \boldsymbol{\mathcal{X}} \to \{0,\, 1\}$ has a lower error. Namely, for every classifier g, $L_{\boldsymbol{\mathcal{D}}}(f_D) \leq L_{\boldsymbol{\mathcal{D}}}(g)$.
Unfortunately, since we do not know $\boldsymbol{\mathcal{D}}$ we cannot compute $\mathbb{P}[y = 1\,|\,x]$. Thus, we cannot utilize the Bayes optimal predictor $f_{\boldsymbol{\mathcal{D}}}$. We cannot hope that the lear+ algorithm will find a hypothesis whose error is smaller than the minimum possible error, that of the Bayes optimal predictor. Furthermore, once we make no prior assumptions about the data-generating distribution, no algorithm can be guaranteed to find a predictor that is as good as the Bayes optimal predictor. Instead, we require that the learning algorithm find a predictor whose error is not much larger than the best possible error of a predictor in some given benchmark hypothesis class. Of course, the strength of such a requirement depends on the choice of that hypothesis class.

Definition 2.2.1 (Agnostic PAC learnability)**.** *A hypothesis class $\boldsymbol{\mathcal{H}}$ is agnostic PAC learnable if there exist a function*

$$m_{\boldsymbol{\mathcal{H}}} : (0,1)^2 \to \mathbb{N}$$

and a learning algorithm such that

$$\forall\, \epsilon,\, \delta \in (0,1),\ \forall\, \boldsymbol{\mathcal{D}}\ \textit{over}\ \boldsymbol{\mathcal{Z}} \equiv \boldsymbol{\mathcal{X}} \times \boldsymbol{\mathcal{Y}}$$

when running the learning algorithm on $m \geq m_{\boldsymbol{\mathcal{H}}}(\epsilon,\, \delta)$ i.i.d. examples generated by $\boldsymbol{\mathcal{D}}$, the algorithm returns a hypothesis $h \in \boldsymbol{\mathcal{H}}$ such that

$$L_{\boldsymbol{\mathcal{D}}}(h) \leq \min_{h' \in \boldsymbol{\mathcal{H}}} L_{\boldsymbol{\mathcal{D}}}(h') + \epsilon$$

with a probability of at least $1 - \delta$ over the choice of the m training examples.

If the realizability assumption holds, agnostic PAC learning provides the same guarantee as PAC learning. On the other hand, when the realizability assumption does not hold, no learner can guarantee an arbitrarily small error. Nevertheless, under agnostic PAC learning, a learner can still declare success if its error is not much larger than the best error achievable by a predictor from the class $\boldsymbol{\mathcal{H}}$. In summary, while in PAC learning, the learner is required to achieve a small error in absolute terms, under the agnostic PAC learning model, it is required to achieve a small error relative to the best error achievable by the hypothesis class.

2.2.2 Generalized loss functions

We can now extend our model to cover various learning tasks.

- **Multiclass classification:** Classification tasks do not strictly have to be binary. Take, for example, the task of document classification: we wish to design a program that will be able to classify given documents according to topics. A learning algorithm for such a task will have access to examples of correctly classified documents. It should output a program that can take as input a new document and output a topic classification for that document. Here, the domain set is the set of all potential documents. Once again, we would usually represent documents by a set of features that could include counts of different keywords in the document and other possibly relevant features like the size of the document or its origin. The label set in this task will be the set of possible document topics.
- **Regression:** In this task, one wished to find some simple pattern in the data, which is a functional relationship between $\boldsymbol{\mathcal{X}}$ and $\boldsymbol{\mathcal{Y}}$ components of the data. For example, one wishes to find a linear function that best predicts a baby's birth weight based on ultrasound measures of his head circumference, abdominal circumference and femur length. Here, our domain set is some subset of $\mathbb{R}^3$ (the three ultrasound measurements) and the *target* set is the set of real numbers $\mathbb{R}$ (the weight in grams).

To accommodate a wide range of learning tasks, we need to generalize our formalism of the measure of success.

Given any set $\boldsymbol{\mathcal{H}}$ and domain $\boldsymbol{\mathcal{Z}}$, let l be any function from $\boldsymbol{\mathcal{H}} \times \boldsymbol{\mathcal{Z}}$ to the set of non-negative real numbers $\mathbb{R}_+$. We call such functions *loss functions*.

We now define the *risk function* to be the expected loss of a classifier $h \in \boldsymbol{\mathcal{H}}$ with respect to a probability distribution $\boldsymbol{\mathcal{D}}$ over $\boldsymbol{\mathcal{Z}}$:

$$L_{\boldsymbol{\mathcal{D}}}(h) = \mathbb{E}_{z \sim \boldsymbol{\mathcal{D}}}[l(h,\, z)] \tag{2.5}$$

or, the expectation of the loss of h over objects z sampled randomly according to $\boldsymbol{\mathcal{D}}$. Similarly we define the *empirical risk* to be the expected loss over a given sample $\boldsymbol{\mathcal{S}} = (z_1, \ldots, z_m) \in \boldsymbol{\mathcal{Z}}^m$, namely

$$L_{\boldsymbol{\mathcal{S}}}(h) = \frac{1}{m} \sum_{i=1}^{m} l(h,\, z_i) \tag{2.6}$$

Loss function examples: Some examples of loss functions are

- **0-1 loss:** This loss function is used in binary (or multiclass) classification problems.

$$l_{0\text{-}1}(h,\,(x,\,y)) = \begin{cases} 0 & \text{if } h(x) = y \\ 1 & \text{if } h(x) \neq y \end{cases} \tag{2.7}$$

  Note that for a random variable $\alpha \in \{0,\,1\}$ we have $\mathbb{E}_{\alpha\sim\boldsymbol{\mathcal{D}}}[\alpha] = \mathbb{P}_{\alpha\sim\boldsymbol{\mathcal{D}}}[\alpha = 1]$, thus the definition of $L_{\boldsymbol{\mathcal{D}}}(h)$ given inEquation 2.2 and Equation 2.5 coincide.

- **Squared loss:** This loss is used in regression problems and typically penalizes few large errors.

$$l_{\text{sq}}(h,\,(x,\,y)) = (h(x) - y)^2 \tag{2.8}$$

- **Absolute value loss:** On the other hand, this loss penalizes many small errors.

$$l_{\text{abs}}(h,\,(x,\,y)) = |h(x) - y| \tag{2.9}$$

To summarize, we now formally re-define agnostic PAC learnability using the generalized loss functions concept we introduced.

Definition 2.2.2 (Agnostic PAC learnability for generalized loss functions)**.** *A hypothesis class $\boldsymbol{\mathcal{H}}$ is agnostic PAC learnable with respect to a set $\boldsymbol{\mathcal{Z}}$ and a loss function $l : \boldsymbol{\mathcal{H}} \times \boldsymbol{\mathcal{Z}} \to \mathbb{R}_+$ if there exist a function*

$$m_{\boldsymbol{\mathcal{H}}} : (0,1)^2 \to \mathbb{N}$$

and a learning algorithm such that

$$\forall\,\epsilon,\,\delta \in (0,1),\ \forall\,\boldsymbol{\mathcal{D}}\ \textit{over}\ \boldsymbol{\mathcal{Z}} \equiv \boldsymbol{\mathcal{X}} \times \boldsymbol{\mathcal{Y}}$$

when running the learning algorithm on $m \geq m_{\boldsymbol{\mathcal{H}}}(\epsilon,\,\delta)$ i.i.d. examples generated by $\boldsymbol{\mathcal{D}}$, the algorithm returns a hypothesis $h \in \boldsymbol{\mathcal{H}}$ such that

$$L_{\boldsymbol{\mathcal{D}}}(h) \leq \min_{h' \in \boldsymbol{\mathcal{H}}} L_{\boldsymbol{\mathcal{D}}}(h') + \epsilon \qquad\qquad \textit{where}\ \ L_{\boldsymbol{\mathcal{D}}}(h) = \mathbb{E}_{z\sim\boldsymbol{\mathcal{D}}}[l(h,\,z)]$$

with a probability of at least $1 - \delta$ over the choice of the m training examples.

Chapter 3

Learning via Uniform Convergence

The first formal learning model that we discussed was the PAC model. We have seen that any finite hypothesis class is PAC learnable under the realizability assumption. In this chapter, we will develop a general tool, that is *uniform convergence*, and apply it to show that any finite class is learnable in the agnostic PAC model with generalized loss functions as long as the range loss function is bounded.

3.1 Uniform convergence

The idea behind the learning condition discussed in this chapter is straightforward. Recall that, given a hypothesis class $\boldsymbol{\mathcal{H}}$, the ERM learning paradigm works as follows: upon receiving a training sample $\boldsymbol{\mathcal{S}}$, the learner evaluates the error of each $h \in \boldsymbol{\mathcal{H}}$ on the given sample and returns a member of $\boldsymbol{\mathcal{H}}$ that minimizes the empirical error. The hope is that an h that minimizes the empirical risk with respect to $\boldsymbol{\mathcal{S}}$ is a risk minimizer with respect to the true data probability distribution $\boldsymbol{\mathcal{D}}$ as well. For that, it suffices to ensure that the empirical risks of all members of $\boldsymbol{\mathcal{H}}$ are good approximations of their true risk. We thus need that, uniformly over all hypothesis $h \in \boldsymbol{\mathcal{H}}$, the empirical risk will be close to their true risk.

Definition 3.1.1 (ϵ-representative set)**.** *A training set $\boldsymbol{\mathcal{S}}$ is called ϵ-representative with respect to a domain $\boldsymbol{\mathcal{Z}}$, a hypothesis class $\boldsymbol{\mathcal{H}}$, a loss function l and a data-generating distribution $\boldsymbol{\mathcal{D}}$ if*

$$\forall\ h \in \boldsymbol{\mathcal{H}},\ \ |L_{\boldsymbol{\mathcal{S}}}(h) - L_{\boldsymbol{\mathcal{D}}}(h)| \leq \epsilon$$

The next theorem states that whenever the sample is $(\epsilon/2)$-representative, the ERM learning rule is guaranteed to return a good hypothesis.

Theorem 3.1.1. *Assume that a training set $\boldsymbol{\mathcal{S}}$ is $\frac{\epsilon}{2}$-representative with respect to a domain $\boldsymbol{\mathcal{Z}}$, a hypothesis class $\boldsymbol{\mathcal{H}}$, a loss function l and a data-generating distribution $\boldsymbol{\mathcal{D}}$. Then, any output of $ERM_{\boldsymbol{\mathcal{H}}}(\boldsymbol{\mathcal{S}})$, namely any $h_{\boldsymbol{\mathcal{S}}} \in \arg\min_{h\in\boldsymbol{\mathcal{H}}} L_{\boldsymbol{\mathcal{S}}}(h)$, satisfies*

$$L_{\boldsymbol{\mathcal{D}}}(h_{\boldsymbol{\mathcal{S}}}) \leq \min_{h\in\boldsymbol{\mathcal{H}}} L_{\boldsymbol{\mathcal{D}}}(h) + \epsilon$$

that is the definition of PAC learnability for the hypothesis class $\boldsymbol{\mathcal{H}}$.

Proof. By using the ϵ-representative definition we have

$$\forall\ \ h \in \boldsymbol{\mathcal{H}},\ \ |L_{\boldsymbol{\mathcal{S}}}(h_{\boldsymbol{\mathcal{S}}}) - L_{\boldsymbol{\mathcal{D}}}(h_{\boldsymbol{\mathcal{S}}})| \leq \frac{\epsilon}{2} \Longrightarrow L_{\boldsymbol{\mathcal{D}}}(h_{\boldsymbol{\mathcal{S}}}) \leq L_{\boldsymbol{\mathcal{S}}}(h_{\boldsymbol{\mathcal{S}}}) + \frac{\epsilon}{2}$$

Using the definition of $h_{\boldsymbol{\mathcal{S}}}$ as an ERM predictor we find $\forall\ h \in \boldsymbol{\mathcal{H}},\ \ L_{\boldsymbol{\mathcal{S}}}(h_{\boldsymbol{\mathcal{S}}}) \leq L_{\boldsymbol{\mathcal{S}}}(h)$. Finally, by ϵ-representative definition again, we get the last inequality

$$\forall\ \ h \in \boldsymbol{\mathcal{H}},\ \ |L_{\boldsymbol{\mathcal{S}}}(h) - L_{\boldsymbol{\mathcal{D}}}(h)| \leq \frac{\epsilon}{2} \Longrightarrow L_{\boldsymbol{\mathcal{S}}}(h) \leq L_{\boldsymbol{\mathcal{D}}}(h) + \frac{\epsilon}{2}$$

Thus, for every $h \in \boldsymbol{\mathcal{H}}$,

$$L_{\boldsymbol{\mathcal{D}}}(h_{\boldsymbol{\mathcal{S}}}) \leq L_{\boldsymbol{\mathcal{S}}}(h_{\boldsymbol{\mathcal{S}}}) + \frac{\epsilon}{2} \leq L_{\boldsymbol{\mathcal{S}}}(h) + \frac{\epsilon}{2} \leq (L_{\boldsymbol{\mathcal{D}}}(h) + \frac{\epsilon}{2}) + \frac{\epsilon}{2}$$

$$\implies L_{\boldsymbol{\mathcal{D}}}(h_{\boldsymbol{\mathcal{S}}}) \leq L_{\boldsymbol{\mathcal{D}}}(h) + \epsilon$$

□

The theorem implies that to ensure that the ERM rule is an agnostic PAC learner, it suffices to show that it will be an ϵ-representative training set with a probability of at least $1 - \delta$. The uniform convergence condition formalizes this requirement.

Definition 3.1.2 (Uniform convergence)**.** *We say that a hypothesis class $\boldsymbol{\mathcal{H}}$ has the uniform convergence property with respect to a domain $\boldsymbol{\mathcal{Z}}$ and a loss function l if there exists a function*

$$m_{\boldsymbol{\mathcal{H}}}^{U.C.} : (0,\, 1)^2 \to \mathbb{N}$$

such that

$$\forall\, \epsilon,\, \delta \in (0,1),\ \forall\, \boldsymbol{\mathcal{D}}\ over\, \boldsymbol{\mathcal{Z}} \equiv \boldsymbol{\mathcal{X}} \times \boldsymbol{\mathcal{Y}}$$

if $\boldsymbol{\mathcal{S}}$ is a sample of $m \geq m_{\boldsymbol{\mathcal{H}}}^{U.C.}(\epsilon,\, \delta)$ examples drawn i.i.d. according to $\boldsymbol{\mathcal{D}}$ then $\boldsymbol{\mathcal{S}}$ is ϵ-representative with probability of at least $1 - \delta$.

Similar to the definition of sample complexity for PAC learning, the function $m_{\boldsymbol{\mathcal{H}}}^{\mathrm{U.C.}}$ measures the minimal sample complexity of obtaining the uniform convergence property, namely how many examples we need to ensure that the sample would be ϵ-representative with a probability of at least $1 - \delta$. The term *uniform* refers to having a fixed sample size that works for all members of $\boldsymbol{\mathcal{H}}$ and all possible probability distributions over the domain.

The following corollary follows directly from Theorem 3.1.1 and the definition of uniform convergence (Definition 3.1.2).

Corollary 3.1.1.1. *If a hypothesis class $\boldsymbol{\mathcal{H}}$ has the uniform convergence property with a function $m_{\boldsymbol{\mathcal{H}}}^{U.C.}$ then*

1. *$\boldsymbol{\mathcal{H}}$ is agnostically PAC learnable with sample complexity $m_{\boldsymbol{\mathcal{H}}}(\epsilon,\, \delta) \leq m_{\boldsymbol{\mathcal{H}}}^{U.C.}(\frac{\epsilon}{2},\, \delta)$*
2. *the $ERM_{\boldsymbol{\mathcal{H}}}$ paradigm is a successful agnostic PAC learner for $\boldsymbol{\mathcal{H}}$*

Note that the "less or equal" sign in the first point of the corollary is due to the fact that uniform convergence has stronger requirements than PAC learnability. Precisely, in the uniform convergence context, we require the validity of the assumptions for all $h \in \boldsymbol{\mathcal{H}}$, while PAC learnability requires only the optimal hypothesis. In fact, $m_{\boldsymbol{\mathcal{H}}}^{\mathrm{U.C.}}$ is a sufficient condition for PAC learnability.

3.2 Finite hypothesis classes

Proposition 3.2.1. *Let $\boldsymbol{\mathcal{H}}$ be a finite hypothesis class, let $\boldsymbol{\mathcal{Z}}$ be a domain and let $l : \boldsymbol{\mathcal{H}} \times \boldsymbol{\mathcal{Z}} \to [0,\, 1]$ be a loss function. Then*

- *$\boldsymbol{\mathcal{H}}$ enjoys the uniform convergence property with sample complexity*

$$m_{\boldsymbol{\mathcal{H}}}^{U.C.}(\epsilon,\, \delta) \leq \left\lceil \frac{\log\left(2\frac{|\boldsymbol{\mathcal{H}}|}{\delta}\right)}{2\epsilon^2} \right\rceil$$

- *$\boldsymbol{\mathcal{H}}$ is agnostically PAC learnable using the ERM algorithm with sample complexity*

$$m_{\boldsymbol{\mathcal{H}}}(\epsilon,\, \delta) \leq m_{\boldsymbol{\mathcal{H}}}^{U.C.}(\epsilon,\, \delta) \leq \left\lceil \frac{\log\left(2\frac{|\boldsymbol{\mathcal{H}}|}{\delta}\right)}{2\epsilon^2} \right\rceil$$

Thus, every finite $\boldsymbol{\mathcal{H}}$ has the uniform convergence property if the sample complexity is low enough. It follows that we can agnostically PAC learn every finite $\boldsymbol{\mathcal{H}}$, provided we have enough data.

3.2.1 Discretization trick

Consider a hypothesis class that is parametrized by d parameters. For example, let $\boldsymbol{\mathcal{X}} = \mathbb{R}$, $\boldsymbol{\mathcal{Y}} = \{-1, +1\}$ and $\boldsymbol{\mathcal{H}}$ to be all the functions of the form $h_\theta = \text{sign}(x - \theta)$. Each hypothesis is parametrized by one parameter $\theta \in \mathbb{R}$ and returns $+1$ for all instances larger than θ and returns -1 for instances smaller than θ. This is a hypothesis class of infinite size. However, if we learn this hypothesis class in practice using a computer, we will probably maintain real numbers using floating point representation. It follows that our hypothesis class is parametrized by the set of scalars that can be represented using 64 bits floating point numbers. There are at most 2^{64} such numbers. Hence the actual size of our hypothesis class is at most 2^{64}. More generally, if d parameters parametrize our hypothesis class, its size is at most 2^{64d}. Applying Proposition 3.2.1, we obtain that the sample complexity of such classes is

$$m_{\boldsymbol{\mathcal{H}}}(\epsilon, \delta) \leq m_{\boldsymbol{\mathcal{H}}}^{\text{U.C.}}(\epsilon, \delta) \leq \frac{2}{\epsilon^2} \log\left(2\frac{2^{64d}}{\delta}\right) \tag{3.1}$$

and we see that this upper bound on the sample complexity depends on the specific representation of real numbers used by our machine. That being said, the discretization trick can be used to get a rough estimate of the sample complexity in many practical situations.

Chapter 4

The Bias-Complexity Tradeoff

We have already seen that the training data can mislead the learner into overfitting. Therefore, we have restricted the search space to some hypothesis class $\mathcal{H}$ to overcome this problem. Such hypothesis class can be seen as reflecting the learner's prior knowledge about the task.

Is such prior knowledge really necessary for the success of learning? Maybe there exists some universal learner without prior knowledge about a particular task that is ready to be challenged by any task.

A specific learning task is defined by an unknown distribution $\mathcal{D}$ over $Z \equiv \mathcal{X} \times \mathcal{Y}$, and the learner's goal is to find a predictor $h : \mathcal{X} \to \mathcal{Y}$ whose risk $L_{\mathcal{D}}(h)$ is small enough. The question is, therefore, whether there exists a learning algorithm A and a training set size $m \in \mathbb{N}$ such that for every distribution $\mathcal{D}$ if A receives m i.i.d. examples from $\mathcal{D}$ then there is a high chance it returns a predictor h that has low risk.

The *no free-lunch* theorem states that no such universal learner exists. The theorem states that for binary classification prediction tasks, for every learner, there exists a distribution on which it fails. In other words, the theorem states that no learner can succeed on all learnable tasks: every learner has tasks on which it fails while other learners succeed.

Therefore, when approaching a particular learning problem defined by some distribution $\mathcal{D}$ we should have prior knowledge of $\mathcal{D}$. One type of such prior knowledge is that $\mathcal{D}$ comes from some specific parametric family of distributions. Another type of prior knowledge on $\mathcal{D}$, which we assumed when defining the PAC learning model, is that there exists a predictor h in some predefined hypothesis class $\mathcal{H}$ such that $L_{\mathcal{D}}(h) = 0$. Finally, a softer prior knowledge on $\mathcal{D}$ assumes that $\min_{h \in \mathcal{H}} L_{\mathcal{D}}(h)$ is small. In a sense, this weaker assumption is a prerequisite for using the agnostic PAC learning model, in which we require that the risk of the output hypothesis will not be much larger than $\min_{h \in \mathcal{H}} L_{\mathcal{D}}(h)$.

We will study the benefits and pitfalls of using a hypothesis class to formalize prior knowledge: we will decompose the error of an ERM algorithm over a class $\mathcal{H}$ into two components. The first component, called the *approximation error*, reflects the quality of our prior knowledge measured by the minimal risk of a hypothesis in our hypothesis class. The second component, called the *estimation error*, is the error due to overfitting and depends on the size or the complexity of the hypothesis class $\mathcal{H}$. These two terms imply a tradeoff between choosing a more complex $\mathcal{H}$ (which can decrease the bias, that is, the approximation error, but increase the risk of overfitting) or a less complex $\mathcal{H}$ (which might increase the bias but decreases the potential overfitting).

4.1 No free-lunch theorem

Let us prove there is no universal learner by showing that no learner can succeed on all tasks.

Theorem 4.1.1 (No free-lunch)**.** *Let A be any learning algorithm for the binary classification task with respect to the 0-1 loss over a domain $\mathcal{X}$. Let m be any number smaller than $\mathcal{X}/2$ representing a training set size. Then, there exists a distribution $\mathcal{D}$ over $\mathcal{X} \times \{0, 1\}$ such that*

1. *there exists a function $f : \mathcal{X} \to \{0, 1\}$ with $L_{\mathcal{D}}(f) = 0$ (existence of a perfect solution)*
2. *we have that $L_{\mathcal{D}}(A(\mathcal{S})) \geq \frac{1}{8}$ with probability of at least $\frac{1}{7}$ over the choice of $\mathcal{S} \sim \mathcal{D}^m$*

Proof. The idea is that our training set is smaller than half of the domain. Thus we have no information on what happens on the other half. So we need to show that there exists a target function f that works on the other half in a way that contradicts our estimated labels. □

This theorem states that for every learner, there exists a task on which it fails even though another learner can successfully learn that task. However, how does it relate to the need for prior knowledge? Let us consider an ERM predictor over the hypothesis class $\boldsymbol{\mathcal{H}}$ of all the functions $f : \boldsymbol{\mathcal{X}} \to \{0, 1\}$. This class represents a lack of prior knowledge. Every possible function from the domain to the label set is considered a good candidate. According to the *no free-lunch* theorem, any algorithm that chooses its output from hypotheses in $\boldsymbol{\mathcal{H}}$ (and in particular the ERM predictor) will fail on some learning task. Therefore this class is not PAC learnable, as formalized in the following corollary.

Corollary 4.1.1.1. *Let $\boldsymbol{\mathcal{X}}$ be an infinite domain set and let $\boldsymbol{\mathcal{H}}$ be the set of all functions from $\boldsymbol{\mathcal{X}}$ to $\{0, 1\}$. Then $\boldsymbol{\mathcal{H}}$ is not PAC learnable.*

Proof. We proceed by contradiction. Let us assume $\boldsymbol{\mathcal{H}}$ is PAC learnable and choose $\epsilon < \frac{1}{8}$, $\delta < \frac{1}{7}$. By definitions of PAC learnability, it exists an algorithm A such that for every distribution $\boldsymbol{\mathcal{D}}$, if the realizability assumption holds, when running the algorithm on m i.i.d. examples generated by $\boldsymbol{\mathcal{D}}$, the algorithm returns a hypothesis $h \in \boldsymbol{\mathcal{H}}$ such that $L_{\boldsymbol{\mathcal{D}}}(A(\boldsymbol{\mathcal{S}})) \leq \epsilon$. Let us now apply the no free-lunch theorem to A: since the size of $\boldsymbol{\mathcal{X}}$ is not finite, every number is less than half the size of $\boldsymbol{\mathcal{X}}$; thus, for every learning algorithm (including A), there exists a distribution $\boldsymbol{\mathcal{D}}$ for which $L_{\boldsymbol{\mathcal{D}}}(A(\boldsymbol{\mathcal{S}})) \geq \frac{1}{8} > \epsilon$. As a result, we get a sum of probabilities that is bigger than 1, which is, of course, absurd. □

We can escape the hazards foreseen by the *no free-lunch* theorem by using our prior knowledge about a specific learning algorithm to avoid the distributions that will cause us to fail when trying to learn that task. Such prior knowledge can be expressed by restricting the hypothesis class.

On the one hand, we want to choose the class $\boldsymbol{\mathcal{H}}$ that includes the hypothesis that has no error at all (in the PAC setting) or at least that the smallest error achievable by a hypothesis from this class is rather small (in the agnostic PAC setting). On the other hand, we have just seen that we cannot simply choose the richest class. In other words:

- **Choosing a small $\boldsymbol{\mathcal{H}}$** leads to little approximation capabilities ($L_{\boldsymbol{\mathcal{S}}}$ is large) but rather good generalization properies ($L_{\boldsymbol{\mathcal{D}}} \approx L_{\boldsymbol{\mathcal{S}}}$).
- **Choosing a large $\boldsymbol{\mathcal{H}}$** leads to good approximation capabilities ($L_{\boldsymbol{\mathcal{S}}}$ is small) but enhances the risk of overfitting.

4.2 Error decomposition

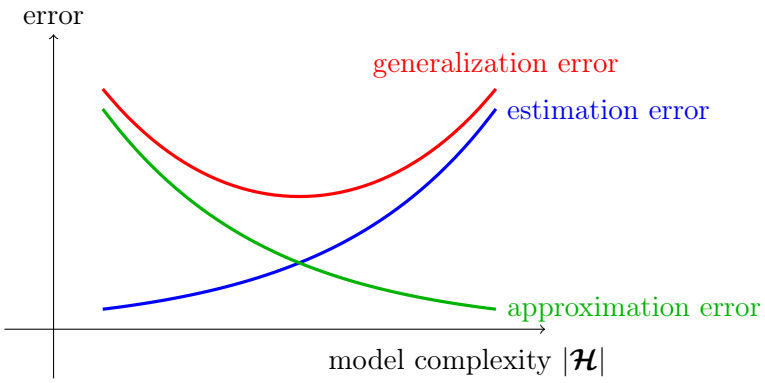
To better understand how to choose the appropriate hypothesis class properly, we decompose the error of an $\text{ERM}_{\boldsymbol{\mathcal{H}}}$ hypothesis. We can write

$$L_{\boldsymbol{\mathcal{D}}}(h_{\boldsymbol{\mathcal{S}}}) = \epsilon_{\text{app}} + \epsilon_{\text{est}} \tag{4.1}$$

where the two terms are respectively

- **the *approximation error*:** $\epsilon_{\text{app}} = \min_{h \in \boldsymbol{\mathcal{H}}} L_{\boldsymbol{\mathcal{D}}}(h)$
  - It is the minimum risk achievable by a predictor in the hypothesis class.
  - It measures how much risk we have because we restrict ourselves to a specific class, namely how much *inductive bias* we have.
  - It does not depend on the sample size.
  - It is determined by the chosen hypothesis class (usually a larger $\boldsymbol{\mathcal{H}}$ leads to a smaller ϵ_{app}).
  - It is zero if the realizability assumption holds; otherwise, it is bounded by the Bayes optimal predictor (can be significantly larger than ϵ_{Bayes}).

- **the *estimation error*:** $\epsilon_{\text{est}} = L_{\mathcal{D}}(h_{\mathcal{S}}) - \epsilon_{\text{app}}$
  - It is the difference between the approximation error and the ERM predictor error.
  - It exists because the training error is only an estimate of the true error. Thus the predictor minimizing the empirical risk is only an estimate of the predictor minimizing the true risk.
  - It depends on the training set $\mathcal{S}$ and the hypothesis class $\mathcal{H}$ (we have seen that for a finite hypothesis class, it increases logarithmically with $|\mathcal{H}|$ and decreases with m).



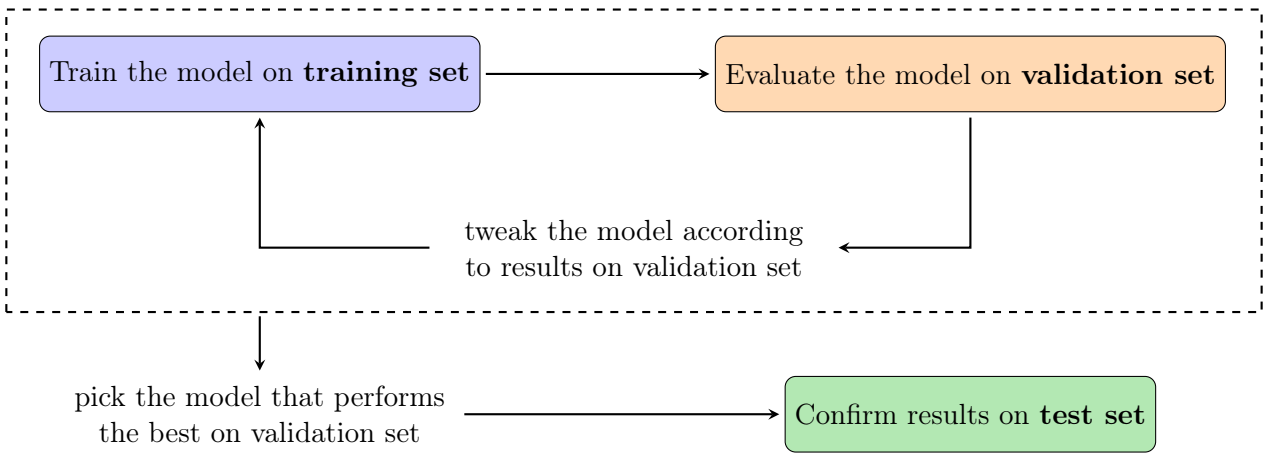
Since our goal is to minimize the *generalization error* (i.e., the total risk), we face a tradeoff called the *bias-complexity tradeoff*. On the one hand, choosing $\mathcal{H}$ to be a very rich class decreases the approximation error but at the same time might increase the estimation error, as a rich $\mathcal{H}$ might lead to *overfitting*. On the other hand, choosing $\mathcal{H}$ to be a very small set reduces the estimation error but might increase the approximation error or, in other words, might lead to *underfitting*.

An excellent choice for $\mathcal{H}$ is the class that contains only the Bayes optimal predictor. However, as we have seen before, it depends on the underlying distribution $\mathcal{D}$, which we do not know.

4.3 Training, validation and test sets

In order to minimize the *generalization error*, we might build an iterative *train-validate-test* strategy to maximize our chances of finding a good hypothesis class.

Let us split the available data into three subsets: the *training* set, *validation* set and *test* set. The training set is necessary to compute $L_{\mathcal{S}}(h)$ and pick the $\text{ERM}_{\mathcal{H}}$ predictor (i.e., the one that minimizes the empirical error). The test set instead is disjoint from the training set. It gives a more reliable estimation of $L_{\mathcal{D}}(h)$ (note that it will always be an approximation) as the examples in the test set are entirely different from the ones used for picking the predictor h. Lastly, the validation set is typically used for selecting the hyper-parameters of the learning algorithms through iterative procedures that include the optimization of such hyper-parameters based on the validation error.



Chapter 5

PAC Learnability for infinite-size classes

In the previous chapter, we decomposed the error of the $\text{ERM}_{\mathcal{H}}$ rule into approximation error and estimation error. The approximation error depends on the fit of our prior knowledge to the underlying unknown distribution $\mathcal{D}$. In contrast, the definition of PAC learnability requires that the estimation error would be bounded uniformly over all distributions.

Our goal is to determine which classes $\mathcal{H}$ are PAC learnable and to characterize exactly the sample complexity of learning a given hypothesis class. For example, we have seen that finite classes are learnable but that the class of all functions over an infinite-sized domain is not.

5.1 The VC-dimension

We have already seen that finite classes are learnable and that the sample complexity of a hypothesis class is upper-bounded by the logarithm of its size. However, to show that the size of the hypothesis class is not the right characterization of its sample complexity, we first present a simple example of an infinite-size class that is learnable.

Let $\mathcal{H}$ be the set of threshold functions (Heaviside functions, step functions) over the real line, namely $\mathcal{H} = \{h_a : a \in \mathbb{R}\}$ where $h_a : \mathbb{R} \to \{0, 1\}$ is a function such that $h_a(x) = \mathbb{I}_{[x<a]}$. Clearly, $\mathcal{H}$ is of infinite size as $a \in \mathbb{R}$. Nevertheless, a demonstrated lemma shows that $\mathcal{H}$ is learnable in the PAC model using the ERM algorithm with a sample complexity of

$$m_{\mathcal{H}}(\epsilon, \delta) \leq \left\lceil \frac{\log\left(\frac{2}{\delta}\right)}{\epsilon} \right\rceil$$

Therefore, while the finiteness of $\mathcal{H}$ is a sufficient condition for learnability, it is not a necessary condition. As we will show, a property called the *VC-dimension* of a hypothesis class gives the correct characterization of its learnability.

Definition 5.1.1 (Restriction of $\mathcal{H}$ to $\mathcal{C}$)**.** *Let $\mathcal{H}$ be a class of functions from $\mathcal{X}$ to $\{0, 1\}$ and let $\mathcal{C} = \{c_1, \ldots, c_m\} \subset \mathcal{X}$. The restriction of $\mathcal{H}$ to $\mathcal{C}$ is the set of functions from $\mathcal{C}$ to $\{0, 1\}$ that can be derived from $\mathcal{H}$. That is*

$$\mathcal{H}_{\mathcal{C}} = \{(h(c_1), \ldots, h(c_m)) : h \in \mathcal{H}\}$$

where we represent each function from $\mathcal{C}$ to $\{0, 1\}$ as a vector in $\{0, 1\}^{|\mathcal{C}|}$.

If the restriction of $\mathcal{H}$ to $\mathcal{C}$ is the set of *all* functions from $\mathcal{C}$ to $\{0, 1\}$ then we say that $\mathcal{H}$ *shatters* the set $\mathcal{C}$.

Definition 5.1.2 (Shattering)**.** *A hypothesis class $\mathcal{H}$ shatters a finite set $\mathcal{C} \subset \mathcal{X}$ if the restriction of $\mathcal{H}$ to $\mathcal{C}$ is the set of all functions from $\mathcal{C}$ to $\{0, 1\}$. That is $|\mathcal{H}_{\mathcal{C}}| = 2^{|\mathcal{C}|}$.*

Theorem 5.1.1. *Let $\mathcal{H}$ be a hypothesis class of functions from $\mathcal{X}$ to $\{0, 1\}$. Let m be a training set size. Assume that there exists a set $\mathcal{C} \subset \mathcal{X}$ of size $2m$ that is shattered by $\mathcal{H}$. Then, for any learning algorithm A there exist a distribution $\mathcal{D}$ over $\mathcal{X} \times \{0, 1\}$ and a predictor $h \in \mathcal{H}$ such that $L_{\mathcal{D}}(h) = 0$ but $L_D(A(\mathcal{S})) \geq \frac{1}{8}$ with probability of at least $\frac{1}{7}$ over the choice of $\mathcal{S} \sim \mathcal{D}^m$.*

Proof. The idea is that by definition of shattering, if $\mathcal{H}$ shatters $\mathcal{C}$, all functions from $\mathcal{C}$ to $\{0, 1\}$ can be selected, and we fall back into the *no free-lunch* situation. □

Theorem 5.1.1 tells us that if $\mathcal{H}$ shatters some set $\mathcal{C}$ of size $2m$ then we cannot learn $\mathcal{H}$ using just m samples. Intuitively, suppose a set HH shatters $\mathcal{C}$, and we receive a sample containing half the instances of $\mathcal{C}$. In that case, the labels of these instances give us no information about the labels of the rest of the instances of $\mathcal{C}$.

This leads us directly to the definition of the *VC-dimension.*

Definition 5.1.3 (VC-dimension)**.** *The VC-dimension of a hypothesis class $\mathcal{H}$ denoted VCdim($\mathcal{H}$) is the maximal size of a set $\mathcal{C} \subset \mathcal{X}$ that can be shattered by $\mathcal{H}$. If $\mathcal{H}$ can shatter sets of arbitrarily large size, we say that $\mathcal{H}$ has infinite VC-dimension.*

A direct consequence of Theorem 5.1.1 is thus

Theorem 5.1.2. *Let $\mathcal{H}$ be a class of infinite VC-dimension. Then, $\mathcal{H}$ is not PAC learnable.*

Proof. Since $\mathcal{H}$ has infinite VC-dimension, for any training set size m, there exists a shattered set of size $2m$, and the claim follows by Theorem 5.1.1. □

5.2 Examples of VC-dimension computation

Let us now compute the VC-dimension of a few hypothesis classes. To show that $\text{VCdim}(\mathcal{H}) = d$ we need to show that

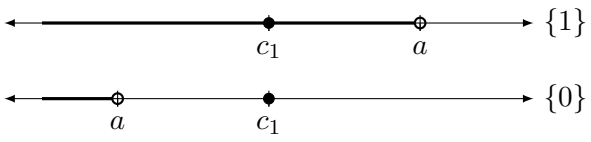
1. there exists a set $\mathcal{C}$ of size d that is shattered by $\mathcal{H}$
2. every set $\mathcal{C}$ of size $d + 1$ is not shattered by $\mathcal{H}$

5.2.1 Threshold function

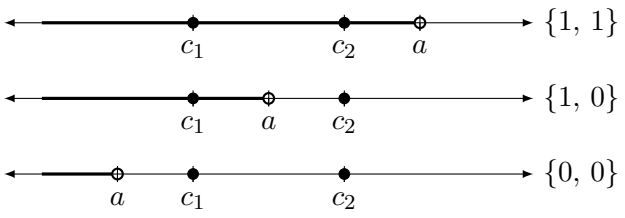
Let $\mathcal{H}$ be the class of threshold functions over $\mathbb{R}$: $\mathcal{H} = \{h_a : a \in \mathbb{R}\}$ where $h_a : \mathbb{R} \to \{0, 1\}$ is

$$h_a(x) = \begin{cases} 1 & \text{if } x < a \\ 0 & \text{if } x \geq a \end{cases}$$

Let us consider an arbitrary set $\mathcal{C} = \{c_1\}$ of size $d = 1$.



The figure above shows us that $\mathcal{C}$ is shattered by $\mathcal{H}$, as different choices of $h_a \in \mathcal{H}$ cover all possible outcomes. Thus, $\text{VCdim}(\mathcal{H}) \geq 1$. Let us consider now an arbitrary set $\mathcal{C} = \{c_1, c_2\}$, where $c_1 < c_2$, of size $d = 2$.



We notice immediately that the outcome $\{0, 1\}$ is impossible to achieve, as there is no choice for $h_a \in \mathcal{H}$ that satisfies the desired inequalities. We thus conclude that $\mathcal{H}$ does not shatter $\mathcal{C}$ and $\text{VCdim}(\mathcal{H}) < 2$. We then have $\text{VCdim}(\mathcal{H}) = 1$.

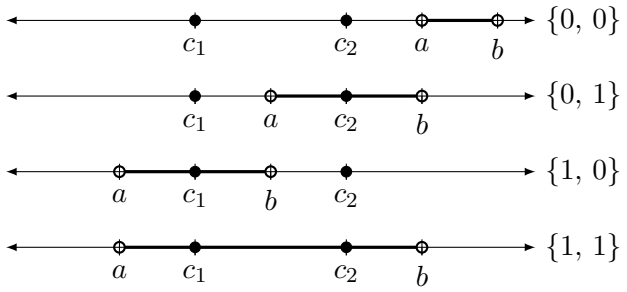
5.2.2 Intervals

Let $\boldsymbol{\mathcal{H}}$ be the class of intervals over $\mathbb{R}$: $\boldsymbol{\mathcal{H}} = \{h_{a,b} : a, b \in \mathbb{R}, a < b\}$ where $h_{a,b} : \mathbb{R} \to \{0, 1\}$ is

$$h_{a,b}(x) = \mathbb{I}_{[x \in (a,b)]}$$

Let us consider an arbitrary set $\boldsymbol{\mathcal{C}} = \{c_1\}$ of size $d = 1$. Of course, we can choose a and b such that c_1 is inside the interval (outcome 1) or outside the interval (outcome 0). Thus $\boldsymbol{\mathcal{H}}$ shatters $\boldsymbol{\mathcal{C}}$.

Let us consider now an arbitrary set $\boldsymbol{\mathcal{C}} = \{c_1, c_2\}$, where $c_1 < c_2$, of size $d = 2$.



The figure above shows us that $\boldsymbol{\mathcal{C}}$ is shattered by $\boldsymbol{\mathcal{H}}$, as all possible outcomes are covered by different choices of $h_{a,b} \in \boldsymbol{\mathcal{H}}$. Thus $\text{VCdim}(\boldsymbol{\mathcal{H}}) \geq 2$.

Finally, let us consider an arbitrary set $\boldsymbol{\mathcal{C}} = \{c_1, c_2, c_3\}$, where $c_1 < c_2 < c_3$, of size $d = 3$. It is clear that the outcomes $\{1, 0, 0\}$, $\{0, 1, 0\}$ and $\{0, 0, 1\}$ are easly achievable, as well as $\{1, 1, 1\}$ and $\{0, 0, 0\}$. Moreover, $\{1, 1, 0\}$ and $\{0, 1, 1\}$ are also possible leaving out of the interval c_3 and c_1 respectively. However, $\{1, 0, 1\}$ is not an achievable outcome, as it is impossible to leave out of the interval the middle point c_2 while including both the outer points. Thus we conclude that $\boldsymbol{\mathcal{H}}$ does not shatter $\boldsymbol{\mathcal{C}}$ and $\text{VCdim}(\boldsymbol{\mathcal{H}}) < 3$. We then have $\text{VCdim}(\boldsymbol{\mathcal{H}}) = 2$.

5.2.3 Finite classes

Let $\boldsymbol{\mathcal{H}}$ be a finite class. Then, clearly, for any set $\boldsymbol{\mathcal{C}}$ we have $|\boldsymbol{\mathcal{H}}_{\boldsymbol{\mathcal{C}}}| < |\boldsymbol{\mathcal{H}}|$ and thus $\boldsymbol{\mathcal{C}}$ cannot be shattered if $|\boldsymbol{\mathcal{H}}| < 2^{|\boldsymbol{\mathcal{C}}|}$. This implies that $\text{VCdim}(\boldsymbol{\mathcal{H}}) \leq \log_2(|\boldsymbol{\mathcal{H}}|)$. This shows that the PAC learnability of finite classes follows from the more general statement of PAC learnability of classes with finite VC-dimension.

5.2.4 Number of parameters

In the previous examples, the VC-dimension happened to equal the number of parameters defining the hypothesis class. While this is often the case, it is not always true.

Consider, for example, the domain $\boldsymbol{\mathcal{X}} = \mathbb{R}$ and the hypothesis class $\boldsymbol{\mathcal{H}} = \{h_\theta : \theta \in \mathbb{R}\}$ where $h_\theta : \mathbb{R} \to \{0, 1\}$ is defined by $h_\theta(x) = \lceil 0.5 \sin(\theta x) \rceil$. It is possible to prove that $\text{VCdim}(\boldsymbol{\mathcal{H}}) = \infty$. Namely, for every d, one can find d points that are shattered by $\boldsymbol{\mathcal{H}}$.

5.3 The fundamental theorem of PAC learning

Theorem 5.3.1 (Fundamental theorem of PAC learning)**.** *Let $\mathcal{H}$ be a hypothesis class of functions from a domain $\mathcal{X}$ to $\{0, 1\}$ and let the loss function be the 0-1 loss. Then, the following are equivalent:*

1. *$\mathcal{H}$ has the uniform convergence property*
2. *Any ERM rule is a successful agnostic PAC learner for $\mathcal{H}$*
3. *$\mathcal{H}$ is agnostic PAC learnable*
4. *$\mathcal{H}$ is PAC learnable*
5. *Any ERM rule is a successful PAC learner for $\mathcal{H}$*
6. *$\mathcal{H}$ has a finite VC-dimension*

A similar result holds for other learning problems, such as regression with the absolute or squared loss. However, the theorem does not hold for all learning tasks. In particular, learnability is sometimes possible even though the uniform convergence property does not hold. Furthermore, the ERM rule fails in some situations, but learnability is possible with other learning rules.

Proof. We have already seen $1 \to 2$ in Chapter 3. The implications $2 \to 3$, $3 \to 4$ and $2 \to 5$ are trivial. The implications $4 \to 6$ and $5 \to 6$ follow from the NFL theorem (Theorem 4.1.1).

The tough part is to show that $6 \to 1$ and the proof can be divided into two main parts:

1. If $\text{VCdim}(\mathcal{H}) = d$ then even though $\mathcal{H}$ might be infinite, when restricting it to a finite set $\mathcal{C} \subset \mathcal{X}$, its effective size $|\mathcal{H}_\mathcal{C}|$ is only $O(|\mathcal{C}|^d)$. Namely, the size of $\mathcal{H}_\mathcal{C}$ grows polynomially rather than exponentially with $|\mathcal{C}|$.
2. We have seen that finite hypothesis classes enjoy the uniform convergence property. It is possible to generalize this result and show that uniform convergence holds whenever the hypothesis class has a small effective size, namely classes for which $|\mathcal{H}_\mathcal{C}|$ grows polynomially with $|\mathcal{C}|$.

□

The fundamental theorem of learning theory characterizes PAC learnability of classes of binary classifiers using the VC-dimension. The VC-dimension of a class is a combinatorial property that denotes the maximal sample size that the class can shatter. The fundamental theorem states that a class is PAC learnable if, and only if, its VC-dimension is finite (and specifies the sample complexity required for PAC learning). The theorem also shows that if a problem is at all learnable, then uniform convergence holds and therefore, the problem is learnable using the ERM rule.

Chapter 6

Linear Predictors

In this chapter, we will study the family of linear predictors. We will introduce several hypothesis classes belonging to this family: halfspaces, linear regression predictors and logistic regression predictors. In addition, this chapter is focused on learning linear predictors using the ERM approach.

6.1 Affine functions

First, we define the class of affine functions as

$$L_d = \{h_{\mathbf{w},b} : \mathbf{w} \in \mathbb{R}^d, b \in \mathbb{R}\}$$

where

$$h_{\mathbf{w},b}(\mathbf{x}) = \langle \mathbf{w}, \mathbf{x} \rangle + b = \sum_{i=1}^{d} w_i x_i + b$$

It will be convenient also to use the notation

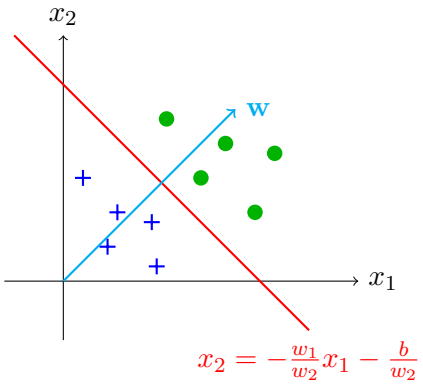
$$L_d = \{\mathbf{x} \mapsto \langle \mathbf{w}, \mathbf{x} \rangle + b : \mathbf{w} \in \mathbb{R}^d, b \in \mathbb{R}\}$$

Thus, L_d is a set of functions where each function is parametrized by 2 parameters, $\mathbf{w} \in \mathbb{R}^d$ and $b \in \mathbb{R}$, and each function takes as input a vector $\mathbf{x}$ and returns as output the scalar $\langle \mathbf{w}, \mathbf{x} \rangle + b$. The dimensionality of the parameter space is, of course, $d + 1$.

The different hypothesis classes of linear predictors are compositions of a function $\phi : \mathbb{R} \to \mathcal{Y}$ on L_d. For example, in binary classification, we can choose ϕ to be the sign function, and for regression problems where $\mathcal{Y} = \mathbb{R}$, the function ϕ is simply the identity function.

$$x \in \mathbb{R}^d \longrightarrow L_d(x) = z \in \mathbb{R} \longrightarrow \phi(z) = y \in \mathcal{Y}$$

6.1.1 2D geometric interpretation



1) the *bias* b is proportional to the offset of the line from the origin

2) the slope is determined by the *weights* $\mathbf{w}$

3) the *weights* vector $\mathbf{w}$ is perpendicular to the line

6.2 Homogeneous linear functions

It may be more convenient to incorporate the *bias* b into $\mathbf{w}$ as an extra coordinate and add an extra coordinate with a value of 1 to all $\mathbf{x} \in \boldsymbol{\mathcal{X}}$. Namely, let

$$\mathbf{w}' = (b, w_1, \ldots, w_d) \in \mathbb{R}^{d+1} \qquad \mathbf{x}' = (1, x_1, \ldots, x_d) \in \mathbb{R}^{d+1}$$

Therefore

$$\begin{aligned} h_{\mathbf{w},b} = \langle \mathbf{w}, \mathbf{x} \rangle + b \quad &= \sum_{i=1}^{d} w_i x_i + b \quad &= b + w_1 x_1 + \ldots + w_d x_d \\ = \langle \mathbf{w}', \mathbf{x}' \rangle \quad &= \sum_{i=1}^{d+1} w'_i x'_i \quad &= b + w_1 x_1 + \ldots + w_d x_d \end{aligned}$$

It follows that each affine function in $\mathbb{R}^d$ can be rewritten as a homogeneous linear function in $\mathbb{R}^{d+1}$ applied over the transformation that appends the constant 1 to each input vector. Therefore, whenever it simplifies the presentation, we will omit the bias term and refer to $L - d$ as the class of homogeneous linear functions of the form $h_{\mathbf{w}}(\mathbf{x}) = \langle \mathbf{w}, \mathbf{x} \rangle$.

6.3 Halfspaces

The first hypothesis class we consider is the class of *halfspaces*, designed for binary classification problems. Namely, $\boldsymbol{\mathcal{X}} = \mathbb{R}^d$ and $\boldsymbol{\mathcal{Y}} = \{-1, +1\}$. The class of halfspaces is defined as follows:

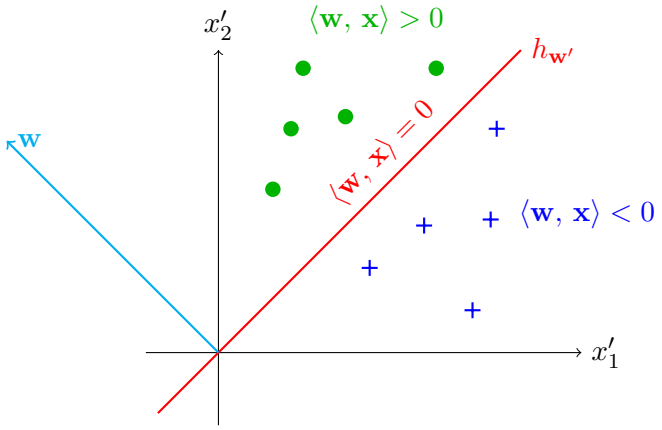
$$HS_d = \text{sign} \circ L_d = \{\mathbf{x} \mapsto \text{sign}(h_{\mathbf{w},b}(\mathbf{x})) \ : \ h_{\mathbf{w},b} \in L_d\}$$

In other words, each halfspace hypothesis in HS_d is parametrized by $\mathbf{w} \in \mathbb{R}^d$ and $b \in \mathbb{R}$ and upon receiving a vector $\mathbf{x}$ the hypothesis returns the label $\text{sign}(\langle \mathbf{w}, \mathbf{x} \rangle + b)$.

In the context of halfspaces, the realizable case is often referred to as the *separable* case. Namely, if data is not linearly separable (i.e., it is impossible to find a hyper-plane that perfectly splits the two classes into two half-spaces), then the realizability assumption falls. Implementing the ERM rule in the non-separable (i.e., agnostic) case is known to be computationally challenging.

6.3.1 2D geometric interpretation

Let us now illustrate this hypothesis class geometrically using homogeneous coordinates: by incorporating the bias b into the weight vector $\mathbf{w}$, we get a similar graphics representation as in Section 6.1.1. However, the hyper-plane dividing the two classes now crosses the origin of the coordinate system leading to a simpler mathematical model. In other words, each hypothesis forms a hyper-plane that is perpendicular to the vector $\mathbf{w}$ and intersects the vertical axis in $(0, 0)$ (instead of $(0, -b/w_2)$). The instances above the hyper-plane are labeled positively, while those below the hyper-plane are labeled negatively.



6.4 Perceptron for halfspaces

The perceptron is an iterative algorithm that constructs a sequence of vectors $\mathbf{w}^{(1)}, \mathbf{w}^{(2)}, \ldots$ until the optimal one is found. Initially, $\mathbf{w}^{(1)}$ is set to be the all-zeros vector. At iteration t, the perceptron finds an example i that is mislabeled by $\mathbf{w}^{(t)}$, namely an example for which $\text{sign}(\langle \mathbf{w}^{(t)}, \mathbf{x}_i \rangle) \neq y_i$. Then, the perceptron updates $\mathbf{w}^{(t)}$ by adding to it the instance $\mathbf{x}_i$ scaled by the label y_i. That is, $\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} + y_i \mathbf{x}_i$. Recall that our goal is to have $y_i \langle \mathbf{w}, \mathbf{x}_i \rangle > 0$ for all i and note that

$$y_i \langle \mathbf{w}^{(t+1)}, \mathbf{x}_i \rangle = y_i \langle \mathbf{w}^{(t)} + y_i \mathbf{x}_i, \mathbf{x}_i \rangle = y_i \langle \mathbf{w}^{(t)}, \mathbf{x}_i \rangle + ||\mathbf{x}_i||^2$$

Hence, the perceptron update guides the solution to be more positive on the i-th example, namely, more correct.

Algorithm: Perceptron

input: training set $\boldsymbol{\mathcal{S}} = ((\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_m, y_m))$
initialize: $\mathbf{w}^{(1)} = (0, \ldots, 0)$
for $t = 1, 2, \ldots$ **do**
  if $\exists\ i\ :\ y_i \langle \mathbf{w}^{(t)}, \mathbf{x}_i \rangle \leq 0$ **then**
    $\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} + y_i \mathbf{x}_i$
  else
    output: $\mathbf{w}^{(t)}$
  end
end

The following theorem guarantees that the algorithm stops with all sample points correctly classified in the realizable case.

Theorem 6.4.1. *Assume that* $(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_m, y_m)$ *is separable.*
Let $B = \min\{||\mathbf{w}|| \ :\ y_i \langle \mathbf{w}^{(t)}, \mathbf{x}_i \rangle \geq 1 \ \ \forall\ i = 1, \ldots, m\}$. *Let* $R = \max_i ||\mathbf{x}_i||$.
Then, the perceptron algorithm stops after at most $(RB)^2$ *iterations. When it stops, it holds that* $y_i \langle \mathbf{w}^{(T)}, \mathbf{x}_i \rangle > 0 \ \ \forall\ i = 1, \ldots, m$.

Proof. By definition of the stopping condition, if the perceptron stops, it must have separated all the examples. We will show that if the perceptron runs for T iterations, then we must have $T < (RB)^2$, which implies that the perceptron must stop after at most $(RB)^2$ iterations.

Let $\mathbf{w}^*$ be a vector that achieves the minimum in the definition of B. That is, $y_i \langle \mathbf{w}^*, \mathbf{x}_i \rangle \geq 1$ for all i, and among all vectors that satisfy these constraints, $\mathbf{w}^*$ is of minimal norm.

The idea of the proof is to show that performing T iterations, the cosine of the angle between $\mathbf{w}^*$ and $\mathbf{w}^{(T+1)}$ is at least $\frac{\sqrt{T}}{RB}$. Namely, we will show that

$$\frac{\langle \mathbf{w}^*, \mathbf{w}^{(T+1)} \rangle}{||\mathbf{w}^*||\ ||\mathbf{w}^{(T+1)}||} \geq \frac{\sqrt{T}}{RB} \tag{6.1}$$

By the Cauchy-Schwartz inequality, the left-hand side of Equation 6.1 is at most 1 (it is a cosine!). Therefore Equation 6.1 would imply that

$$1 \geq \frac{\sqrt{T}}{RB} \qquad \Longrightarrow \qquad T \leq (RB)^2$$

which will conclude our proof.

To show that Equation 6.1 holds we first show that $\langle \mathbf{w}^*, \mathbf{w}^{(T+1)} \rangle \geq T$ (numerator). Indeed, at the first iteration $\mathbf{w}^{(1)} = (0, \ldots, 0)$ and therefore $\langle \mathbf{w}^*, \mathbf{w}^{(1)} \rangle = 0$.

On iteration t, if we update using example $(\mathbf{x}_i,\, y_i)$ we have that

$$\begin{aligned}\langle \mathbf{w}^*,\, \mathbf{w}^{(t+1)}\rangle - \langle \mathbf{w}^*,\, \mathbf{w}^{(t)}\rangle &= \langle \mathbf{w}^*,\, \mathbf{w}^{(t+1)} - \mathbf{w}^{(t)}\rangle \\ &= \langle \mathbf{w}^*,\, y_i\mathbf{x}_i\rangle = y_i\langle \mathbf{w}^*,\, \mathbf{x}_i\rangle \\ &\geq 1\end{aligned}$$

Therefore, after performing T iterations we get

$$\langle \mathbf{w}^*,\, \mathbf{w}^{(T+1)}\rangle = \sum_{t=1}^{T}\Big(\langle \mathbf{w}^*,\, \mathbf{w}^{(t+1)}\rangle - \langle \mathbf{w}^*,\, \mathbf{w}^{(t)}\rangle\Big) \geq T \tag{6.2}$$

as required.

Next, we deal with the denominator of Equation 6.1, and we start by showing the upper bound of $||\mathbf{w}^{(T+1)}||$. For each iteration t, we have that

$$\begin{aligned}||\mathbf{w}^{(t+1)}||^2 &= ||\mathbf{w}^{(t)} + y_i\mathbf{x}_i||^2 \\ &= ||\mathbf{w}^{(t)}||^2 + y_i^2||\mathbf{x}_i||^2 + 2y_i\langle \mathbf{w}^{(t)},\, \mathbf{x}_i\rangle \\ &\leq ||\mathbf{w}^{(t)}||^2 + R^2\end{aligned} \tag{6.3}$$

The last inequality is because the example i is necessarily such that $y_i\langle \mathbf{w}^{(t)},\, \mathbf{x}_i\rangle \leq 0$ and the norm of $\mathbf{x}_i$ is at most R.

Since $||\mathbf{w}^{(1)}||^2 = 0$, if we use Equation 6.3 recursively for T iterations we obtain that

$$||\mathbf{w}^{(T+1)}||^2 \leq TR^2 \qquad \Longrightarrow \qquad ||\mathbf{w}^{(T+1)}|| \leq \sqrt{T}R \tag{6.4}$$

Combining Equation 6.2 with Equation 6.4, and using the fact that $||\mathbf{w}^*|| = B$ we obtain that

$$\frac{\langle \mathbf{w}^*,\, \mathbf{w}^{(T+1)}\rangle}{||\mathbf{w}^*||\;||\mathbf{w}^{(T+1)}||} \geq \frac{T}{RB\sqrt{T}} = \frac{\sqrt{T}}{RB}$$

We have thus shown that Equation 6.1 holds, and this concludes our proof. □

The perceptron is simple to implement and is guaranteed to converge. However, the convergence rate depends on the parameter B, which, in some situations, might be exponentially large in d.

We can introduce a new parameter λ, *learning rate*, such that the perceptron update rule becomes

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} + \lambda\, y_i\mathbf{x}_i$$

This parameter regulates how fast the perceptron reaches the solution. However, while gaining speed, the algorithm might decrease its stability.

6.5 VC-dimension of halfspaces

Theorem 6.5.1. *The VC-dimension of the homogeneous half-spaces class in $\mathbb{R}^d$ is d.*

Theorem 6.5.2. *The VC-dimension of the class of non-homogeneous halfspaces in $\mathbb{R}^d$ is $d+1$.*

Proof (Homogeneous case). To demonstrate Theorem 6.5.1 we need to show that

1. $\text{VCdim}(HS_d) \geq d$
2. $\text{VCdim}(HS_d) < d+1$

First, consider the set of vectors $\mathbf{e}_1, \ldots, \mathbf{e}_d$ where the vector $\mathbf{e}_i$ is the all-zeros vector for all i except 1 in the i-th coordinate. The class of homogeneous halfspaces shatters this set. Indeed, for every labeling $y_1, \ldots, y_d$ set $\mathbf{w} = (y_1, \ldots, y_d)$ and then $\langle \mathbf{w}, \mathbf{e}_i \rangle = y_i$ for all i.

Next, let $\mathbf{x}_1, \ldots, \mathbf{x}_{d+1}$ be a set of $d+1$ vectors in $\mathbb{R}^d$. Then, there must exist real numbers $a_1, \ldots, a_{d+1}$ such that $\sum_{i=1}^{d+1} a_i \mathbf{x}_i = \vec{0}$. In other words, in a d-dimensional space, $d+1$ vectors must be linearly dependent.

Let $I = \{i \ : \ a_i > 0\}$ and $J = \{i \ : \ a_i < 0\}$. Either I or J is not empty (we cannot have all coefficients be zero). Let us first assume that both of them are not empty. Then

$$\sum_{i \in I} a_i \mathbf{x}_i = \sum_{j \in J} |a_j| \mathbf{x}_j$$

Now, suppose that $\mathbf{x}_1, \ldots, \mathbf{x}_{d+1}$ are shattered by the class of homogeneous halfspaces. Then, there must exist a vector $\mathbf{w}$ such that $\langle \mathbf{w}, \mathbf{x}_i \rangle > 0$ for all $i \in I$ while $\langle \mathbf{w}, \mathbf{x}_j \rangle < 0$ for all $j \in J$. Then

$$\begin{aligned} 0 < \sum_{i \in I} a_i \langle \mathbf{x}_i, \mathbf{w} \rangle &= \left\langle \sum_{i \in I} a_i \mathbf{x}_i, \mathbf{w} \right\rangle \\ &= \left\langle \sum_{j \in J} |a_j| \mathbf{x}_j, \mathbf{w} \right\rangle = \sum_{j \in J} |a_j| \langle \mathbf{x}_j, \mathbf{w} \rangle < 0 \end{aligned}$$

which leads to a contradiction. Finally, if J (respectively I) is empty, then the right-most (left-most) inequality should be replaced by equality, which still leads to a contradiction. □

6.6 Linear regression

Linear regression is a common statistical tool for modeling the relationship between some "explanatory" variables and some real-valued outcomes.

As a learning problem, the domain set $\boldsymbol{\mathcal{X}}$ is a subset of $\mathbb{R}^d$ and the label set $\boldsymbol{\mathcal{Y}}$ is the set of real numbers $\mathbb{R}$. We want to learn a linear function $h : \mathbb{R}^d \to \mathbb{R}$ that best approximates the relationship between our variables.

The hypothesis class $\boldsymbol{\mathcal{H}}$ of linear regression is simply the set of linear functions

$$\boldsymbol{\mathcal{H}}_{\text{reg}} = L_d = \{\mathbf{x} \mapsto \langle \mathbf{w}, \mathbf{x} \rangle + b \ : \ \mathbf{w} \in \mathbb{R}^d, b \in \mathbb{R}\}$$

Next, we need to define a loss function for regression. While in classification the definition of the loss is straightforward, in regression, instead, we need to define how much we shall be penalized for the discrepancy between $h(\mathbf{x})$ and y. One common way is to use the squared-loss function

$$l(h, (\mathbf{x}, y)) = (h(\mathbf{x}) - y)^2$$

For this loss function, the empirical risk function is called the *mean squared error*

$$L_{\boldsymbol{\mathcal{S}}}(h) = \frac{1}{m} \sum_{i=1}^{m} (h(\mathbf{x}_i) - y_i)^2$$

6.6.1 Least squares

Least squares is the algorithm that solves the ERM problem for the hypothesis class of linear regression predictors with respect to the squared loss.

The ERM problem with respect to this class, given a training set $\boldsymbol{\mathcal{S}}$ and using the homogeneous version of L_d, is to find

$$\arg\min_{\mathbf{w}} L_{\boldsymbol{\mathcal{S}}}(h_{\mathbf{w}}) = \arg\min_{\mathbf{w}} \frac{1}{m} \sum_{i=1}^{m} (\langle \mathbf{w}, \mathbf{x}_i \rangle - y_i)$$

To solve the problem we compte the gradient of the objective function and compare it to zero. That is, we need to solve

$$\frac{\partial L_{\boldsymbol{\mathcal{S}}}}{\partial \mathbf{w}} = \frac{2}{m} \sum_{i=1}^{m} \left(\langle \mathbf{w}, \mathbf{x}_i \rangle - y_i \right) \mathbf{x}_i = 0$$

We can rewrite the probem as $A\mathbf{w} = \mathbf{b}$ where

$$A = \sum_{i=1}^{m} x_i x_i^{\mathrm{T}} \qquad\qquad \mathbf{b} = \sum_{i=1}^{m} y_i \mathbf{x}_i \tag{6.5}$$

If A is invertible, then the solution to the ERM problem is

$$\mathbf{w} = A^{-1}\,\mathbf{b}$$

6.6.2 Polynomial regression

Some learning tasks call for non-linear predictors, such as polynomial predictors. Take, for instance, a one dimensional polynomial function of degree n

$$p(x) = a_0 + a_1 x + a_2 x^2 + \ldots + a_n x^n$$

where $(a_0, \ldots, a_n) \equiv \mathbf{a}$ is a vector of coefficients of size $n+1$. We will focus here on the class of one dimensional, n-degree, polynomial regression predictors

$$\boldsymbol{\mathcal{H}}^n_{\text{poly}} = \{x \mapsto p(x)\}$$

where p is a one dimensional polynomial of degree n parametrized by a vector of coefficients $\mathbf{a} = (a_0, \ldots, a_n)$. Note that $\boldsymbol{\mathcal{X}} = \mathbb{R}$ since is is a one dimensional polynomial and $\boldsymbol{\mathcal{Y}} = \mathbb{R}$ as this is a regression problem.

One way to learn this class is by reduction to the problem of linear regression, which we have already shown how to solve. To translate a polynomial regression problem to a linear regression problem we define the mapping $\psi : \mathbb{R} \to \mathbb{R}^{n+1}$ such that $\psi(x) = (1, x, x^2, \ldots, x^n)$. Then we have that

$$p(\psi(x)) = a_0 + a_1 x + a_2 x^2 + \ldots + a_n x^n = \langle \mathbf{a}, \psi(x) \rangle$$

and we can find the optimal vector of coefficients $\mathbf{a}$ by using the least squares algorithm as shown earlier. Basically we transorm a n-degree polynomial into a $(n+1)$-dimensional linear problem.

6.7 Logistic regression

In logistic regression we learn a family of functions h from $\mathbb{R}^d$ to the interval $[0, 1]$. However, logistic regression is used for classification tasks. We can interpret $h(\mathbf{x})$ as the *probability* that the label of $\mathbf{x}$ is 1. The hypothesis class associated with logistic regression is the composition of a sigmoid function $\phi_{\text{sig}} : \mathbb{R} \to [0, 1]$ over the class of linear functions L_d. In particular, the sigmoid function used in logistic regression is the *logistic function* defined as

$$\phi_{\text{sig}}(z) = \frac{1}{1 + e^{-z}} \tag{6.6}$$

The hypothesis class is, therefore,

$$\boldsymbol{\mathcal{H}}_{\text{sig}} = \phi_{\text{sig}} \circ L_d = \{\phi_{\text{sig}}(\langle \mathbf{w}, \mathbf{x} \rangle) \;:\; \mathbf{w} \in \mathbb{R}^d\}$$

Note that when $\langle \mathbf{w}, \mathbf{x} \rangle$ is very large then $\phi_{\text{sig}}(\langle \mathbf{w}, \mathbf{x} \rangle)$ is close to 1, whereas if $\langle \mathbf{w}, \mathbf{x} \rangle$ is very small then $\phi_{\text{sig}}(\langle \mathbf{w}, \mathbf{x} \rangle)$ is close to 0. Recall that the prediction of the halfspace corresponding to a vector $\mathbf{w}$ is $\text{sign}(\langle \mathbf{w}, \mathbf{x} \rangle)$. Therefore, the predictions of the halfspace hypothesis and the logistic regression

hypothesis are very similar whenever $|\langle \mathbf{w}, \mathbf{x} \rangle|$ is large. However, when $|\langle \mathbf{w}, \mathbf{x} \rangle|$ is close to 0 we have that $\phi_{\text{sig}}(\langle \mathbf{w}, \mathbf{x} \rangle) \approx \frac{1}{2}$.

Intuitively, the logistic hypothesis is not sure about the label's value, so it guesses that the label is $\text{sign}(\langle \mathbf{w}, \mathbf{x} \rangle)$ with a probability slightly larger than 50%. In contrast, the halfspace hypothesis always outputs a deterministic prediction of either $+1$ or -1 even if $|\langle \mathbf{w}, \mathbf{x} \rangle|$ is very close to 0.

Next, we need to specify a loss function. That is, we need to define how bad it is to predict some $h_{\mathbf{w}}(\mathbf{x}) \in [0, 1]$ given that the true label is $y \in \{-1, +1\}$. We would like that $h_{\mathbf{w}}(\mathbf{x})$ would be large if $y = +1$ and that $1 - h_{\mathbf{w}}(\mathbf{x})$ (i.e., the probability of predicting -1) would be large if $y = -1$.

Note that

$$1 - h_{\mathbf{w}}(\mathbf{x}) = 1 - \frac{1}{1 + e^{-\langle \mathbf{w}, \mathbf{x} \rangle}} = \frac{1}{1 + e^{\langle \mathbf{w}, \mathbf{x} \rangle}}$$

Therefore, any reasonable loss function would increase monotonically with $\frac{1}{1+\exp(y\langle \mathbf{w}, \mathbf{x} \rangle)}$ or equivalently would increase monotonically with $1 + \exp(-y\langle \mathbf{w}, \mathbf{x} \rangle)$. The logistic loss function used in logistic regression penalizes $h_{\mathbf{w}}$ based on the logarithm of $1 + \exp(-y\langle \mathbf{w}, \mathbf{x} \rangle)$ (recall that log is a monotonic function). That is,

$$l(h_{\mathbf{w}}, (\mathbf{x}, y)) = \log\left(1 + e^{-y\langle \mathbf{w}, \mathbf{x} \rangle}\right)$$

Therefore, given a training set $\boldsymbol{\mathcal{S}}$, the ERM problem associated with logistic regression is

$$\arg\min_{\mathbf{w}} \frac{1}{m} \sum_{i=1}^{m} \log\left(1 + e^{-y_i \langle \mathbf{w}, \mathbf{x}_i \rangle}\right)$$

The advantage of the logistic loss function is that it is a convex function with respect to $\mathbf{w}$; hence the ERM problem can be solved efficiently using standard methods.

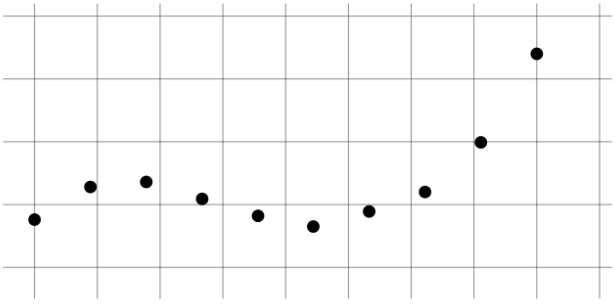
The ERM problem associated with logistic regression is identical to the problem of finding a *maximum likelihood estimator*, a well-known statistical approach for finding the parameters that maximize the joint probability of a given dataset assuming a specific parametric probability function (that is the reason of why we place the logarithm in the loss function!).

Chapter 7

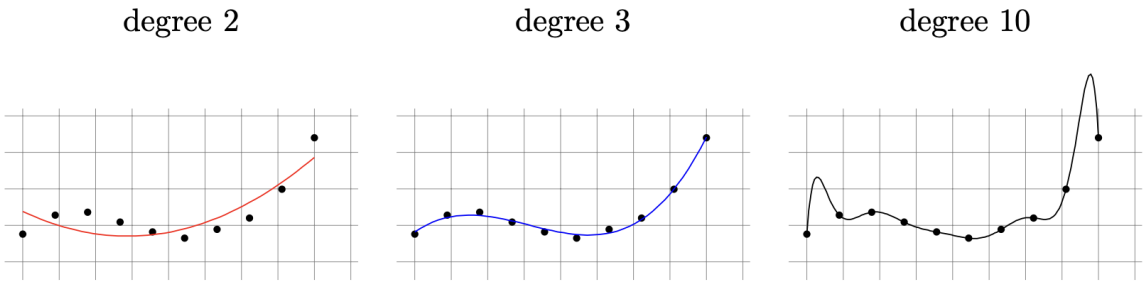
Model Selection and Validation

When approaching some practical problem, we usually think of several algorithms that may yield a good solution, each of which might have several parameters. How can we choose the best algorithm for the particular problem at hand? Moreover, how do we set the algorithm parameters? This task is often called *model selection*.

To illustrate the model selection task, consider the problem of learning a one-dimensional regression function $h : \mathbb{R} \to \mathbb{R}$. Suppose that we obtain a training set as depicted in the figure below.



We can consider fitting a polynomial to the data. However, we might be uncertain regarding which degree d would give the best results for our dataset. A small degree may not fit the data well (i.e., large approximation error), whereas a high degree may lead to overfitting (i.e., large estimation error). In the following, we depict the result of fitting a polynomial of degrees 2, 3 and 10. It is easy to see that the empirical risk decreases as we enlarge the degree. However, looking at the plots, our intuition tells us that setting the degree to 3 may be better than setting it to 10. It follows that the empirical risk alone is not enough for model selection.



The approach we present in this chapter for model selection relies on the concept of *validation*. The basic idea is to partition the training set into two sets. One is used for training each candidate model, and the other is used for deciding which yields the best results.

In model selection tasks, we try to find the right balance between approximation error and estimator error. More generally, if our learning algorithm fails to find a predictor with a small risk, it is crucial to understand whether we suffer from overfitting or underfitting.

7.1 Validation

We would often like to estimate better the true risk of the output predictor of a learning algorithm. So far, we have derived bounds on the estimation error of a hypothesis class, which tells us that for all the class hypotheses, the true risk is not very far from the empirical risk. However, these bounds might be loose and pessimistic, as they hold for all hypotheses and all possible data distributions. Therefore, a more accurate estimation of the true risk can be obtained by using some training data as a validation set, over which one can evaluate the success of the algorithm output predictor. This procedure is called *validation*.

7.1.1 Validation set

The simplest way to estimate the true error of a predictor h is by sampling an additional set of examples independent of the training set and using the empirical error on this validation set as our estimator. Formally, let $V = ((\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_{m_v}, y_{m_v}))$ be a set of fresh m_v examples that are sampled according to $\boldsymbol{\mathcal{D}}$ independently of the m examples of the training set $\boldsymbol{\mathcal{S}}$.

Theorem 7.1.1. *Let h be some predictor and assume that the loss function is in $[0, 1]$. Then, for every $\delta \in (0, 1)$, with probability of at least $1 - \delta$ over the choice of a validation set v of size m_v holds*

$$|L_V(h) - L_{\boldsymbol{\mathcal{D}}}(h)| \leq \sqrt{\frac{\log(2/\delta)}{2m_v}}$$

in which it appears neither the dimensionality of the training set nor the VC-dimension of a hypothesis class.

The bound in Theorem 7.1.1 does not depend on the algorithm or the training set used to construct h and is tighter than the usual bound we have seen. The reason for the tightness of this bound is that it is in terms of an estimate on a fresh validation set that is independent of the way h was generated. To illustrate this, suppose that h was obtained by applying an ERM predictor with respect to a hypothesis class of VC-dimension d over a training set of m examples. Then, from the quantitative version of the fundamental theorem of statistical learning, we have

$$L_{\boldsymbol{\mathcal{D}}}(h) \leq L_{\boldsymbol{\mathcal{S}}}(h) + \sqrt{C\frac{d + \log(1/\delta)}{m}} \qquad\qquad L_{\boldsymbol{\mathcal{D}}}(h) \leq L_V(h) + \sqrt{\frac{\log(2/\delta)}{2m_v}}$$

where the left-hand inequality comes from the quantitative version of the fundamental theorem of statistical learning, while the right-hand inequality is the one we just saw in Theorem 7.1.1.

Therefore, taking m_v to be the order of m, we obtain an estimate that is more accurate by a factor that depends on the VC-dimension. On the other hand, the price we pay for using such an estimate is that it requires an additional sample on top of the one used for training the learner.

Sampling a training set and then an independent validation set is equivalent to randomly partitioning our random set of examples into two parts.

7.1.2 Validation for model selection

Validation can be naturally used for model selection as follows. We first train different algorithms (or the same algorithm with different hyper-parameters) on the given training set $\boldsymbol{\mathcal{S}}$. Let $H = \{h_1, h_2, \ldots, h_r\}$ be the set of all output predictors of the different algorithms. Now, to choose a single predictor from H, we sample a fresh validation set and choose the predictor that minimizes the error over the validation set.

This process is very similar to learning a finite hypothesis class. The only difference is that H is not fixed ahead of time but depends on the training set. However, since the validation set is independent of the training set, we get that it is also independent of H. Therefore, the same technique we used to derive bounds for finite hypothesis classes holds here as well. In particular, we obtain

Theorem 7.1.2. *Let $H = \{h_1, h_2, \ldots, h_r\}$ be an arbitrary set of predictors and assume the loss function is in $[0, 1]$. Assume that a validation set V of size m_v is sampled independently of H. Then, with a probability of at least $1 - \delta$ over the choice of V, we have*

$$\forall h \in H, \quad |L_{\mathcal{D}}(h) - L_V(h)| \leq \sqrt{\frac{\log\left(\frac{2|H|}{\delta}\right)}{2m_v}}$$

Proof. We can split the proof of this theorem into three main steps:

1. Apply Theorem 7.1.1 to each ERM predictor $h_i \in H = \{h_1, h_2, \ldots, h_r\}$ that is in the set of ERM predictors from different learning algorithms. We have that

$$P_{\text{bad-valid}} = P\left(|L_{\mathcal{D}}(h) - L_V(h)| > \sqrt{\frac{\log\left(\frac{2}{\delta}\right)}{2m_v}}\right) \leq \delta$$

2. Repeat the theorem application for $|H|$ times. Recall the union bound from statistics theory $P(\cup_i A_i) \leq \sum_i P(A_i)$ and apply it to get

$$P_{\text{bad-valid-all}} \leq \sum_{|H|} \delta = |H|\delta$$

3. To find $P_{\text{bad-valid-all}} \leq \delta_{\text{all}}$ simply set $\delta' = \frac{\delta}{|H|}$ thus $\delta_{\text{all}} = \sum_{|H|} \delta' = |H|\frac{\delta}{|H|}$

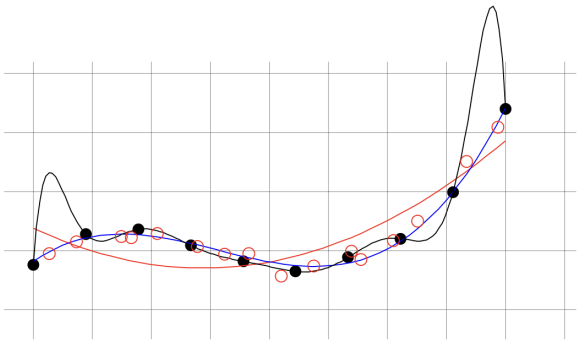
We finally obtain the wished result

$$\forall h \in H, \quad |L_{\mathcal{D}}(h) - L_V(h)| \leq \sqrt{\frac{\log\left(\frac{2}{\delta'}\right)}{2m_v}} = \sqrt{\frac{\log\left(\frac{2|H|}{\delta}\right)}{2m_v}}$$

□

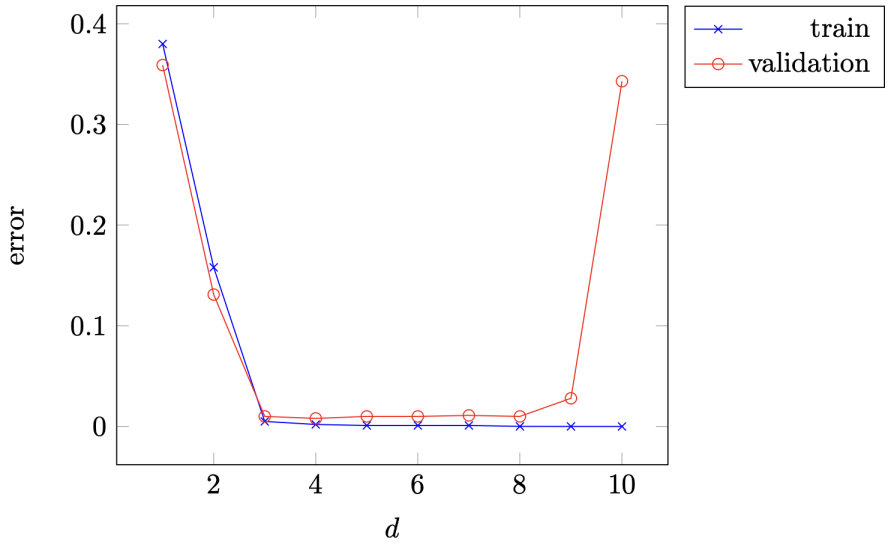
This theorem tells us that the error on the validation set approximates the true error as long as H is not too large. However, if we try too many methods, then we are in danger of overfitting.

To illustrate how the validation set is useful for model selection, consider the example of fitting a one-dimensional polynomial, as described at the beginning of this chapter. In the following, we depict the same training set (black, filled dots) with ERM polynomials of degrees 2, 3 and 10. This time we also depict an additional validation set (red, unfilled dots). The polynomial of degree 10 has minimal training error, yet a polynomial of degree 3 has minimal validation error, and hence it will be chosen as the best model.



7.1.3 Model-selection curve

The model selection curve shows the training error and validation error as a function of the complexity of the model considered. For example, for the polynomial fitting problem mentioned previously, the curve will look like the picture below

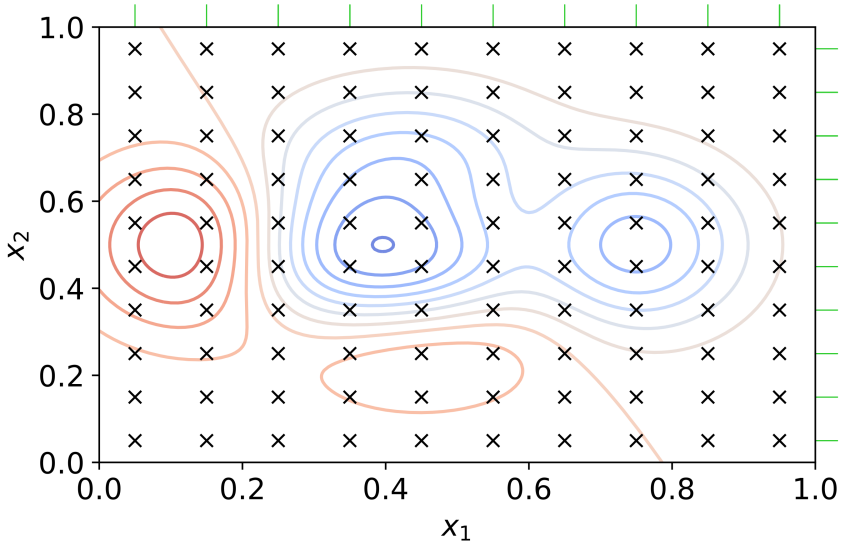


As can be shown, the training error is monotonically decreasing as we increase the polynomial degree (which is the complexity of the model in our case). On the other hand, the validation error decreases but then increases, indicating that we are starting to suffer from overfitting.

Plotting such curves can help us understand whether we are searching for the correct regime of our parameter space.

7.1.4 Grid-search for multiple parameters

Often, there may be more than a single parameter to tune, and the possible number of values each parameter can take might be quite large. In such cases, we start with a rough grid of values for the parameters and plot the corresponding model selection curve. Based on the curve, we will zoom in on the correct regime and employ a finer grid to search over. It is crucial to verify that we are in the relevant regime. For example, in the polynomial fitting problem described, if we start searching degrees from the set of values {1, 10, 20} and do not employ a finer grid based on the resulting curve, we will end up with a rather poor model.



Note that the empirical risk on the validation set is not an estimate of the true risk, particularly if we choose among many models (i.e., large $|H|$). Furthermore, grid search does not always find the global minimum for the set of parameters but gives a reasonable approximation.

7.1.5 Train-validation-test split

In most practical applications, we split the available examples into three sets. The first set is used for training our algorithm, and the second is used as a validation set for the model selection. After we select the best model, we test the performance of the output predictor on the third set, which is often called the *test set*. The number obtained is used as an estimator of the true error of the learned predictor. Namely

- **Training set:** used to learn the best model h_i^{ERM} inside each hypothesis class $\boldsymbol{\mathcal{H}}_i$
- **Validation set:** used to pick one hypothesis h^* from the set of all h_i^{ERM}, that is H
- **Test set:** used to estimate the true risk $L_{\boldsymbol{\mathcal{D}}}(h^*)$

The estimate from the test set has the guarantees provided by the proposition on the estimate of $L_{\boldsymbol{\mathcal{D}}}(h^*)$ for one hypothesis class. Note that if we choose another hypothesis after using the test set to estimate $L_{\boldsymbol{\mathcal{D}}}(h^*)$, we cannot use the test set again to estimate the new $L_{\boldsymbol{\mathcal{D}}}(h^*)$.

7.1.6 k-Fold cross validation

The validation procedure described so far assumes that data is plentiful and that we can sample a fresh validation set. However, in some applications, data is scarce, and we do not want to "waste" data on validation. Therefore, the k-fold cross-validation technique is designed to estimate the true error accurately without wasting too much data.

In k-fold cross-validation, the original training set is partitioned into k subsets (folds) of size m/k. Then, for each fold, the algorithm is trained on the union of the other folds, and the error of its output is estimated using the fold. Finally, the average of all these errors estimates the true error. The case $k = m$, where m is the number of examples in the training set, is called *leave-one-out*.

k-Fold cross-validation is often used for model selection or parameter tuning, and once the best parameter is chosen, the algorithm is retrained using this parameter on the entire training set.

Algorithm: k-Fold cross validation for model selection

input:

training set $\boldsymbol{\mathcal{S}} = ((\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_m, y_m))$
set of parameter values Θ
learning algorithm A
integer k ; `// the number of folds`

partition: $\boldsymbol{\mathcal{S}}$ into $\boldsymbol{\mathcal{S}}_1, \boldsymbol{\mathcal{S}}_2, \ldots \boldsymbol{\mathcal{S}}_k$

foreach $\theta \in \Theta$ **do**

  for $i = 1, \ldots, k$ **do**

    $h_{i,\theta} = A(S/S_i\,;\,\theta)$; `// remove the` i`-th fold`

  end

  $\mathrm{error}(\theta) = \frac{1}{k}\sum_{i=1}^{k} L_{\boldsymbol{\mathcal{S}}_i}(h_{i,\theta})$; `// use the` i`-th fold for testing`

end

output:

$\theta^* = \arg\min_\theta[\mathrm{error}(\theta)]$
$h_{\theta^*} = A(\boldsymbol{\mathcal{S}}\,;\,\theta^*)$; `// using the whole training set`

7.2 What if learning fails

Consider the following scenario: you were given a learning task and have approached it with a choice of a hypothesis class, a learning algorithm and its hyper-parameters. You used a validation set to tune the parameters and tested the learned predictor on a test set. The test results, unfortunately, turn out to be unsatisfactory. What went wrong then, and what should you do next?

Many elements can be fixed. The main approaches are

- Get a larger sample
- Change the hypothesis class by
  - enlarging it
  - reducing it
  - completely changing it
  - changing the hyper-parameters you consider
- Change the feature representation of the data
- Change the optimization algorithm used to apply your learning rule

In order to find the best remedy, it is essential first to understand the cause of the bad performance. Recall that in Section 4.2, we decomposed the true error of the learned predictor into approximation error and estimation error.

The approximation error is defined to be $L_{\boldsymbol{\mathcal{D}}}(h^*)$ for some $h^* \in \arg\min_{h \in \boldsymbol{\mathcal{H}}} L_{\boldsymbol{\mathcal{D}}}(h)$. The estimation error is defined to be $L_{\boldsymbol{\mathcal{D}}}(h_{\boldsymbol{\mathcal{S}}}) - L_{\boldsymbol{\mathcal{D}}}(h^*)$, where $h_{\boldsymbol{\mathcal{S}}}$ is the learned predictor (which is based on the training set $\boldsymbol{\mathcal{S}}$).

The approximation error of the class does not depend on the sample size or the algorithm being used. Instead, it depends on the distribution $\boldsymbol{\mathcal{D}}$ and the hypothesis class $\boldsymbol{\mathcal{H}}$. Therefore, if the approximation error is large, it will not help us to enlarge the training set size, and it also does not make sense to reduce the hypothesis class. Instead, it can be beneficial to enlarge the hypothesis class or completely change it. We can also consider applying the same hypothesis class but on different feature representations of the data.

The estimation error of the class does depend on the sample size. Therefore, if we have a large estimation error, we can make an effort to obtain more training examples. We can also consider reducing the hypothesis class. However, it does not make sense to enlarge the hypothesis class in this case.

7.2.1 Error decomposition using validation

Understanding whether our problem is due to approximation or estimation error is very useful for finding the best remedy. In the previous section, we have seen how to estimate $L_{\boldsymbol{\mathcal{D}}}(h_{\boldsymbol{\mathcal{S}}})$ using the empirical risk on a validation set. However, it is more difficult to estimate the approximation error of the class.

Instead, we give a different error decomposition, one that can be estimated from the train and validation sets

$$L_{\boldsymbol{\mathcal{D}}}(h_{\boldsymbol{\mathcal{S}}}) = (L_{\boldsymbol{\mathcal{D}}}(h_{\boldsymbol{\mathcal{S}}}) - L_V(h_{\boldsymbol{\mathcal{S}}})) + (L_V(h_{\boldsymbol{\mathcal{S}}}) - L_{\boldsymbol{\mathcal{S}}}(h_{\boldsymbol{\mathcal{S}}})) + L_{\boldsymbol{\mathcal{S}}}(h_{\boldsymbol{\mathcal{S}}}) \tag{7.1}$$

The first term $(L_{\boldsymbol{\mathcal{D}}}(h_{\boldsymbol{\mathcal{S}}}) - L_V(h_{\boldsymbol{\mathcal{S}}}))$ can be bounded quite tightly using Theorem 7.1.1. Intuitively, when the second term $(L_V(h_{\boldsymbol{\mathcal{S}}}) - L_{\boldsymbol{\mathcal{S}}}(h_{\boldsymbol{\mathcal{S}}}))$ is large, we say that our algorithm suffers from *overfitting*. In contrast, when the empirical risk term $L_{\boldsymbol{\mathcal{S}}}(h_{\boldsymbol{\mathcal{S}}})$ is large, we say that our algorithm suffers from *underfitting*.

Note that these two terms are not necessarily good estimates of the estimation and approximation errors. To illustrate this, consider the case in which $\boldsymbol{\mathcal{H}}$ is a class of VC-dimension d, and $\boldsymbol{\mathcal{D}}$ is a

distribution such that the approximation error of $\mathcal{H}$ with respect to $\mathcal{D}$ is 1/4. As long as the size of our training set is smaller than d, we will have $L_{\mathcal{S}}(h_{\mathcal{S}}) = 0$ for every ERM hypothesis. Therefore, the training risk $L_{\mathcal{S}}(h_{\mathcal{S}})$ and the approximation error $L_{\mathcal{D}}(h^*)$ can be significantly different. Nevertheless, as we will show later, the values of $L_{\mathcal{S}}(h_{\mathcal{S}})$ and $(L_V(h_{\mathcal{S}}) - L_{\mathcal{S}}(h_{\mathcal{S}}))$ still provide us useful information.

Consider first the case in which $L_{\mathcal{S}}(h_{\mathcal{S}})$ is large. We can write

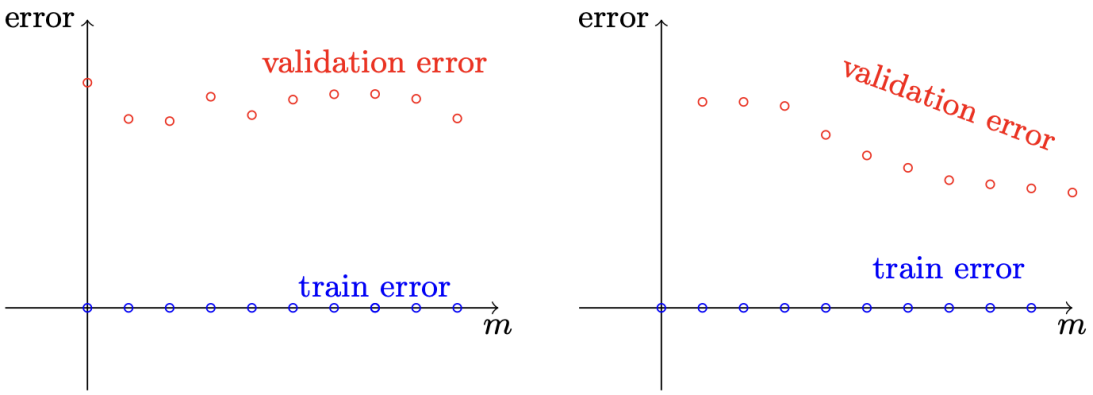
$$L_{\mathcal{S}}(h_{\mathcal{S}}) = (L_{\mathcal{S}}(h_{\mathcal{S}}) - L_{\mathcal{S}}(h^*)) + (L_{\mathcal{S}}(h^*) - L_{\mathcal{D}}(h^*)) + L_{\mathcal{D}}(h^*)$$

When $h_{\mathcal{S}}$ is an ERM$_{\mathcal{H}}$ hypothesis we have that $L_{\mathcal{S}}(h_{\mathcal{S}}) - L_{\mathcal{S}}(h^*) \leq 0$. In addition, since h^* does not depend on $\mathcal{S}$, the term $L_{\mathcal{S}}(h^*) - L_{\mathcal{D}}(h^*)$ can be bounded quite tightly as in Theorem 7.1.1. The last term is the approximation error. It follows that if $L_{\mathcal{S}}(h_{\mathcal{S}})$ is large, so is the approximation error, and the remedy to the failure of the algorithm should be tailored accordingly.

Next, consider the case in which $L_{\mathcal{S}}(h_{\mathcal{S}})$ is small. As we argued before, this does not necessarily imply that the approximation error is small. Indeed, consider two scenarios in which we are trying to learn a hypothesis class of VC-dimension d using the ERM learning rule. In the first scenario, we have a training set of $m < d$ examples, and the approximation error of the class is high. In the second scenario, we have a training set of $m > 2d$ examples, and the approximation error of the class is zero. In both cases $L_{\mathcal{S}}(h_{\mathcal{S}}) = 0$. How can we distinguish between the two cases?

7.2.2 Learning curves

One possible way to distinguish between the two cases is by plotting *learning curves*. To produce a learning curve, we train the algorithm on prefixes of the data of increasing size. For example, we can first train the algorithm on the first 10% of the examples, then on 20% of them and so on. Then, we compute the training and validation errors for each prefix on a predefined validation set. Such learning curves can help us distinguish between the two scenarios above. In the first scenario, we expect the validation error to be approximately 1/2 for all prefixes, as we did not learn anything. In the second scenario, the validation error will start as a constant but then should start decreasing (it must start decreasing once the training set size is larger than the VC-dimension). An illustration of the two cases is given in the figure below.



Left: the number of examples is always smaller than the VC-dimension of the class. Right: the approximation error is zero, and the number of examples is larger than the VC-dimension of the class.

In general, as long as the approximation error is greater than zero, we expect the training error to grow with the sample size, as a larger amount of data points makes it harder to explain all of them. On the other hand, the validation error tends to decrease with the increase in sample size. If the VC-dimension of the class is finite, when the sample size goes to infinity, the validation and training errors converge to the approximation error. Therefore, by extrapolating the training and validation curves, we can guess the value of the approximation error or at least get a rough estimate in an interval in which the approximation error resides.

Getting back to the problem of finding the best remedy for the failure of our algorithm, if we observe that $L_{\boldsymbol{\mathcal{S}}}(h_{\boldsymbol{\mathcal{S}}})$ is small while the validation error is large, then in any case, we know that the size of our training set is not sufficient for learning the class $\boldsymbol{\mathcal{H}}$. We can then plot a learning curve. If the validation error is starting to decrease, then the best solution is to increase the number of examples. Another reasonable solution is to decrease the complexity of the hypothesis class. On the other hand, if the validation error is kept around 1/2, then we have no evidence that the approximation error of $\boldsymbol{\mathcal{H}}$ is good. Increasing the training set size may not help us at all. Obtaining more data can still help us, as at some point, we can see whether the validation error starts to decrease or whether the training error starts to increase. However, if more data is expensive, it may be better first to try to reduce the complexity of the hypothesis class.

To summarize the discussion, the following steps should be applied:

1. If the learning involves parameter tuning, plot the model-selection curve to ensure that you tuned the parameter appropriately.
2. If the training error is excessively large, consider enlarging the hypothesis class, completely changing it or changing the feature representation of the data.
3. If the training error is small, plot learning curves and try to deduce from them whether the problem is the estimation error or approximation error.
4. If the approximation error is small enough, try to obtain more data. If this is not possible, consider reducing the complexity of the hypothesis class.
5. If the approximation error is large as well, try to change the hypothesis class or the feature representation of the data completely.

Chapter 8

Regularization and Stability

The new learning paradigm we introduce in this chapter is called *regularized loss minimization* (RLM for short). In RLM, we minimize the sum of the empirical error and a regularization function. Intuitively, the regularization function measures the complexity of the hypotheses. A possible interpretation of the regularization function is to see it as a *stabilizer* of the learning algorithm. An algorithm is considered stable if a slight change in its input does not change its output much. We will formally define the notion of stability and prove its close relation to learnability.

8.1 Regularized loss minimization

Regularized loss minimization is a learning rule in which we jointly minimize the empirical risk and a regularization function. Formally, a regularization function is a mapping $R : \mathbb{R}^d \to \mathbb{R}$, and the regularized loss minimization rule outputs a hypothesis in

$$\arg\min_{\mathbf{w}} \left(L_{\boldsymbol{\mathcal{S}}}(\mathbf{w}) + R(\mathbf{w}) \right) \tag{8.1}$$

Intuitively, the complexity of hypotheses is measured by the value of the regularization function and the algorithm balances between low empirical risk and simpler hypotheses.

One can use many possible regularization functions, reflecting some prior belief about the problem. Throughout this section, we will focus on one of the most simple regularization functions, called Tikhonov regularization

$$R(\mathbf{w}) = \lambda \left|\left|\mathbf{w}\right|\right|^2 \tag{8.2}$$

where $\lambda > 0$ is a scalar and the norm is the l_2 norm, namely $||w||^2 = \sum_{i=1}^{d} w_i^2$. This yields the learning rule

$$A(\boldsymbol{\mathcal{S}}) = \arg\min_{\mathbf{w}} \left(L_{\boldsymbol{\mathcal{S}}}(\mathbf{w}) + \lambda \left|\left|\mathbf{w}\right|\right|^2 \right) \tag{8.3}$$

As mentioned before, the norm $||\mathbf{w}||^2$ measures the complexity of the hypothesis defined by $\mathbf{w}$. The λ parameter controls the amount of regularization, which controls the tradeoff between empirical error and complexity.

8.1.1 Ridge regression

Applying the RLM rule with Tikhonov regularization to linear regression with the squared loss leads to the following learning rule:

$$\arg\min_{\mathbf{w}\in\mathbb{R}^d} \left(\lambda \left|\left|\mathbf{w}\right|\right|^2 + \frac{1}{m} \sum_{i=1}^{m} \frac{1}{2} \left(\langle \mathbf{w}, \mathbf{x}_i \rangle - y_i \right)^2 \right) \tag{8.4}$$

where λ balances between the two targets, the $1/m$ factor before the sum makes the balancing independent of the training set size.

To solve Equation 8.4, we compare the gradient of the objective to zero and obtain the set of linear equations

$$(2\lambda m\mathbb{I} + A)\mathbf{w} = \mathbf{b}$$

where $\mathbb{I}$ is the identity matrix and A, $\mathbf{b}$ are defined as

$$A = \sum_{i=1}^{m} \mathbf{x}_i \mathbf{x}_i^{\mathrm{T}} \qquad\qquad \mathbf{b} = \sum_{i=1}^{m} y_i \mathbf{x}_i$$

Since A is a positive semidefinite matrix, the matrix $2\lambda m\mathbb{I} + A$ has all its eigenvalues bounded below by $2\lambda m$. Hence, this matrix is invertible, and the solution to ridge regression becomes

$$\mathbf{w} = (2\lambda m\mathbb{I} + A)^{-1}\,\mathbf{b} \tag{8.5}$$

8.2 Stable rules do not overfit

Intuitively, a learning algorithm is stable if a small change in the input does not change the algorithm's output much. In this section, we define a specific notion of stability and prove that, under this definition, stable rules do not overfit.

Let A be a learning algorithm, let $\boldsymbol{\mathcal{S}} = (z_1, \ldots, z_m)$ be a training set of m examples, and let $A(\boldsymbol{\mathcal{S}})$ denote the output of A. The algorithm A suffers from overfitting if the difference between the true risk of its input $L_{\boldsymbol{\mathcal{D}}}(A(\boldsymbol{\mathcal{S}}))$ and the empirical risk of its output $L_{\boldsymbol{\mathcal{S}}}(A(\boldsymbol{\mathcal{S}}))$ is large.

Given the training set $\boldsymbol{\mathcal{S}}$ and an additional example z', let $\boldsymbol{\mathcal{S}}^{(i)}$ be the training set obtained by replacing the i-th example of $\boldsymbol{\mathcal{S}}$ with z'. Namely

$$\boldsymbol{\mathcal{S}}^{(i)} = (z_1, \ldots, z_{i-1}, z', z_{i+1}, \ldots, z_m)$$

In our definition of stability, "a small change of the input" means that we feed A with $\boldsymbol{\mathcal{S}}^{(i)}$ instead of with $\boldsymbol{\mathcal{S}}$. That is, we only replace one training example. We measure the effect of this small change of the input on the output of A by comparing the loss of the hypothesis $A(\boldsymbol{\mathcal{S}})$ on z_i to the loss of the hypothesis $A(\boldsymbol{\mathcal{S}}^{(i)})$ on z_i. Intuitively, a good learning algorithm will have

$$l\left(A(\boldsymbol{\mathcal{S}}^{(i)}, z_i)\right) - l\left(A(\boldsymbol{\mathcal{S}}, z_i)\right) \geq 0$$

since in the first term, the learning algorithm does not observe the sample z_i, in the second term, z_i is indeed observed. If the preceding difference is very large, we suspect the learning algorithm might overfit. This is because the learning algorithm drastically changes its prediction on z_i if it observes it in the training set. This is formalized in the following theorem.

Theorem 8.2.1. *Let $\boldsymbol{\mathcal{D}}$ be a distribution. Let $\boldsymbol{\mathcal{S}}$ be an i.i.d. sequence of examples, and let z' be another i.i.d. example. Let $U(m)$ be the uniform distribution over $[1, m]$. Then, for any learning algorithm*

$$\mathbb{E}_{\boldsymbol{\mathcal{S}}\sim\boldsymbol{\mathcal{D}}^m}\left[L_{\boldsymbol{\mathcal{D}}}(A(\boldsymbol{\mathcal{S}})) - L_{\boldsymbol{\mathcal{S}}}(A(\boldsymbol{\mathcal{S}}))\right] = \mathbb{E}_{(\boldsymbol{\mathcal{S}},z')\sim\boldsymbol{\mathcal{D}}^{m+1},\, i\sim U(m)}\left[l\left(A(\boldsymbol{\mathcal{S}}^{(i)}, z_i)\right) - l\left(A(\boldsymbol{\mathcal{S}}, z_i)\right)\right] \tag{8.6}$$

When the right-hand side of Equation 8.6 is small, we say that A is a *stable* algorithm.

Definition 8.2.1 (OAROS)**.** *Let $\epsilon : \mathbb{N} \to \mathbb{R}$ be a monotonically decreasing function. We say that a learning algorithm is on-average-replace-one-stable with rate $\epsilon(m)$ if for every distribution $\boldsymbol{\mathcal{D}}$*

$$\mathbb{E}_{(\boldsymbol{\mathcal{S}},z')\sim\boldsymbol{\mathcal{D}}^{m+1},\, i\sim U(m)}\left[l\left(A(\boldsymbol{\mathcal{S}}^{(i)}, z_i)\right) - l\left(A(\boldsymbol{\mathcal{S}}, z_i)\right)\right] \leq \epsilon(m)$$

Theorem 8.2.1 tells us that a learning algorithm does not overfit if and only if it is OAROS. Of course, a learning algorithm that does not overfit is not necessarily good. A useful algorithm should find a hypothesis that, on the one hand, fits the training set (i.e., has a low empirical risk) and, on the other hand, does not overfit (i.e., be stable). As we shall see, the parameter λ of the RLM rule balances between fitting the training set and being stable.

8.3 Tikhonov regularization as a stabilizer

In the previous section, we saw that stable rules do not overfit. This section shows that applying the RLM rule with Tikhonov regularization leads to a stable algorithm. We will assume that the loss function is convex and Lipschitz.

Definition 8.3.1 (Lipschitzness). *Let $C \subset \mathbb{R}^d$. A function $f : \mathbb{R}^d \to \mathbb{R}^k$ is ρ-Lipschitz over C if $\forall \mathbf{w}_1, \mathbf{w}_2 \in C$ we have that*

$$||f(\mathbf{w}_1) - f(\mathbf{w}_2)|| \leq \rho ||\mathbf{w}_1 - \mathbf{w}_2||$$

Theorem 8.3.1. *Assume that the loss function is convex and ρ-Lipschitz. Then, the RLM rule with the regularizer $\lambda ||\mathbf{w}||^2$ is OAROS with rate $\frac{2\rho^2}{\lambda m}$. It follows (using Theorem 8.2.1) that*

$$\mathbb{E}_{\boldsymbol{\mathcal{S}} \sim \boldsymbol{\mathcal{D}}^m} \left[L_{\boldsymbol{\mathcal{D}}}(A(\boldsymbol{\mathcal{S}})) - L_{\boldsymbol{\mathcal{S}}}(A(\boldsymbol{\mathcal{S}})) \right] \leq \frac{2\rho^2}{\lambda m}$$

Thus, Theorem 8.3.1 proves that Tikhonov regularization is a stabilizer: a larger λ parameter leads to a more stable solution (i.e., less overfitting). On top of that, the theorem also proves that a larger training set helps get a more stable solution.

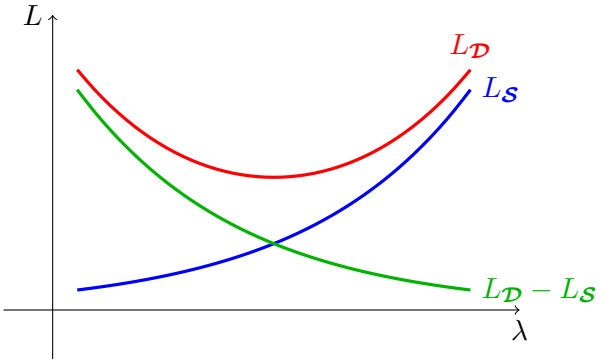
8.4 Fitting-stability tradeoff

We can rewrite the expected risk of a learning algorithm as

$$\mathbb{E}_{\boldsymbol{\mathcal{S}}} \left[L_{\boldsymbol{\mathcal{D}}}(A(\boldsymbol{\mathcal{S}})) \right] = \mathbb{E}_{\boldsymbol{\mathcal{S}}} \left[L_{\boldsymbol{\mathcal{S}}}(A(\boldsymbol{\mathcal{S}})) \right] + \mathbb{E}_{\boldsymbol{\mathcal{S}}} \left[L_{\boldsymbol{\mathcal{D}}}(A(\boldsymbol{\mathcal{S}})) - L_{\boldsymbol{\mathcal{S}}}(A(\boldsymbol{\mathcal{S}})) \right] \tag{8.7}$$

The first term reflects how well $A(\boldsymbol{\mathcal{S}})$ fits the training set, while the second term reflects the difference between the true and empirical risks of $A(\boldsymbol{\mathcal{S}})$ (i.e., overfitting). As we have shown in Theorem 8.2.1, the second term is equivalent to the stability of A. Since our goal is to minimize the risk of the algorithm, we need that the sum of both terms will be small.

In the previous section, we have bounded the stability term. We have shown that the stability term decreases as the regularization parameter λ increases. On the other hand, the empirical risk increases with λ. We, therefore, face a tradeoff between fitting and overfitting. This tradeoff is quite similar to the bias-complexity tradeoff.



To properly choose the value of λ that minimizes the sum of the two terms, it is common to use validation for model selection, as discussed in the previous chapter.

Chapter 9

Support Vector Machines

This chapter discusses a handy machine learning tool: the support vector machine paradigm (SVM) for learning linear predictors in high-dimensional feature spaces. However, the feature space's high dimensionality raises both sample and computational complexity challenges.

The SVM algorithmic parading tackles the sample complexity challenge by searching for "large margin" separators. Roughly speaking, a halfspace separates a training set with a large margin if all the examples are not only on the correct side of the separating hyperplane but also far away from it. Restricting the algorithm to output a large margin separator can yield a small sample complexity even if the dimensionality of the feature space is high.

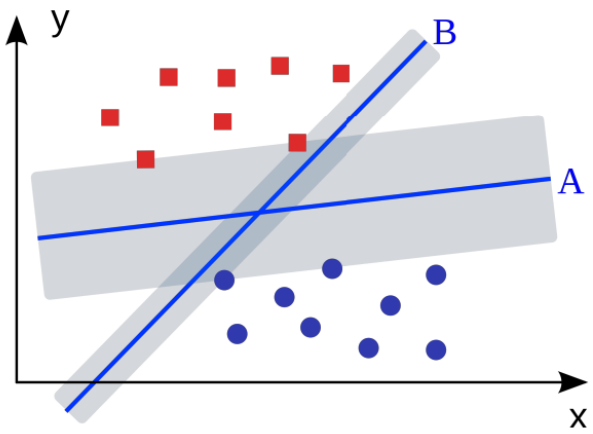
9.1 Classification margin

Let $\boldsymbol{\mathcal{S}} = ((\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_m, y_m))$ be a training set of examples where each $\mathbf{x}_i \in \mathbb{R}^d$ and $y_i \in \{-1, +1\}$. We say that the training set is linearly separable if there exists a halfspace $(\mathbf{w}, b)$ such that $y_i = \text{sign}(\langle \mathbf{w}, \mathbf{x}_i \rangle + b)$ for all $i = 1, ldots, m$. Alternatively, this condition can be rewritten as

$$y_i \left(\langle \mathbf{w}, \mathbf{x}_i \rangle + b \right) > 0 \quad \forall i = 1, \ldots, m$$

In other words, the training set is linearly separable if there exists a halfspace that perfectly classifies all examples. All halfspaces that satisfy this condition are ERM hypotheses (their 0-1 error is zero, which is the minimum possible error). For any separable training sample, there are many ERM halfspaces. Which one of them should the learner pick?

Consider, for example, the training set described in the following picture.



While both A and B hyperplanes perfectly classify all examples, our intuition would probably lead us to prefer A over B. One way to formalize this intuition is using the concept of *margin*.

The margin of a hyperplane with respect to a training set is defined as the minimal distance between a point in the training set and the hyperplane. If a hyperplane has a large margin, it will still separate the training set if we slightly perturb each instance.

We will see later that the true error of a halfspace can be bounded in terms of the margin over the training sample (the larger the margin, the smaller the error), regardless of the Euclidean dimension in which this halfspace resides.

Hard-SVM is the learning rule in which we return an ERM hyperplane that separates the training set with the largest possible margin. To define Hard-SVM formally, we first express the distance between a point $\mathbf{x}$ to a hyperplane using the parameters defining the halfspace.

Given a separating hyperplane defined by $P = \{\mathbf{v} : \langle \mathbf{v}, \mathbf{w} \rangle + b = 0\}$ and given an example $\mathbf{x}$, the distance of $\mathbf{x}$ to P is

$$d(\mathbf{x}, P) = \min\{||\mathbf{x} - \mathbf{v}|| : \mathbf{v} \in P\}$$

Theorem 9.1.1. *The distance between a hyperplane defined by* $(\mathbf{w}, b)$ *(assuming* $||\mathbf{w}|| = 1$*) and a point* $\mathbf{x}$ *is*

$$d(\mathbf{x}, P) = |\langle \mathbf{w}, \mathbf{x} \rangle + b|$$

Proof. The distance between a point $\mathbf{x}$ and the hyperplane is defined as

$$\min\{||\mathbf{x} - \mathbf{v}|| : \langle \mathbf{w}, \mathbf{v} \rangle + b = 0\}$$

Taking $\mathbf{v} = \mathbf{x} - (\langle \mathbf{w}, \mathbf{x} \rangle + b)\mathbf{w}$ we have that

$$\langle \mathbf{w}, \mathbf{v} \rangle + b = \langle \mathbf{w}, \mathbf{x} \rangle - (\langle \mathbf{w}, \mathbf{x} \rangle + b)||\mathbf{w}||^2 + b = 0$$

and

$$||\mathbf{x} - \mathbf{v}|| = |\langle \mathbf{w}, \mathbf{x} \rangle + b|\, ||\mathbf{w}|| = |\langle \mathbf{w}, \mathbf{x} \rangle + b|$$

Hence, the distance is at most $|\langle \mathbf{w}, \mathbf{x} \rangle + b|$. Next, take any other point $\mathbf{u}$ on the hyperplane, thus $\langle \mathbf{w}, \mathbf{u} \rangle + b = 0$. We have

$$\begin{aligned} ||\mathbf{x} - \mathbf{u}||^2 &= ||\mathbf{x} - \mathbf{v} + \mathbf{v} - \mathbf{u}||^2 \\ &= ||\mathbf{x} - \mathbf{v}||^2 + ||\mathbf{v} - \mathbf{u}||^2 + 2\langle \mathbf{x} - \mathbf{v}, \mathbf{v} - \mathbf{u} \rangle \\ &\geq ||\mathbf{x} - \mathbf{v}||^2 + 2\langle \mathbf{x} - \mathbf{v}, \mathbf{v} - \mathbf{u} \rangle \\ &= ||\mathbf{x} - \mathbf{v}||^2 + 2(\langle \mathbf{w}, \mathbf{x} \rangle + b)\langle \mathbf{w}, \mathbf{v} - \mathbf{u} \rangle \qquad = ||\mathbf{x} - \mathbf{v}||^2 \end{aligned}$$

where the last equality is because $\langle \mathbf{w}, \mathbf{v} \rangle = \langle \mathbf{w}, \mathbf{u} \rangle = -b$. Hence, the distance between $\mathbf{x}$ and $\mathbf{u}$ is at least the distance between $\mathbf{x}$ and $\mathbf{v}$, which concludes our proof. □

The margin is defined as the minimal distance between a point and the hyperplane, thus

$$\min_i d(\mathbf{x}_i, P) = \min_i |\langle \mathbf{w}, \mathbf{x}_i \rangle + b| \qquad \mathbf{x}_i \in \boldsymbol{\mathcal{S}}$$

The set of points closest to the hyperplane (typically, the set is just the closest one, but multiple examples can be at the same distance from the hyperplane) are called *support vectors*.

9.2 Hard-SVM

Based on the preceding claim, the closest point in the training set to the separating hyperplane is $\min_{i \in [1,m]} |\langle \mathbf{w}, \mathbf{x}_i \rangle + b|$. Therefore, the Hard-SVM rule is

$$\operatorname*{arg\,max}_{(\mathbf{w},b):||\mathbf{w}||=1} \min_{i \in [1,m]} |\langle \mathbf{w}, \mathbf{x}_i \rangle + b| \;\text{ subject to }\; \forall i,\; y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle + b) > 0 \tag{9.1}$$

Whenever there is a solution to the preceding problem (i.e., we are in the separable case), we can write an equivalent problem as follows

$$\operatorname*{arg\,max}_{(\mathbf{w},b):||\mathbf{w}||=1} \min_{i \in [1,m]} y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle + b) \tag{9.2}$$

Next, we give another equivalent formulation of the Hard-SVM rule as a quadratic optimization problem. Note that a quadratic optimization problem is an optimization problem in which the objective function is a convex quadratic function, and the constraints are linear inequalities.

Algorithm: Hard-SVM

input: training set $\boldsymbol{S} = ((\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_m, y_m))$

solve:

$$(\mathbf{w}_0, b_0) = \arg\min_{(\mathbf{w},b)} ||\mathbf{w}||^2 \ \text{ subject to } \ y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle + b) \geq 1 \tag{9.3}$$

output: $\hat{\mathbf{w}} = \frac{\mathbf{w}_0}{||\mathbf{w}_0||}$, $\hat{b} = \frac{b_0}{||\mathbf{w}_0||}$

The following lemma shows that the output of Hard-SVM is the separating hyperplane with the largest margin. Intuitively, Hard-SVM searches for $\mathbf{w}$ of minimum norm among all the vectors that separate the data and for which $|\langle \mathbf{w}, \mathbf{x}_i \rangle + b| \geq 1$ for all i. In other words, we enforce the margin to be 1, but now the units in which we measure the margin scale with the norm of $\mathbf{w}$. Therefore, finding the largest margin halfspace boils down to finding $\mathbf{w}$ whose norm is minimal. Formally:

Lemma 9.2.1. *The output of Hard-SVM is a solution of Equation 9.2.*

Proof. Let $(\mathbf{w}^*, b^*)$ be a solution of Equation 9.2 and define the maring achieved by $(\mathbf{w}^*, b^*)$ to be $\gamma^* = \min_i y_i(\langle \mathbf{w}^*, \mathbf{x}_i \rangle + b^*)$. Therefore, for all i, we have

$$y_i(\langle \mathbf{w}^*, \mathbf{x}_i \rangle + b^*) \geq \gamma^*$$

or equivalently

$$y_i \left(\left\langle \frac{\mathbf{w}^*}{\gamma^*}, \mathbf{x}_i \right\rangle + \frac{b^*}{\gamma^*} \right) \geq 1$$

Hence, the pair $\left(\frac{\mathbf{w}^*}{\gamma^*}, \frac{b^*}{\gamma^*}\right)$ satisfies the conditions of the quadratic optimization problem given in Equation 9.3. Therefore, $||\mathbf{w}_0|| \leq ||\mathbf{w}^*/\gamma^*|| = 1/\gamma^*$. It follows that for all i,

$$y_i \left(\langle \hat{\mathbf{w}}, \mathbf{x}_i \rangle + \hat{b} \right) = \frac{1}{||\mathbf{w}_0||} y_i \left(\langle \mathbf{w}_0, \mathbf{x}_i \rangle + b_0 \right) \geq \frac{1}{||\mathbf{w}_0||} \geq \gamma^*$$

Since $||\hat{\mathbf{w}}|| = 1$ we obtain that $(\hat{\mathbf{w}}, \hat{b})$ is an optimal solution of Equation 9.2. □

9.2.1 The homogeneous case

It is often more convenient to consider homogeneous halfspaces, namely halfspaces that pass through the origin and are thus defined by $\text{sign}(\langle \mathbf{w}, \mathbf{x} \rangle)$, where the bias term b is set to be zero. Hard-SVM for homogeneous halfspaces amounts to solving

$$\min_{\mathbf{w}} \ ||\mathbf{w}||^2 \ \text{ subject to } \ \forall i, \ y_i \langle \mathbf{w}, \mathbf{x}_i \rangle \geq 1 \tag{9.4}$$

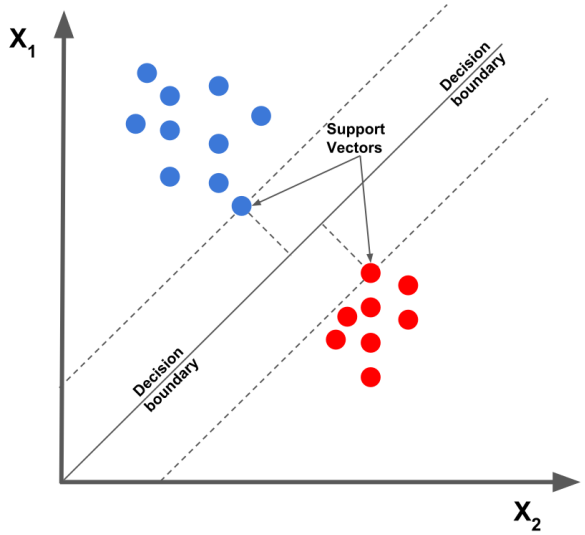
As we already discussed, we can reduce the problem of learning non-homogeneous halfspaces to the problem of learning homogeneous halfspaces by adding one more feature to each instance of x_i, thus increasing the dimensionality to $d + 1$.

However, the optimization problem given in Equation 9.3 does not regularize the bias term b. In contrast, if we learn a homogeneous halfspace in $\mathbb{R}^{d+1}$ using Equation 9.4, then we regularize the bias term (i.e., the $d + 1$ component of the weight vector $\mathbf{w}$) as well. However, regularizing b usually does not significantly affect the sample complexity.

9.2.2 Support vectors

The name "Support Vector Machine" stems from the fact that the solution of Hard-SVM, $\mathbf{w}_0$, is supported by (i.e., is in the linear span of) the examples that are exactly at a distance $1/||\mathbf{w}_0||$ from the separating hyperplane. These vectors are therefore called *support vectors*. In other words, support

vectors are the examples at the minimum distance from $\mathbf{w}_0$ and are the only training examples that matter for defining $\mathbf{w}_0$.



Theorem 9.2.2. *Let* $w_0 = \min_{\mathbf{w}} ||\mathbf{w}||^2$ *subject to* $\forall i,\ y_i \langle \mathbf{w}, \mathbf{x}_i \rangle \geq 1$ *be the hyperplane defined as before. Let* $I = \{i : |\langle \mathbf{w}_0, \mathbf{x}_i \rangle| = 1\}$ *be the set of indexes of support vectors. Then, there exist coefficients* $\alpha_1, \ldots, \alpha_m$ *such that*

$$\mathbf{w}_0 = \sum_{i \in I} \alpha_i \mathbf{x}_i$$

The examples $\{\mathbf{x}_i : i \in I\}$ are called *support vectors*. Thus, solving Hard-SVM is equivalent to finding the set of coefficients $\alpha_1, \ldots, \alpha_m$ that define the separating hyperplane when combined with the support vectors $\mathbf{x}_i$ for $i \in I$.

9.2.3 Duality

The Hard-SVM minimization problem defined by Equation 9.4 can be rewritten as a maximization problem

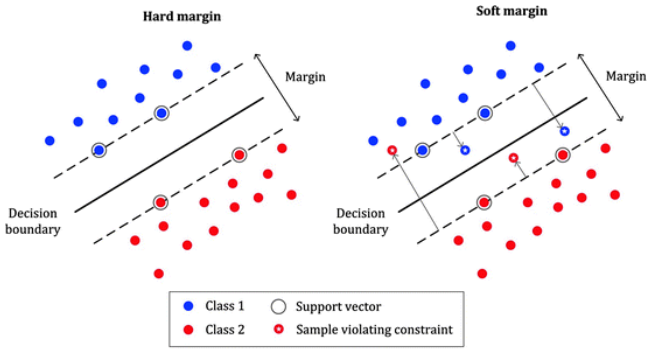
$$\max_{\alpha \in \mathbb{R}^m : \alpha \geq 0} \sum_{i=1}^{m} \alpha_i - \frac{1}{2} \sum_{i=1}^{m} \sum_{j=1}^{m} \alpha_i \alpha_j\, y_i y_j \langle \mathbf{x}_i,\, \mathbf{x}_j \rangle \tag{9.5}$$

called the *dual* problem. Note that the dual problem in Equation 9.5 only involves inner products between instances and does not require direct access to specific elements within an instance. As discussed in the next chapter, this property is crucial when implementing SVM with kernels.

9.3 Soft-SVM

The Hard-SVM formulation assumes that the training set is linearly separable, which is a relatively strong assumption. Therefore, soft-SVM can be viewed as a relaxation of the Hard-SVM rueìle that can be applied even if the training set is not linearly separable.

The optimization problem in Equation 9.3 enforces the hard constraints $y_i \left(\langle \mathbf{w}, \mathbf{x}_i \rangle + b\right) \geq 1$ for all i. For some examples in the training set, a natural relaxation allows the constraint to be violated. This can be modeled by introducing non-negative slack variables $\xi_1, \ldots, \xi_m \geq 0$ and replacing each constraint $y_i \left(\langle \mathbf{w}, \mathbf{x}_i \rangle + b\right) \geq 1$ by the constraint $y_i \left(\langle \mathbf{w}, \mathbf{x}_i \rangle + b\right) \geq 1 - \xi_i$. ξ_i measures how much the original hard constraint is violated. Soft-SVM jointly minimizes the norm of $\mathbf{w}$ (corresponding to the margin) and the average of ξ_i (corresponding to the violations of the constraints). The tradeoff between the two terms is controlled by a parameter λ. This leads to the Soft-SVM optimization problem:



Algorithm: Soft-SVM

input: training set $\boldsymbol{\mathcal{S}} = ((\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_m, y_m))$

parameter: $\lambda > 0$

solve:

$$\min_{\mathbf{w},b,\boldsymbol{\xi}} \left(\lambda \left|\left|\mathbf{w}\right|\right|^2 + \frac{1}{m} \sum_{i=1}^{m} \xi_i \right) \tag{9.6}$$

subject to

$$\forall i, \quad y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle + b) \geq 1 - \xi_i \quad \text{and} \quad \xi_i \geq 0$$

output: $\mathbf{w}$, b

We can rewrite Equation 9.6 as a regularized loss minimization problem. Let us introduce first the *hinge* loss function

$$l^{\text{hinge}}((\mathbf{w}, b), (\mathbf{x}, y)) = \max\{0, 1 - y\left(\langle \mathbf{w}, \mathbf{x} \rangle + b\right)\} \tag{9.7}$$

Given $(\mathbf{w}, b)$ and a training set $\boldsymbol{\mathcal{S}}$, the averaged hinge loss on $\boldsymbol{\mathcal{S}}$ is denoted by $L_{\boldsymbol{\mathcal{S}}}^{\text{hinge}}((\mathbf{w}, b))$. Now, consider the regularized loss minimization problem

$$\min_{\mathbf{w},b} \left(\lambda \left|\left|\mathbf{w}\right|\right|^2 + L_{\boldsymbol{\mathcal{S}}}^{\text{hinge}}((\mathbf{w}, b)) \right) \tag{9.8}$$

Lemma 9.3.1. *Equation 9.6 and Equation 9.8 are equivalent.*

Proof. Fix some $\mathbf{w}$, b and consider the minimization over $\boldsymbol{\xi}$ in Equation 9.6. Fix some i. Since ξ_i must be non-negative, the best assignment to ξ_i would be 0 if $y_i\left(\langle \mathbf{w}, \mathbf{x}_i \rangle + b\right) \geq 1$ and would be $1 - y_i\left(\langle \mathbf{w}, \mathbf{x}_i \rangle + b\right.$ otherwise. In other words, $\xi_i = l^{\text{hinge}}((\mathbf{w}, b), (\mathbf{x}, y))$ for all i. □

We, therefore, see that Soft-SVM falls into the paradigm of regularized loss minimization that we studied in the previous chapter. A Soft-SVM algorithm, that is, a solution for Equation 9.8, has a bias toward low norm separators. The objective function that we aim to minimize in Equation 9.8 penalizes not only for training errors but also for large norm.

It is often more convenient to consider Soft-SVM for learning a homogeneous halfspace, where the bias term b is set to zero, which yields the following optimization problem

$$\min_{\mathbf{w}} \left(\lambda \left|\left|\mathbf{w}\right|\right|^2 + L_{\boldsymbol{\mathcal{S}}}^{\text{hinge}}(\mathbf{w}) \right) \tag{9.9}$$

where

$$L_{\boldsymbol{\mathcal{S}}}^{\text{hingle}}(\mathbf{w}) = \frac{1}{m} \sum_{i=1}^{m} \max\{0, 1 - y_i \left\langle \mathbf{w}, \mathbf{x}_i \right\rangle\}$$

Chapter 10

Kernel Methods

The previous chapter described the SVM paradigm for learning halfspaces in high-dimensional feature spaces. This enables us to enrich the expressive power of halfspaces by mapping the data into a high-dimensional feature space and then learning a linear predictor.

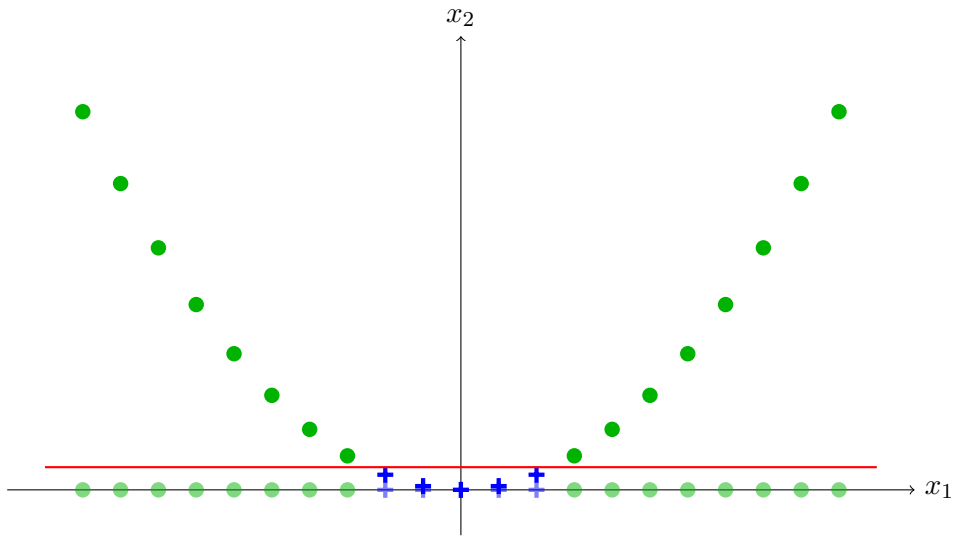
While this approach greatly extends the expressiveness of halfspace predictors, it raises both sample and computational complexity challenges. In the previous chapter, we tackled the sample complexity issue using the concept of margin. In this chapter, we tackle the computational complexity challenge using the method of *kernels*.

We start the chapter by describing embedding the data into a high-dimensional feature space. We then introduce the idea of kernels. A kernel is a type of similarity measure between instances. The special property of kernel similarities is that they can be viewed as inner products in some Hilbert spaces where the instance space is virtually embedded. Finally, we introduce the "kernel trick" that enables computationally efficient implementation of learning without explicitly handling the high dimensional representation of the domain instances.

10.1 Embeddings into feature spaces

The expressive power of halfspaces is rather restricted. For example, let the domain be the real line. Consider the domain points $\{-10, -9, -8, \ldots, 0, 1, \ldots, 9, 10\}$ where the labels are $+1$ for all x such that $|x| > 2$ and -1 otherwise. To make the class of halfspaces more expressive, we can first map the original instance space into another space (possibly of a higher dimension) and then learn a halfspace in that space. For example, consider the example mentioned previously. Instead of learning a halfspace in the original representation, let us first define a mapping $\psi : \mathbb{R} \to \mathbb{R}^2$ as $\psi(x) = (x, x^2)$.

We use the term *feature space* to denote the range of ψ. After applying the mapping ψ, data can be easly explained using the halfspace $h(x) = \text{sign}(\langle \mathbf{w}, \psi(x) \rangle - b)$, where $\mathbf{w} = (0, 1)$ and $b = 5$.



The basic paradigm is as follows:

1. Given some domain set $\boldsymbol{\mathcal{X}}$ and a learning task, choose a mapping $\psi : \boldsymbol{\mathcal{X}} \to \boldsymbol{\mathcal{F}}$, for some *feature space* $\boldsymbol{\mathcal{F}}$, that will usually be $\mathbb{R}^n$ fir sine n (however, the range of such a mapping can be any Hilbert space).

2. GIven a sequence of labeled examples $\boldsymbol{\mathcal{S}} = ((\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_m, y_m))$, create the image sequence $\hat{\boldsymbol{\mathcal{S}}} = ((\psi(\mathbf{x}_1), y_1), ldots, (\psi(\mathbf{x}_m), y_m))$.

3. Train a linear predictor h over $\hat{\boldsymbol{\mathcal{S}}}$.

4. Predict the label of a test point $\mathbf{x}$ to be $h(\psi(\mathbf{x}))$.

The success of this learning paradigm depends on choosing a good ψ for a given learning task: that is, a ψ that will make the image of the data distribution (close to being) linearly separable in the feature space, thus making the resulting algorithm a good learner for a given task. In addition, however, some generic mappings that enable us to enrich the class of halfspaces and extend its expressiveness are often used. One notable example is polynomial mappings, which are a generalization of the ψ we have seen in the previous example.

The bottom line of this discussion is that we can enrich the class of halfspaces by first applying a non-linear mapping ψ that maps the instance space into some feature space and then learning a halfspace in that feature space. However, if the range of ψ is a high dimensional space, we face two problems. First, the VC-dimension of halfspaces in $\mathbb{R}^n$ is $n + 1$, and therefore, if the range of ψ is very large, we need many more samples in order to learn a halfspace in the range of ψ. Second, from the computational point of view, performing calculations in the high dimensional space might be too costly.

The first issue can be tackled using a large margin paradigm, as we discussed in the previous chapter. In the following section, we address the computational issue.

10.2 The kernel trick

We have seen that embedding the input space into some high-dimensional feature space makes halfspace learning more expressive. However, the computational complexity of such learning may still pose a serious hurdle: computing linear separators over very high dimensional data may be computationally expensive.

The common solution to this concern is kernel-based learning. In this context, the term *kernels* is used to describe inner products in the feature space. Given an embedding ψ of some domain space $\boldsymbol{\mathcal{X}}$ into some Hilbert space, we define the kernel function

$$K(\mathbf{x}, \mathbf{x}') = \langle \psi(\mathbf{x}), \psi(\mathbf{x}') \rangle$$

One can think of K as a specifying similarity between instances and of the embedding ψ as mapping the domain set $\boldsymbol{\mathcal{X}}$ into a space where these similarities are realized as inner products. Many learning algorithms for halfspaces can be carried out based on the values of the kernel function over pairs of domain points.

The main advantage of such algorithms is that they implement linear separators in high-dimensional feature spaces without specifying points in that space or explicitly expressing the embedding ψ.

In the previous chapter, we saw that regularizing the norm of $\mathbf{w}$ yields a small sample complexity even if the dimensionality of the feature space is high. Interestingly regularizing the norm of $\mathbf{w}$ is also helpful in overcoming the computational problem. To do so, first note that all versions of the SVM optimization problem we have derived are instances of the following general problem:

$$\min_{\mathbf{w}} \left(f(\langle \mathbf{w}, \psi(\mathbf{x}_1) \rangle, \ldots, \langle \mathbf{w}, \psi(\mathbf{x}_m) \rangle) + R(||\mathbf{w}||) \right) \tag{10.1}$$

where $f : \mathbb{R}^m \to \mathbb{R}$ is an arbitrary function and $R : \mathbb{R}_+ \to \mathbb{R}$ is a monotonically nondecreasing function.

For example, Soft-SVM for homogeneous halfspaces (Equation 9.9) can be derived from Equation 10.1 by letting

$$f(a_1, \ldots, a_m) = \frac{1}{m}\sum_{i=1}^{m} \max\{0, 1 - y_i a_i\}$$
$$R(a) = \lambda a^2$$

Similarly, Hard-SVM for non-homogeneous halfspaces (Equation 9.3) can be derived from equation Equation 10.1 by letting

$$f(a_1, \ldots, a_m) = \begin{cases} 0 & \text{if } \exists b : y_i(a_i + b) \geq 1 \ \forall i \\ \infty & \text{otherwise} \end{cases}$$
$$R(a) = a^2$$

The following theorem shows that there exists an optimal solution of Equation 10.1 that lies in the span of $\{\psi(\mathbf{x}_1), \ldots, \psi(\mathbf{x}_m)\}$.

Theorem 10.2.1 (Representer theorem)**.** *Assume that ψ is a mapping from $\boldsymbol{\mathcal{X}}$ to a Hilbert space. Then, there exists a vector $\mathbf{a} \in \mathbb{R}^m$ such that $\mathbf{w} = \sum_{i=1}^{m} a_i \psi(\mathbf{x}_i)$ is an optimal solution of Equation 10.1. In other words, an optimal solution of Equation 10.1 is a linear combination of vectors in the feature space with a set of coefficients a_i.*

Proof. Let $\mathbf{w}^*$ be an optimal solution of Equation 10.1. Because $\mathbf{w}^*$ is an element of a Hilbert space, we can rewrite $\mathbf{w}^*$ as

$$\mathbf{w}^* = \sum_{i=1}^{m} a_i \psi(\mathbf{x}_i) + \mathbf{u}$$

where $\langle \mathbf{u}, \psi(\mathbf{x}_i) \rangle = 0$ for all i. Set $\mathbf{w} = \mathbf{w}^* - \mathbf{u}$. Clearly, $||\mathbf{w}^*||^2 = ||\mathbf{w}||^2 + ||\mathbf{u}||^2$ thus $||\mathbf{w}|| \leq ||\mathbf{w}^*||$. Since R is non-decreasing, we obtain that $R(||\mathbf{w}||)\ le R(||\mathbf{w}^*||)$. Additionally, for all i we have that

$$\langle \mathbf{w}, \psi(\mathbf{x}_i) \rangle = \langle \mathbf{w}^* - \mathbf{u}, \psi(\mathbf{x}_i) \rangle = \langle \mathbf{w}^*, \psi(\mathbf{x}_i) \rangle$$

Hence

$$f(\langle \mathbf{w}, \psi(\mathbf{x}_1) \rangle, \ldots, \langle \mathbf{w}, \psi(\mathbf{x}_m) \rangle) = f(\langle \mathbf{w}^*, \psi(\mathbf{x}_1) \rangle, \ldots, \langle \mathbf{w}^*, \psi(\mathbf{x}_m) \rangle)$$

We have shown that the objective function of Equation 10.1 at $\mathbf{w}$ cannot be larger than the objective at $\mathbf{w}^*$, and therefore, $\mathbf{w}$ is also an optimal solution. Since $\mathbf{w} = \sum_{i=1}^{m} a_i \psi(\mathbf{x}_i)$ we conclude our proof. □

Based on the representer theorem, we can optimize Equation 10.1 with respect to the coefficients $\mathbf{a}$ instead of $\mathbf{w}$ as follows. Writing $\mathbf{w} = \sum_{j=1}^{m} a_j \psi(\mathbf{x}_j)$ we have that for all i

$$\langle \mathbf{w}, \psi(\mathbf{x}_i) \rangle = \left\langle \sum_j a_j \psi(\mathbf{x}_j), \psi(\mathbf{x}_i) \right\rangle = \sum_{j=1}^{m} a_j \langle \psi(\mathbf{x}_j), \psi(\mathbf{x}_i) \rangle$$

Similarly

$$||\mathbf{w}||^2 = \left\langle \sum_j a_j \psi(\mathbf{x}_j), \sum_j a_j \psi(\mathbf{x}_j) \right\rangle = \sum_{i,j=1}^{m} a_i a_j \langle \psi(\mathbf{x}_j), \psi(\mathbf{x}_i) \rangle$$

Let $K(\mathbf{x}, \mathbf{x}') = \langle \psi(\mathbf{x}), \psi(\mathbf{x}') \rangle$ be a function that implements the kernel function with respect to the embedding ψ. Instead of solving Equation 10.1, we can solve the equivalent problem

$$\min_{\mathbf{a} \in \mathbb{R}^m} f\left(\sum_{j=1}^{m} a_j K(\mathbf{x}_j, \mathbf{x}_1), \ldots, \sum_{j=1}^{m} a_j K(\mathbf{x}_j, \mathbf{x}_m) \right) + R\left(\sqrt{\sum_{i,j=1}^{m} a_i a_j K(\mathbf{x}_j, \mathbf{x}_i)} \right) \tag{10.2}$$

We do not need direct access to elements in the feature space to solve the optimization problem given in Equation 10.2. To calculate the kernel function, we should only know how to calculate inner products in the feature space or equivalently. In fact, to solve Equation 10.2 we solely need to know the values of the $m \times n$ matrix G ($G_{i,j} = K(\mathbf{x}_i, \mathbf{x}_j)$) called the *Gram* matrix.

In particular, specifying the preceding to the Soft-SVM problem given in Equation 9.9, we can rewrite the problem as

$$\min_{\mathbf{a} \in \mathbb{R}} \left(\lambda \mathbf{a}^T G \mathbf{a} + \frac{1}{m} \sum_{i=1}^{m} \max\{0, 1 - y_i (G\mathbf{a})_i\} \right) \tag{10.3}$$

where $(G\mathbf{a})_i$ is the i-th element of the vector obtained by multiplying the Gram matrix G by the vector $\mathbf{a}$. Note that Equation 10.3 can be written as quadratic programming and thus can be solved efficiently.

Once we learn the coefficients $\mathbf{a}$, we can compute the prediction on a new instance by

$$\langle \mathbf{w}, \psi(\mathbf{x}) \rangle = \sum_{i=1}^{m} a_j K(\mathbf{x}_j, \mathbf{x})$$

The advantage of working with kernels rather than directly optimizing $\mathbf{w}$ in the feature space is that in some situations, the dimension of the feature space is extremely large, while implementing the kernel function is very simple.

10.2.1 Polynomial kernels

The k degree polynomial kernel is defined to be

$$K(\mathbf{x}, \mathbf{x}') = (1 + \langle \mathbf{x}, \mathbf{x}' \rangle)^k$$

Now we will show that this is indeed a kernel function. That is, we will show that there exists a mapping ψ from the original space to some higher dimensional space for which $K(\mathbf{x}, \mathbf{x}') = \langle \psi(\mathbf{x}), \psi(\mathbf{x}') \rangle$. For simplicity, denote $x_0 = x'_0 = 1$ (homogeneous trick). Then we have

$$\begin{aligned}
K(\mathbf{x}, \mathbf{x}') = (1 + \langle \mathbf{x}, \mathbf{x}' \rangle)^k &= (1 + \langle \mathbf{x}, \mathbf{x}' \rangle) \cdot \cdots \cdot (1 + \langle \mathbf{x}, \mathbf{x}' \rangle) \\
&= \left(\sum_{j=0}^{n} x_j x'_j \right) \cdot \cdots \cdot \left(\sum_{j=0}^{n} x_j x'_j \right) \\
&= \sum_{J \in \{0, 1, \ldots, n\}^k} \prod_{i=1}^{k} x_{J_i} x'_{J_i} \\
&= \sum_{J \in \{0, 1, \ldots, n\}^k} \prod_{i=1}^{k} x_{J_i} \prod_{i=1}^{k} x'_{J_i}
\end{aligned}$$

Now, if we define $\psi : \mathbb{R}^n \to \mathbb{R}^{(n+1)^k}$ such that for $J \in \{0, 1, \ldots, n\}^k$ there is an element of $\psi(\mathbf{x})$ that equals $\prod_{i=1}^{k} x_{J_i}$, we obtain that

$$K(\mathbf{x}, \mathbf{x}') = \langle \psi(\mathbf{x}), \psi(\mathbf{x}') \rangle$$

Since ψ contains all the monomials up to degree k, a halfspace over the range of ψ corresponds to a polynomial predictor of degree k over the original space. Hence, learning a halfspace with a k degree polynomial kernel enables us to learn polynomial predictors of degree k over the original space. Here, the complexity of implementing K is $O(n)$ while the dimension of the feature space is $O(n^k)$.

10.2.2 Radial basis functions

Let the original instance space be $\mathbb{R}$ and consider the mapping ψ where for each non-negative integer $n \geq 0$ there exists an element $\psi(x)_n$ that equals

$$\psi(x)_n = \frac{1}{\sqrt{n!}} e^{-\frac{x^2}{2}} x^n$$

Then

$$\begin{aligned}
\langle \psi(x), \psi(x') \rangle &= \sum_{n=0}^{\infty} \left(\frac{1}{\sqrt{n!}} e^{-\frac{x^2}{2}} x^n \right) \left(\frac{1}{\sqrt{n!}} e^{-\frac{(x')^2}{2}} (x')^n \right) \\
&= e^{-\frac{x^2+(x')^2}{2}} \sum_{n=0}^{\infty} \left(\frac{(xx')^n}{n!} \right) \\
&= e^{-\frac{||x-x'||^2}{2}}
\end{aligned}$$

Here the feature space is of infinite dimension while evaluating the kernel is very simple. More generally, given a scalar $\sigma > 0$, the Gaussian kernel is defined to be

$$K(\mathbf{x}, \mathbf{x}') = e^{-\frac{||\mathbf{x}-\mathbf{x}'||^2}{2\sigma}} \tag{10.4}$$

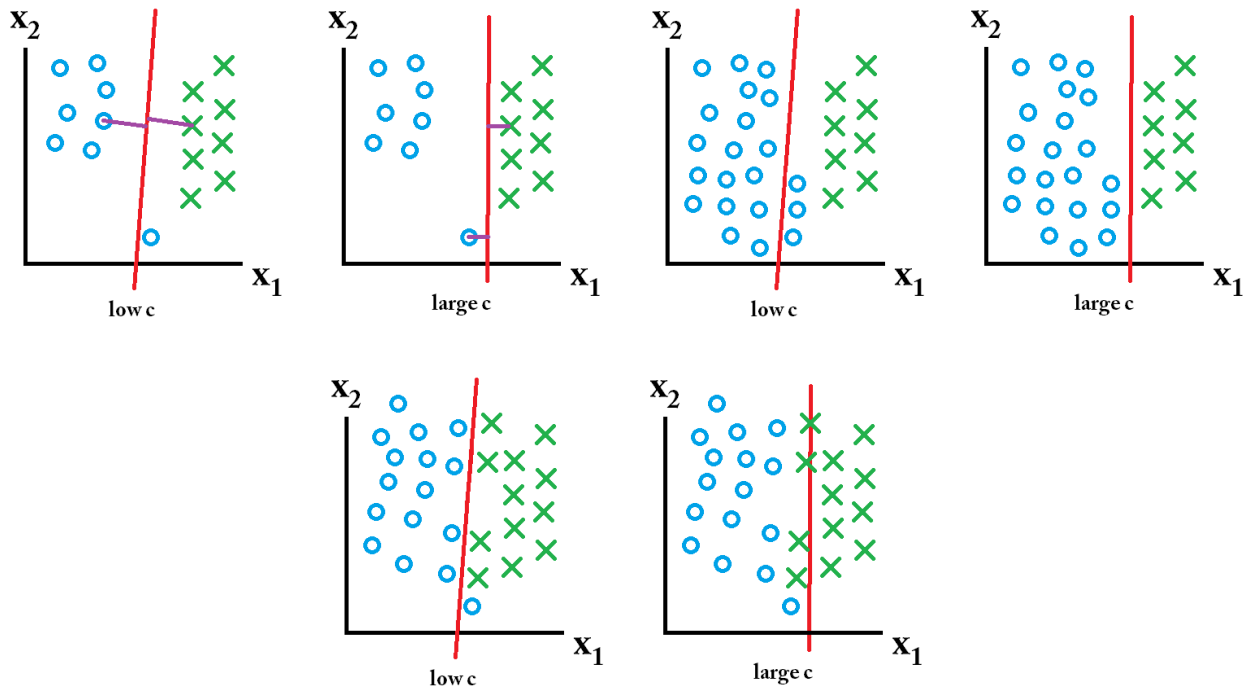
Intuitively, the Gaussian kernel sets the inner product in the feature space between $\mathbf{x}$ and $\mathbf{x}'$ to be close to zero if the instances are far away from each other (in the original domain) and close to 1 if they are close. The parameter σ controls the scale determining what we mean by "close." Using a Gaussian kernel, we can learn any polynomial predictor over the original space.

10.2.3 The λ parameter

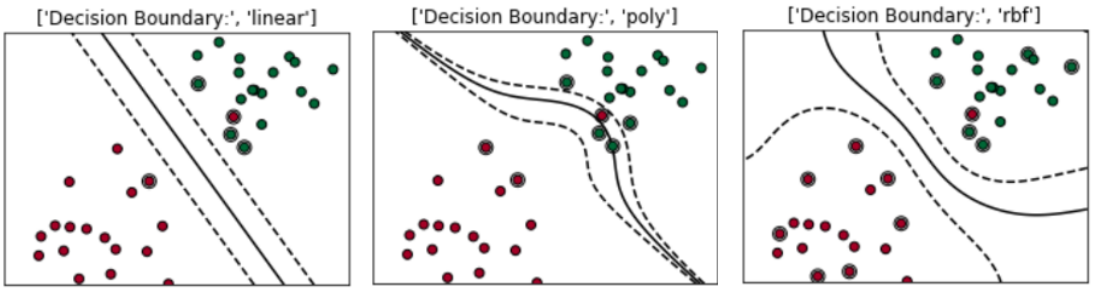
Recall the Soft-SVM optimization problem

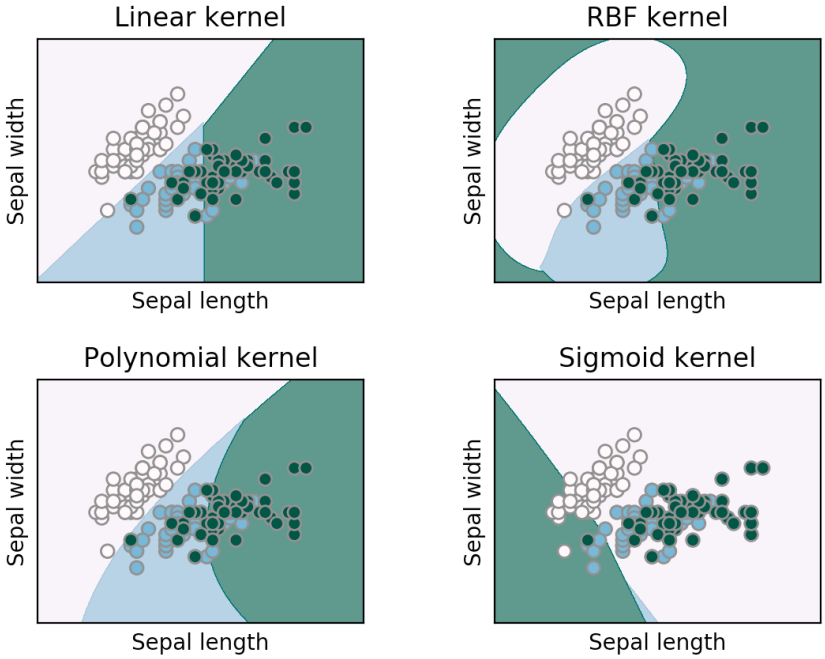
$$\min_{\mathbf{w}} \left(\lambda ||\mathbf{w}||^2 + L_{\mathcal{S}}^{\text{hinge}}(\mathbf{w}) \right) \tag{10.5}$$

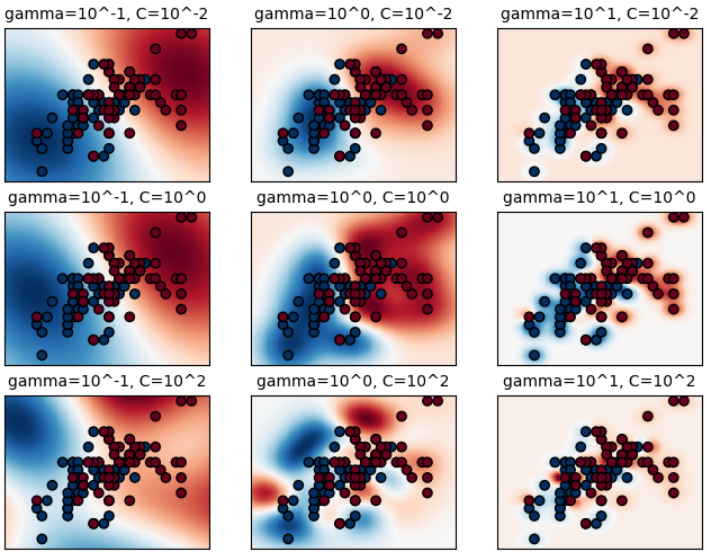
where the λ parameter controls the tradeoff between maximizing the margin and minimizing the violation of the constraints. In the figures, we use the convention $c = 1 - \lambda$ and show the different behaviors when setting a low and high value of λ.



10.2.4 Kernels comparison







Chapter 11

Stochastic Gradient Descent

Recall that the goal of learning is to minimize the risk function $L_{\mathcal{D}}(h) = \mathbb{E}_{z\sim\mathcal{D}}[l(h, z)]$. However, we cannot directly minimize the risk function since it depends on the unknown distribution $\mathcal{D}$. So far, we have discussed learning methods that depend on the empirical risk. That is, we first sample a training set $\mathcal{S}$ and define the empirical risk function $L_{\mathcal{S}}(h)$. Then, the learner picks a hypothesis based on the value of $L_{\mathcal{S}}(h)$. For example, the ERM rule tells us to pick the hypothesis that minimizes $L_{\mathcal{S}}(h)$ over the hypothesis class $\mathcal{H}$. In Chapter 8, we discussed the regularized risk minimization, in which we pick a hypothesis that jointly minimizes $L_{\mathcal{S}}(h)$ and a regularization function over h.

In this chapter, we describe and analyze a rather different learning approach called *Stochastic Gradient Descent* (SGD). We will focus on the family of convex learning problems and refer to hypotheses as vectors $\mathbf{w}$ that come from a convex hypothesis class $\mathcal{H}$. In SGD, we try to minimize the risk function $L_{\mathcal{D}}(\mathbf{w})$ directly using a gradient descent procedure. Gradient descent is an iterative optimization procedure in which, at each step, we improve the solution by taking a step along the negative of the gradient of the function to be minimized at the current point. Of course, in our case, we are minimizing the risk function, and since we do not know $\mathcal{D}$, we also do not know the gradient of $L_{\mathcal{D}}(\mathbf{w})$. SGD circumvents this problem by allowing the optimization procedure to take a step along a random direction as long as the expected direction value is the gradient's negative value. As we shall see, finding a random direction whose expected value corresponds to the gradient is rather simple, even though we do not know the underlying distribution $\mathcal{D}$.

The advantage of SGD in the context of convex learning problems over the regularized risk minimization learning rule is that SGD is an efficient algorithm that can be implemented in a few lines of code yet still enjoys the same sample complexity as the regularized risk minimization learning rule.

11.1 Gradient descent

Before we describe the stochastic gradient descent method, we would like to describe the standard gradient descent approach for minimizing a differentiable convex function $f(\mathbf{w})$.

The gradient of a differentiable function $f : \mathbb{R}^d \to \mathbb{R}$ at $\mathbf{w}$, denoted $\nabla f(\mathbf{w})$ is the vector of partial derivatives of f, namely

$$\nabla f(\mathbf{w}) = \left(\frac{\partial f(\mathbf{w})}{\partial w_1}, \ldots, \frac{\partial f(\mathbf{w})}{\partial w_d}\right)$$

Gradient descent is an iterative algorithm. We start with an initial value of $\mathbf{w}$, say $\mathbf{w}^{(1)} = 0$. Then, at each iteration, we take a step in the direction of the negative of the gradient at the current point. That is, the update step is

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta\, \nabla f(\mathbf{w}^{(t)}) \tag{11.1}$$

where $\eta > 0$ is the learning rate and controls the size of the steps. Intuitively, since the gradient points in the direction of the greatest rate of increase of f around $\mathbf{w}^{(t)}$, the algorithm makes a small step in the opposite direction, thus decreasing the value of the function. Eventually, after T iterations, the

algorithm outputs the averaged vector $\bar{\mathbf{w}} = \frac{1}{T}\sum_{t=1}^{T}\mathbf{w}^{(t)}$, the last vector $\mathbf{w}^{(T)}$ or the best performing vector $\arg\min_t f(\mathbf{w}^{(t)})$. However, taking the average turns out to be rather useful, especially when we generalize gradient descent to the stochastic case.

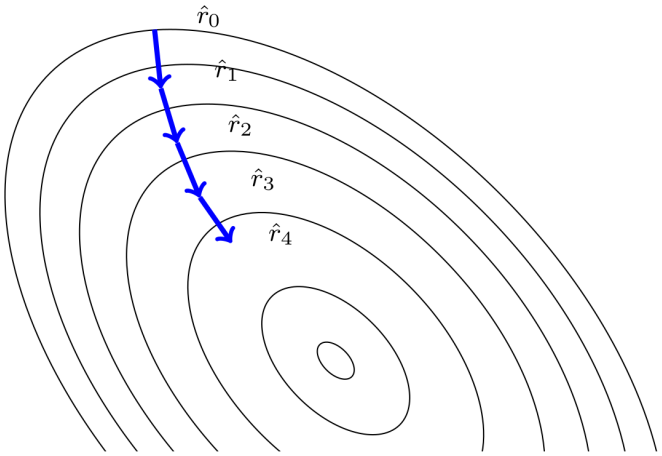
Another way to motivate gradient descent is by relying on the Taylor approximation. The gradient of f at $\mathbf{w}$ yields the first order Taylor approximation of f around $\mathbf{w}$ by $f(\mathbf{u}) \approx f(\mathbf{w}) + \langle \mathbf{u} - \mathbf{w}, \nabla f(\mathbf{w})\rangle$. When f is convex, this approximation lower bounds f

$$f(\mathbf{u}) \geq f(\mathbf{w}) + \langle \mathbf{u} - \mathbf{w}, \nabla f(\mathbf{w})\rangle$$

Therefore, for $\mathbf{w}$ close to $\mathbf{w}^{(t)}$ we have that $f(\mathbf{w}) \approx f(\mathbf{w}^{(t)}) + \langle \mathbf{w} - \mathbf{w}^{(t)}, \nabla f(\mathbf{w}^{(t)})\rangle$. Hence, we can minimize the approximation of $f(\mathbf{w})$. However, the approximation might become loose for $\mathbf{w}$, which is far from $\mathbf{w}^{(t)}$. Therefore, we would like to jointly minimize the distance between $\mathbf{w}$ and $\mathbf{w}^{(t)}$ and the approximation of f around $\mathbf{w}^{(t)}$. If the parameter η controls the tradeoff between the two terms, we obtain the update rule

$$\mathbf{w}^{(t+1)} = \arg\min_{\mathbf{w}} \frac{1}{2}||\mathbf{w} - \mathbf{w}^{(t)}||^2 + \eta \left(f(\mathbf{w}^{(t)}) + \langle \mathbf{w} - \mathbf{w}^{(t)}, \nabla f(\mathbf{w}^{(t)})\rangle \right) \tag{11.2}$$

Solving the preceding by taking the derivative with respect to $\mathbf{w}$ and comparing it to zero yields the same update rule as in Equation 11.1.



11.1.1 Accuracy and convergence

To analyze the convergence rate of the GD algorithm, we limit ourselves to the case of convex-Lipschitz functions (recall Definition 8.3.1). Let $\mathbf{w}^*$ be the vector that minimizes $f(\mathbf{w})$ (i.e. the ERM solution) and let B be an upper bound on $||\mathbf{w}^*||$.

Theorem 11.1.1. *Let f be a convex, ρ-Lipschitz function, and let $\mathbf{w}^* \in \arg\min_{\{\mathbf{w}:||\mathbf{w}||\leq B\}} f(\mathbf{w})$. If the run the gradient descent algorithm on f for T steps with*

$$\eta = \sqrt{\frac{B^2}{\rho^2 T}}$$

then the output vector $\bar{\mathbf{w}}$ satisfies

$$f(\bar{\mathbf{w}}) - f(\mathbf{w}^*) \leq \frac{B\rho}{\sqrt{T}}$$

Furthermore, for every $\epsilon > 0$, to achieve $f(\bar{\mathbf{w}}) - f(\mathbf{w}^*) \leq \epsilon$ it suffices to run the gradient descent algorithm for a number of iterations that satisfies

$$T \geq \frac{B^2\rho^2}{\epsilon^2}$$

11.2 Stochastic gradient descent

In stochastic gradient descent, we do not require the update direction to be based exactly on the gradient. Instead, we allow the direction to be a random vector and only require that its *expected value* at each iteration will equal the gradient direction.

Algorithm: Stochastic gradient descent

parameters: Scalar $\eta > 0$, integer $T > 0$

initialize: $\mathbf{w}^{(1)} = 0$

for $t = 1, \ldots, T$ **do**

  choose $\mathbf{v}_t$ at random from a distribution such that $\mathbb{E}[\mathbf{v}_t \mid \mathbf{w}^{(t)} = \nabla f(\mathbf{w}^{(t)})$

  update $\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta\, \mathbf{v}_t$

end

output: $\bar{\mathbf{w}} = \frac{1}{T} \sum_{t=1}^{T} \mathbf{w}^{(t)}$

11.2.1 SGD vs GD

Let us consider the machine learning ERM setting to compare the stochastic gradient descent method with the standard gradient descent method. That is, we need to find $\mathbf{w}$ that minimizes $L_{\boldsymbol{\mathcal{S}}}(\mathbf{w})$.

The standard gradient descent algorithm must process all the training set at each iteration, as $\nabla f(\mathbf{w})$ depends on all the m pairs $(\mathbf{x}_i, y_i) \in \boldsymbol{\mathcal{S}}$. This leads to a long computation time if the training set is reasonably large, as in most real-world ML problems.

Instead, the stochastic gradient descent algorithm needs to pick $\mathbf{v}_t$ such that $\mathbb{E}[\mathbf{v}_t \mid \mathbf{w}^{(t)}] = \nabla f(\mathbf{w}^{(t)}) = \nabla L_{\boldsymbol{\mathcal{S}}}(\mathbf{w}^{(t)})$. Thus, we can pick a random instance $(\mathbf{x}_i, y_i) \in \boldsymbol{\mathcal{S}}$ and let $\mathbf{v}_t = \nabla l(\mathbf{w}^{(t)}, (\mathbf{x}_i, y_i))$ and processing just a single sample is indeed much faster.

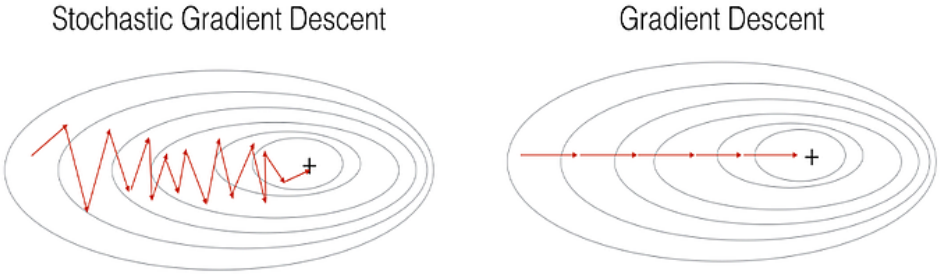


Figure 1 : SGD vs GD
"+" denotes a minimum of the cost. SGD leads to many oscillations to reach convergence. But each step is a lot faster to compute for SGD than for GD, as it uses only one training example (vs. the whole batch for GD).

The tradeoff is that the stochastic gradient descent algorithm is less stable than the standard one. The overall trajectory is noisier, but note that this behavior could be helpful to jump out of possible local minima. A clever choice could be to return the average of a set of final iterations instead of returning the average of all of them. This is because the bigger initial fluctuations may interfere with the final result. Moreover, another possibility is to use an adaptive learning rate, namely setting the η parameter to be bigger at the beginning and becoming smaller as the algorithm reaches convergence. This way, the first iterations are fast, and the algorithm identifies the optimal region, while the last iterations are precise to find the best solution. Finally, the last possible improvement we see is called *mini-batch* gradient descent and represents a tradeoff between the two extreme cases, the standard gradient descent algorithm and the stochastic gradient descent algorithm. In mini-batch gradient descent, we introduce another parameter, k, that regulates the number of samples used to compute the gradient. Usually, this parameter is rather small. Thus, the behavior is similar to one of the stochastic methods while being more stable. It is commonly used to train deep neural networks.

11.3 Learning with SGD

We have introduced and analyzed the SGD algorithm for general convex functions. Now we shall consider its applicability to learning tasks.

11.3.1 SGD for Risk Minimization

Recall that in learning, we face the problem of minimizing the risk function

$$L_{\mathcal{D}}(\mathbf{w}) = \mathbb{E}_{z\sim\mathcal{D}}[l(\mathbf{w}, z)]$$

We have seen the method of empirical risk minimization, where we minimize the empirical risk $L_{\mathcal{S}}(\mathbf{w})$. Stochastic gradient descent allows us to take a different approach and minimize $L_{\mathcal{D}}(\mathbf{w})$ directly. However, since we do not know $\mathcal{D}$, we cannot simply compute $\nabla L_{\mathcal{D}}(\mathbf{w}^{(t)})$ and minimize it with the standard gradient descent method. With SGD, however, all we need is to find an unbiased estimate of the gradient of $L_{\mathcal{D}}(\mathbf{w})$, that is, a random vector whose conditional expected value is $\nabla L_{\mathcal{D}}(\mathbf{w}^{(t)})$.

For simplicity, let us first consider the case of differentiable loss functions. Hence, the risk function $L_{\mathcal{D}}$ is also differentiable. The construction of the random vector $\mathbf{v}_t$ will be as follows. First, we sample $z \sim \mathcal{D}$, then we define $\mathbf{v}_t$ to be the gradient of the function $l(\mathbf{w}, z)$ with respect to $\mathbf{w}$ at the point $\mathbf{w}^{(t)}$. Then, by the linearity of the gradient and expected value, we have

$$\mathbb{E}[\mathbf{v}_t \mid \mathbf{w}^{(t)}] = \mathbb{E}_{z\sim\mathcal{D}}[\nabla l(\mathbf{w}^{(t)}, z)] = \nabla \mathbb{E}_{z\sim\mathcal{D}}[l(\mathbf{w}^{(t)}, z)] = \nabla L_{\mathcal{D}}(\mathbf{w}^{(t)})$$

The gradient of the loss function $l(\mathbf{w}, z)$ at $\mathbf{w}^{(t)}$ is, therefore, an unbiased estimate of the gradient of the risk function $L_{\mathcal{D}}(\mathbf{w}^{(t)})$ and is easily constructed by sampling a fresh example $z \sim \mathcal{D}$ at each iteration t. Note that if we do not sample a fresh example at each iteration, then we have a *biased* estimate of the risk function $L_{\mathcal{D}}(\mathbf{w}^{(t)})$!

To summarize, the stochastic gradient descent framework for minimizing the risk is as follows.

Algorithm: Stochastic gradient descent for ERM

parameters: Scalar $\eta > 0$, integer $T > 0$

initialize: $\mathbf{w}^{(1)} = 0$

for $t = 1, \ldots, T$ **do**

  sample $z \sim \mathcal{D}$

  pick $\mathbf{v}_t \in \nabla l(\mathbf{w}^{(t)}, z)$

  update $\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta\, \mathbf{v}_t$

end

output: $\bar{\mathbf{w}} = \frac{1}{T}\sum_{t=1}^{T} \mathbf{w}^{(t)}$

We shall now use our analysis of SGD to obtain a sample complexity analysis for learning the convex-Lipschitz problem. Theorem 11.1.1 yields the following corollary.

Corollary 11.3.0.1. *Consider a convex-Lipschitz-bounded learning problem with parameters ρ, B. Then, for every $\epsilon > 0$, if we run the SGD method for minimizing $L_{\mathcal{D}}(\mathbf{w})$ with a number of iterations (that also is the number of fresh examples we need to sample)*

$$T \geq \frac{B^2\rho^2}{\epsilon^2}$$

and with $\eta = \sqrt{\frac{B^2}{\rho^2 T}}$, then the output of SGD satisfies

$$\mathbb{E}[L_{\mathcal{D}}(\bar{\mathbf{w}})] \leq \min_{\mathbf{w}\in\mathcal{H}} L_{\mathcal{D}}(\mathbf{w}) + \epsilon$$

Interestingly, the required sample complexity is of the same order of magnitude as the sample complexity guarantee we derived for regularized loss minimization. Moreover, the sample complexity of SGD is even better by a factor of 8.

11.3.2 SGD for Support Vector Machines

In this section, we describe a very simple algorithm for solving the optimization problem of Soft-SVM, namely

$$\min_{\mathbf{w}} \left(\frac{\lambda}{2}||\mathbf{w}||^2 + \frac{1}{m}\sum_{i=1}^{m} \max\{0,\, 1 - y\langle \mathbf{w},\, \mathbf{x}_i\rangle\} \right) \tag{11.3}$$

We rely on the SGD framework for solving regularized loss minimization problems variant for λ-strongly convex functions (not part of the course).

We can rewrite the update rule of SGD as

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta\, \mathbf{v}_t \longrightarrow -\frac{1}{\lambda t}\sum_{j=1}^{t} \mathbf{v}_j \tag{11.4}$$

where $\mathbf{v}_j$ is a sub-gradient of the loss function at $\mathbf{w}^{(j)}$ on the random example chosen at the j-th iteration. For the hinge loss, given an example $(\mathbf{x},\, y)$, we can choose $\mathbf{v}_j$ to be

$$\mathbf{v}_j^{\text{hinge}} = \begin{cases} \vec{0} & \text{if } \; y\langle \mathbf{w}^{(j)},\, \mathbf{x}\rangle \geq 1 \\ -y\mathbf{x} & \text{otherwise} \end{cases}$$

Denoting $\vec{\theta}^{(t)} = -\sum_{j<t} \mathbf{v}_j$ we obtain the following procedure.

Algorithm: Stochastic gradient descent for Soft-SVM

parameters: Integer $T > 0$

initialize: $\vec{\theta}^{(t)} = \vec{0}$

for $t = 1, \ldots, T$ **do**

  Let $\mathbf{w}^{(t)} = \frac{1}{\lambda t}\vec{\theta}^{(t)}$

  Choose i uniformly at random from $[1,\, m]$

  if $y_i\langle \mathbf{w}^{(t)},\, \mathbf{x}_i\rangle < 1$ **then**

    Set $\vec{\theta}^{(t+1)} = \vec{\theta}^{(t)} + y_i\mathbf{x}_i$

  end

  else

    Set $\vec{\theta}^{(t+1)} = \vec{\theta}^{(t)}$

  end

end

output: $\bar{\mathbf{w}} = \frac{1}{T}\sum_{t=1}^{T} \mathbf{w}^{(t)}$

11.4 Final considerations on SGD

11.4.1 Learning rate

First, we stress that the selection of the learning rate η is a critical point: if we choose η to be too small, the optimization is stable, but the convergence can be *really* slow. On the other hand, a too-large learning rate makes the convergence fast, but the overall algorithm stability could be lacking.

A simple yet effective workaround is to implement an adaptive learning rate. For example, we could set the η parameter to be progressively reduced according to a pre-defined schedule. For example, in the RLM and SVM optimization problems, we use $\eta_t = \frac{1}{\lambda t}$. However, these approaches require rules and thresholds to be defined in advance and thus are difficult to adapt to different problems, as they are to be selected for the specific task at hand.

Furthermore, considering $\eta \in \mathbb{R}_+$ means that the same learning rate is applied to all parameter updates (i.e., we treat all features as equal). However, the various parameters usually have different

behaviors, and a single learning rate could be too fast for some and too slow for others. Sometimes it is thus better to have a different learning rate for each component of the weights $\mathbf{w}$ vector.

11.4.2 Momentum

Usually, stochastic gradient descent has troubles in areas where the surface curves much more steeply in one dimension than the others. This surface behavior is typical around local minima.

A possible optimization of the SGD update rule is the linear combination of the previous gradient, and a new one is adding a new parameter $\gamma \sim 0.9$, called momentum, such that

$$\mathbf{v}_t = \gamma\, \mathbf{v}^{(t-1)} + (1-\gamma)\, \nabla L_{\boldsymbol{\mathcal{D}}}(\mathbf{w}^{(t)})$$

and use it in

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \mathbf{v}_t$$

This parameter helps accelerate the convergence of SGD in the relevant direction and dampens its typical oscillations. Using momentum is like pushing a ball down a hill. The ball accumulates momentum as it rolls downhill, becoming faster and faster. The same thing happens to our parameter updates: the momentum term increases for dimensions whose gradients point in the same direction and reduces updates for dimensions whose gradients change direction. As a result, we gain faster convergence and reduced oscillations.

11.4.3 Advanced SGD schemes

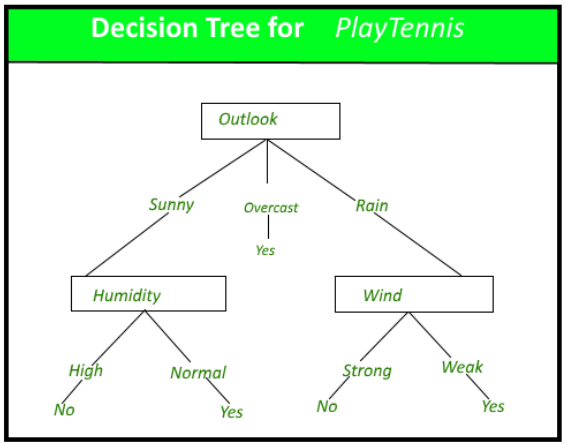
- **ADAgrad:** adapts the learning rate for each parameter independently
  - it performs smaller updates for parameters associated with frequently occurring features
  - it performs larger updates for parameters associated with infrequent features
- **ADAdelta:** improved version of ADAgrad
- **RMSprop:** improved version of ADAgrad
- **Adam (Adaptive Moment Estimation):** it combines ADAgrad and momentum, also introducing an adaptive learning rate for each parameter
  - whereas momentum can be seen as a ball running down a slope, Adam behaves like a heavy ball with friction, which thus prefers flat minima in the error surface

Chapter 12

Decision Trees

A decision tree is a predictor $h : \boldsymbol{\mathcal{X}} \to \boldsymbol{\mathcal{Y}}$ that predicts the label associated with an instance $\mathbf{x}$ by traveling from a tree's root node to a leaf. For simplicity, we focus on the binary classification setting (*binary trees*), namely $\boldsymbol{\mathcal{Y}} = \{0, 1\}$. However, note that decision trees can also be applied to other prediction problems.

At each node on the root-to-leaf path, the successor child is chosen based on splitting the input space. Usually, this splitting is based on one of the features of $\mathbf{x}$ or a predefined set of splitting rules. In addition, a leaf contains a specific label.



A popular splitting rule at internal nodes of the tree is based on thresholding the value of a single feature. That is, we move to the right or the left child of the node based on $\mathbb{I}_{[x_i<\theta]}$, where $i = 1, \ldots, d$ is the index if the relevant feature and $\theta \in \mathbb{R}$ is the threshold. In such cases, we can think of a decision tree as splitting the instance space $\boldsymbol{\mathcal{X}} = \mathbb{R}^d$ into cells, where each leaf of the tree corresponds to one cell. A tree with k leaves can shatter a set of k instances. Hence, if we allow decision trees of arbitrary size, we obtain a hypothesis class of infinite VC dimension. Such an approach can easily lead to overfitting.

12.1 Decision tree algorithms

Since theoretical bounds lead to computationally hard solutions, practical decision tree learning algorithms are based on heuristics such as a greedy approach, where the tree is constructed gradually, and locally optimal decisions are made at the construction of each node. Such algorithms cannot guarantee to return the globally optimal decision tree but tend to work reasonably well in practice.

A general framework for growing a decision tree is as follows. We start with a tree with a single leaf (that is also the root) and assign this leaf a label according to a majority vote among all labels

over the training set. We now perform a series of iterations. On each iteration, we examine the effect of splitting a single leaf. First, we define some "gain" measure that quantifies the improvement due to this split. Then, among all possible splits, we either choose the one that maximizes the gain and perform it (thus, the leaf becomes a node and splits into two children) or choose not to split the leaf at all.

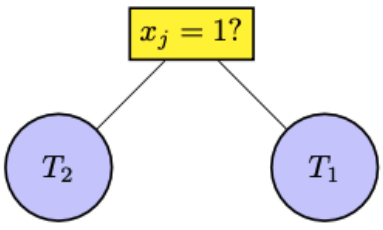
12.1.1 Iterative dichotomizer 3

We describe the algorithm for the case of binary features, namely $\boldsymbol{\mathcal{X}} = \{0, 1\}^d$, and therefore the splitting rules are of the form $\mathbb{I}_{[x_i=1]}$ for some feature $i = 1, \ldots, d$.

The algorithm works by recursive calls, the first one being $\texttt{ID3}(\boldsymbol{\mathcal{S}}, [1, d])$, and returns a decision tree. In the pseudo-code below, we use a call to a procedure $\texttt{Gain}(\boldsymbol{\mathcal{S}}, i)$, which receives a training set $\boldsymbol{\mathcal{S}}$ and an index i and evaluates the gain of a split of the tree according to the i-th feature.

Algorithm: Iterative dichotomizer 3

input: Training set $\boldsymbol{\mathcal{S}}$, feature subset $A \subset [1, d]$
if *all examples in* $\boldsymbol{\mathcal{S}}$ *are labeled as* $y = 1$ **then**
| **output:** return a leaf 1
end
if *all examples in* $\boldsymbol{\mathcal{S}}$ *are labeled as* $y = 0$ **then**
| **output:** return a leaf 0
end
if $A = \emptyset$; `// all features tested`
then
| **output:** return a leaf whose value is equal to the majority of labels in $\boldsymbol{\mathcal{S}}$
end
else
| Let $j = \arg\max_{i \in A} \texttt{Gain}(\boldsymbol{\mathcal{S}}, i)$; `// find which split leads to the maximum gain`
| **if** *all examples in* $\boldsymbol{\mathcal{S}}$ *have the same label* **then**
| | **output:** return a leaf whose value equals the majority of labels in $\boldsymbol{\mathcal{S}}$
| **end**
| **else**
| | let T_1 be the tree returned by $\texttt{ID3}(\{(\mathbf{x}, y) \in \boldsymbol{\mathcal{S}} : x_j = 1\}, A/\{j\})$
| | let T_2 be the tree returned by $\texttt{ID3}(\{(\mathbf{x}, y) \in \boldsymbol{\mathcal{S}} : x_j = 0\}, A/\{j\})$
| | **output:** return the tree:



| **end**
end

12.1.2 Gain measure

Different algorithms use different implementations of $\texttt{Gain}(\boldsymbol{\mathcal{S}}, i)$.

Train error: The simplest definition of gain is the decrease in training error.

Gini index: Another popular gain measure is the Gini index

$$G = \sum_{i=1}^{C} p(i)(1 - p(i))$$

where i is the index over the possible classes and $p(i)$ is the probability associated to the i-th class.

The Gini index is a measure of statistical dispersion in a frequency distribution. In a binary classification task, $G = 0$ if all the samples are in a single class, and $G = 1/2$ if the samples are equally divided into the two classes. We thus want to minimize the Gini index, as $G = 0$ means perfect classification. Usually, in practice, we define a lower threshold on the Gini index, and every time such a value is reached, we perform a split. Each split should lead to a smaller Gini index.

Intuitively, the Gini index measures how "pure" the distribution is after the split. The lower the index, the purer the distribution. Note that the Gini index is a smooth and concave upper bound if the training error, i.e., a good approximation.

Threshold-based splitting rules for real-valued features: We can extend the previous results to real-valued features by creating a set of thresholds and testing all the various combinations of features and thresholds.

12.1.3 Pruning

The `ID3` algorithm suffers from a big problem: the returned tree will usually be very large. Such trees may have low empirical risk, but their true risk will tend to be high. One solution is to limit the number of iterations of `ID3`, leading to a tree with a bounded number of nodes. Another common solution is to *prune* the tree after it is built, hoping to reduce it to a much smaller tree, but still with a similar empirical error.

Usually, the pruning is performed by a bottom-up walk on the tree. For example, each node might be replaced with one of its sub-trees or with a leaf, based on some bound or estimate of $L_{\mathcal{D}}(h)$.

Algorithm: Iterative dichotomizer 3

input:

tree T

function $f(T, m)$; `// bound/estimate of` $L_{\mathcal{D}}(T)$ `based on a` m`-sized sample`

foreach *node j in a bottom-up walk on T* **do**

find T' which minimizes $f(T', m)$ where T' is any of the following:

the current tree after replacing node j with a leaf 1

the current tree after replacing node j with a leaf 0

the current tree after replacing node j with its left subtree

the current tree after replacing node j with its right subtree

the current tree

let $T := T'$

end

12.2 Random forests

As mentioned before, the class of decision trees of arbitrary size has an infinite VC dimension. We, therefore, restricted the size of the decision tree. Another way to reduce the danger of overfitting is by constructing an ensemble of trees.

A *random forest* is a classifier consisting of a collection of decision trees, where each tree is constructed by applying an algorithm A on the training set $\boldsymbol{\mathcal{S}}$ and an additional random vector θ, where θ is sampled i.i.d. from some distribution. A majority vote over the individual trees' prediction obtains the random forest's prediction.

To specify a particular random forest, we need to define the algorithm A and the distribution over θ. We generate θ as follows. First, we take a random subsample from $\boldsymbol{\mathcal{S}}$ with replacements. Namely, we sample a new training set $\boldsymbol{\mathcal{S}}'$ of size m (same size, $\boldsymbol{\mathcal{S}}'$ is not a subset of $\boldsymbol{\mathcal{S}}$) using the uniform distribution over $\boldsymbol{\mathcal{S}}$. Second, we construct a sequence $I_1, I_2, \ldots$ where each I_t is a subset of $[1, d]$ of size k which is generated by sampling uniformly at random elements from $[1, d]$. All these random variables form the vector θ. Then, the algorithm A grows a decision tree (e.g., using `ID3`) based on the sample $\boldsymbol{\mathcal{S}}'$, where the algorithm is restricted to each splitting stage of the algorithm choosing a feature that maximizes gain from the set I_t. Intuitively, if k is small, this restriction may prevent overfitting.

12.2.1 Bootstrap aggregation

Decision trees are very sensitive to the particular dataset they are trained on. Therefore, small changes to the training set can result in significantly different tree structures. Random forest takes advantage of this by allowing each tree to sample randomly with replacement from the dataset, resulting in different training sets and thus producing different decision trees. This process is known as *bagging*.

12.2.2 Feature randomness

In a standard decision tree, when it is time to split a node into two children, we consider every possible feature and pick the one that produces the greater gain. Instead, each tree can pick only from a random subset of features in random forests. This forces even more variation amongst the trees in the model and ultimately results in lower correlation across trees and more diversification.

Chapter 13

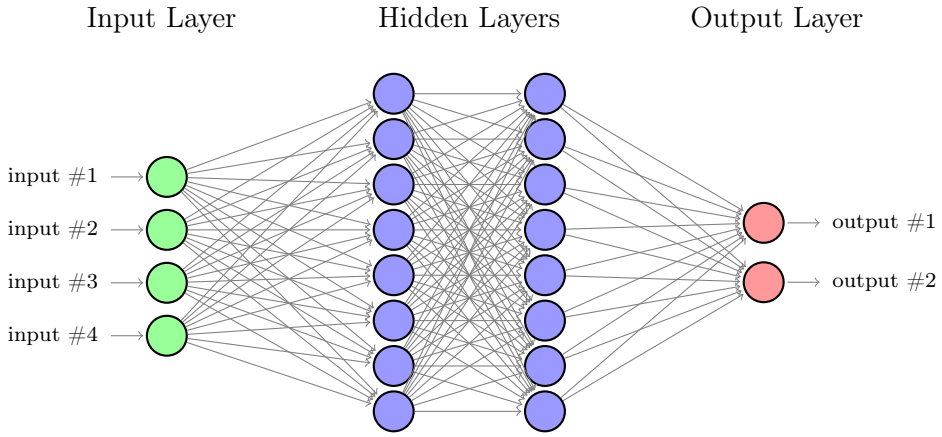
Neural Networks

An artificial neural network is a model of computation inspired by the structure of neural networks in the brain. In simplified models of the brain, it consists of many essential computing devices (*neurons*) connected in a complex communication network through which the brain can carry out highly complex computations. Artificial neural networks are formal computation constructs modeled after this computation paradigm.

A neural network can be described as a directed graph whose nodes correspond to neurons and edges correspond to links between them. Each neuron receives as input a weighted sum of the outputs of the neurons connected to its incoming edges. We focus on *feed-forward* networks in which the underlying graph does not contain cycles.

13.1 Feed-forward neural networks

The idea behind neural networks is that many neurons can be joined together by communication links to carry out complex computations. It is common to describe the structure of a neural network as a graph whose nodes are the neurons, and each directed edge in the graph links the output of some neuron to the input of another neuron.



A feed-forward neural network is described by a directed acyclic graph $G(V, E)$ and a weight function over the edges $\mathbf{w} : E \to \mathbb{R}$. The nodes of the graph correspond to neurons. Every single neuron is modeled as a simple scalar function $\sigma : \mathbb{R} \to \mathbb{R}$. We will focus on three possible functions for σ: the sign function $\sigma(a) = \text{sign}(a)$, the threshold function $\sigma(a) = \mathbb{I}_{[a>0]}$ and the sigmoid function $\sigma(a) = 1/(1 + \exp(-a))$, which is a smooth approximation to the threshold function. We call σ the *activation function* of the neuron. Each edge in the graph links some neuron's output to another neuron's input. A neuron's input is obtained by taking the weighted sum of the outputs of all the neurons connected to it, where the weighting is according to w.

To simplify the description of the calculation performed by the network, we further assume that the network is organized in *layers*. That is, the set of nodes can be decomposed into a union of non-empty

disjoint subsets $V = \bigcup_{t=0}^{T} V_t$, such that every edge in E connects some node in V_{t-1} to some node in V_t, for some $t \in [1, T]$. The first layer V_0 is called the input layer. It contains $n+1$ neurons, where n is the dimensionality of the input space. For every $i \in [1, n]$, the output of neuron i in V_0 is simply x_i. The last neuron in V_0 is called the "constant" neuron, which always outputs 1 (we incorporate the bias, homogeneous trick). In some notations, we add the constant neuron to each layer to make it act like a bias. We denote by $v_{t,i}$ the i-th neuron of the t-th layer and by $o_{t,i}(\mathbf{x}) = x_i$ the output of $v_{t,i}$ when the network is fed with the input vector $\mathbf{x}$. Therefore, for $i \in [1, n]$ we have $o_{0,i}(\mathbf{x}) = x_i$ and for $i = i + 1$ we have $o_{0,i}(\mathbf{x}) = 1$.

We now proceed with the calculation in a layer-by-layer manner. Suppose we have computed the outputs of the neurons at layer t. Then, we can calculate the outputs of neurons at layer $t+1$ as follows.

Fix some $v_{t+1,\,j} \in V_{t+1}$. Let $a_{t+1,\,j}(\mathbf{x})$ denote the input to $v_{t+1,\,j}$ when the network is fed with the input vector $\mathbf{x}$. Then

$$a_{t+1,\,j}(\mathbf{x}) = \sum_{r\,:\,(v_{t,\,r},\,v_{t+1,\,j}) \in E} w((v_{t,\,r},\, v_{t+1,\,j}))\, o_{t,\,r}(\mathbf{x})$$

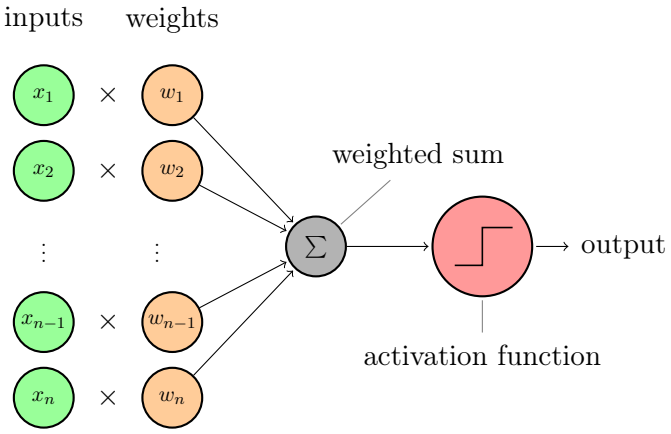
and thus

$$o_{t+1,\,j}(\mathbf{x}) = \sigma(a_{t+1,\,j}(\mathbf{x}))$$

That is, the input to $v_{t+1,\,j}$ is a weighted sum of the outputs of the neurons in V_t that are connected to $v_{t+1,\,j}$, where weighting is according to w, and the output of $v_{t+1,\,j}$ is simply the application of the activation function σ on its input.

Layers $V_1, \ldots, V_{T-1}$ are often called *hidden layers*. The top layer V_T is called the output layer. In simple prediction problems, the output layer contains a single neuron whose output is the network's output. Usually, in classification tasks, the output layer has the same number of neurons as the number of classes of the problem at hand.

We refer to T as the number of layers in the network (excluding the input layer V_0) or the *depth* of the network. The *size* of the network is $|V|$. The *width* of the network is instead $\max_t |V_t|$.



In a more compact vector notation, we can define the neuron vector that contains all neurons of layer t as $\mathbf{v}^{(t)} = (1, v_1^{(t)}, \ldots, v_n^{(t)})$. Note that the neuron at index $i = 0$ is the constant neuron, i.e., the bias. Hence, the wight arc from neuron r of layer t to neuron j of layer $t+1$ is $w_{r,\,j}^{(t+1)} = w((v_r^{(t)},\, v_j^{(t+1)}))$. We can now define the vector of all weight arcs in input to neuron j at layer t as $\mathbf{w}_j^{(t)} = (w_{0,\,j}^{(t)}, \ldots\ w_{n^{t-1},\,j}^{(t)})$. Finally, we get the matrix of all weight arcs incoming to layer t:

$$\hat{w}^{(t)} = \begin{bmatrix} w_{0,\,1}^{(t)} & w_{0,\,2}^{(t)} & \cdots & w_{0,\,n^t}^{(t)} \\ w_{1,\,1}^{(t)} & w_{1,\,2}^{(t)} & & w_{1,\,n^t}^{(t)} \\ \vdots & \vdots & \ddots & \vdots \\ w_{n^{t-1},\,1}^{(t)} & w_{n^{t-1},\,2}^{(t)} & \cdots & w_{n^{t-1},\,n^t}^{(t)} \end{bmatrix}$$

Thus, the input to neuron $v_{t+1,\,j}$ when the network is fed with the input vector $\mathbf{x}$ becomes

$$\begin{aligned} a_{t+1,\,j}(\mathbf{x}) &= \sum_{r\,:\,(v_{t,\,r},\,v_{t+1,\,j})\in E} w((v_{t,\,r},\,v_{t+1,\,j}))\,o_{t,\,r}(\mathbf{x}) \\ &= \langle \mathbf{w}_j^{(t+1)},\,\mathbf{v}^{(t)} \rangle \end{aligned}$$

and the output is simply the application of the activation function σ to the input $a_{t+1,\,j}(\mathbf{x})$.

13.2 Activation functions

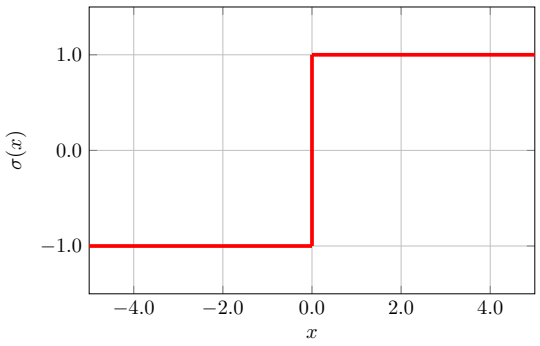
There are plenty of possible activation functions that are employed when dealing with neural networks. We will analyze only five of them, these being

1. Sign function
2. Threshold function
3. Sigmoid function
4. Hyperbolic tangent
5. Rectified linear unit

Note that they all share a non-linear nature: an essential property of activation functions is that they are not linear, as if they were, then the network would turn into a linear classifier. Instead, the expressive power of neural networks is due to these non-linear functions that map the linear input into a non-linear output.

13.2.1 Sign and threshold functions

The sign function outputs the sign of the input, namely $\sigma : \mathbb{R}^n \to \{-1,\,+1\}$.

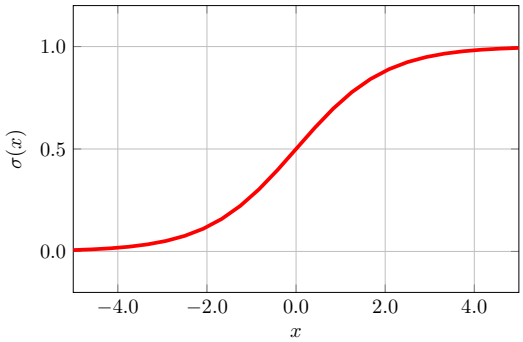


It is simple to implement and leads to high-speed computations. On top of that, it has a clear interpretation as the *firing rate* of a neuron. However, the output is neither smooth nor continuous: its step-like shape kills gradients, and thus the neural network will barely learn when optimized with gradient descent techniques. Of course, the threshold function behaves similarly, and we will not discuss further details.

13.2.2 Sigmoid function

The sigmoid function takes a real-valued number and squeezes it into the $[0,\,1]$ range: $\sigma : \mathbb{R}^n \to [0,\,1]$.

$$\sigma(a) = \frac{1}{1 + e^{-a}}$$

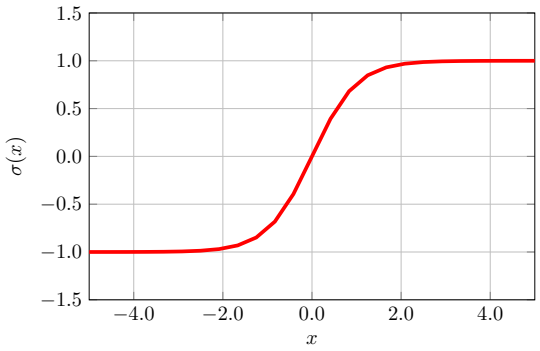


It is still quite an easy function and keeps the firing rate interpretation, where $\sigma(a) = 0$ means not firing at all and $\sigma(a) = 1$ means firing at its full. On top of that, the output is smooth. However, sigmoid neurons still saturate and kill gradients. Thus the neural network will have issues in learning through gradient descent techniques. Namely, when the activation function is near 0 or 1, the gradient is almost zero, and no signal will flow to its weights, leading to little to no information being transmitted to the following layers. Moreover, if the initial weights, chosen randomly, are too large, then most neurons would saturate immediately, killing the neural network's expressive power.

13.2.3 Hyperbolic tangent

The hyperbolic tangent takes a real-valued number and squashes it into the $[-1, +1]$ range: $\sigma : \mathbb{R}^n \to [-1, +1]$.

$$\sigma(a) = \tanh(a) = \frac{e^a - e^{-a}}{e^a + e^{-a}} = \frac{e^{2a} - 1}{e^{2a} + 1}$$



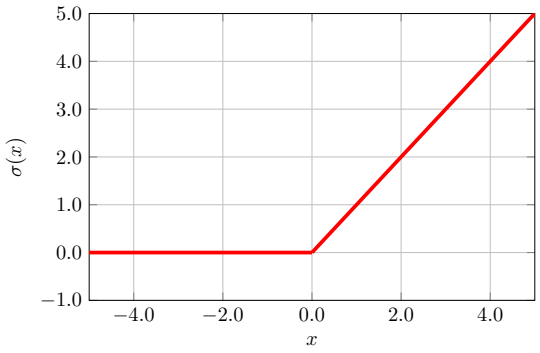
The hyperbolic tangent is a scaled and shifted sigmoid function: $\tanh(x) = 2\,\text{sigmoid}(2x) - 1$. Hence, most of the sigmoid properties hold for the hyperbolic tangent too. The critical difference is that the hyperbolic tangent output is zero-centered, making the optimization more stable.

13.2.4 Rectified linear unit

The ReLU function takes a real-valued number and thresholds it at zero: $\sigma : \mathbb{R}^n \to \mathbb{R}^n_+$.

$$\sigma(a) = \max(0,\, a) = \begin{cases} a & \text{if } \; a > 0 \\ 0 & \text{if } \; a \leq 0 \end{cases}$$

Most deep networks employ the ReLU activation function nowadays: the training is much faster as the function is even simpler than the previous ones. Moreover, it accelerates the convergence of SGD and has a linear, non-saturating shape. On top of that, it leads to less expensive operations when compared to the sigmoid or hyperbolic tangent functions that involve exponentials. Furthermore, the rectified linear unit is more expressive and prevents the gradient vanishing issue we have introduced.



13.3 Forward propagation

The so-called *forward propagation* is the neural network output computation process. Namely, we take an input sample $\mathbf{x}$ and compute the network's output using the rules introduced in Section 13.1. We start from the input layer V_0 and propagate the input sample into layer V_1. Here we compute its output and propagate the information into layer V_2. We iterate this process layer by layer until we get to the output layer V_T, whose output is the neural network's output.

Algorithm: Forward propagation

input: input sample $\mathbf{x} = (x_1, \ldots, x_d)$, neural network with a single output node

$V_0 = (1, x_1, \ldots, x_d)$

for $t = 1, \ldots, T$ **do**

$\quad a^{(t)} = \left(\hat{w}^{(t)}\right)^T o^{(t-1)}$

$\quad o^{(t)} = \left(1, \sigma(a^{(t)})\right)$

end

$y = v^{(T)}$

output: neural network prediction y

13.4 Learning neural networks

Once we have specified a neural network by (V, E, σ, w) we obtain a function

$$h_{V,E,\sigma,w} : \mathbb{R}^{|V_0|-1} \to \mathbb{R}^{|V_T|}$$

Any set of such functions can serve as a hypothesis class for learning. Usually, we define a hypothesis class of neural networks predictors by fixing the graph (V, E) as well as the activation function σ and letting the hypothesis class be all functions of the form $h_{V,E,\sigma,w}$ for some $w : E \to \mathbb{R}$. The triplet (V, E, σ) is often called the *architecture* of the network. We denote the hypothesis class by

$$\boldsymbol{\mathcal{H}}_{V,E,\sigma} = \{h_{V,E,\sigma,w} : w : E \to \mathbb{R}\} \tag{13.1}$$

The parameters specifying a hypothesis in the hypothesis class are the weights over the network's edges.

13.4.1 The expressive power of neural networks

In this section, we study the expressive power of neural networks, namely, what functions can be implemented using a neural network. More concretely, we will fix some architecture (V, E, σ) and will study what function hypotheses in $\boldsymbol{\mathcal{H}}_{V,E,\sigma}$ can implement, as a function of the size of V.

We start the discussion by studying which type of boolean functions (i.e., function from $\{\pm 1\}^n$ to $\{\pm 1\}$) can be implemented by $\boldsymbol{\mathcal{H}}_{V,E,\text{sign}}$. We begin with a simple claim, showing that without

restricting the size of the network, every boolean function can be implemented using a neural network of depth 2.

Proposition 13.4.1. *For every n, there exists a graph (V, E) of depth 2 such that $\boldsymbol{\mathcal{H}}_{V,E,sign}$ contains all functions from $\{\pm 1\}^n$ to $\{\pm 1\}$.*

The preceding claim shows that neural networks can implement any boolean function. However, this is a very weak property, as the size of the resulting network might be exponentially large.

Theorem 13.4.1. *For every n, let $s(n)$ be the minimal integer such that there exists a graph (V, E) with $|V| = s(n)$ such that the hypothesis class $\boldsymbol{\mathcal{H}}_{V,E,sign}$ contains all the functions from $\{\pm 1\}^n$ to $\{\pm 1\}$. Then, $s(n)$ is an exponential function of n.*

It is possible to derive a similar theorem for $\boldsymbol{\mathcal{H}}_{V,E,\sigma}$ for any σ as long as we restrict the weights so that it is possible to express every weight using a number of bits which is bound by a universal constant. We can even consider hypothesis classes where different neurons can employ different activation functions as long as the number of allowed activation functions is also finite.

So far we have discussed boolean functions. However, it is possible to show that neural networks are universal approximators. That is, for every fixed precision parameter $\epsilon > 0$ and every Lipschitz function $f : [-1, +1]^n \to [-1, +1]$, it is possible to construct a network such that for every input $\mathbf{x} \in [-1, +1]^n$ the network outputs a number between $f(\mathbf{x}) - \epsilon$ and $f(\mathbf{x}) + \epsilon$. However, as in the case of boolean functions, the network size cannot be polynomial in n.

13.4.2 Conjunction and disjunction

A boolean circuit is a type of network in which the individual neurons implement conjunction, disjunction and negation of their input. To relate boolean circuits to networks with the sign activation function, we need to show that we can implement the operations of conjunction, disjunction and negation using the sign activation function. Clearly, we can implement the negation operator using the sign activation function. The following lemma shows that the sign activation function can also implement conjunctions and disjunctions of its inputs.

Lemma 13.4.2. *Suppose that a neuron v, that implements the sign activation function has k incoming edges, connecting it to neurons whose outputs are in $\{\pm 1\}$. Then, by adding one more edge linking a constant neuron to v and adjusting the weights on the edges to v, the output of v can implement the conjunction or disjunction of its inputs.*

Proof. Simply observe that if $f : \{\pm 1\}^k \to \{\pm 1\}$ then the conjunction function $f(\mathbf{x}) = \wedge_i x_i$ can be rewritten as

$$f(\mathbf{x}) = \text{sign}\left(1 - k + \sum_{i=1}^{k} x_i\right)$$

Similarly the disjunction function $f(\mathbf{x}) = \vee_i x_i$ can be rewritten as

$$f(\mathbf{x}) = \text{sign}\left(k - 1 + \sum_{i=1}^{k} x_i\right)$$

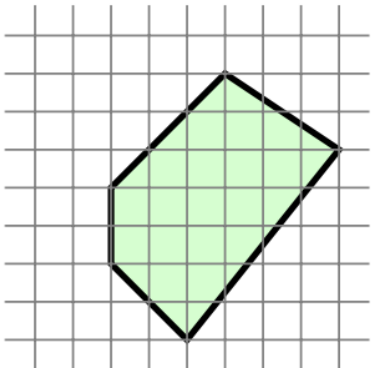
□

13.4.3 Geometric intuition

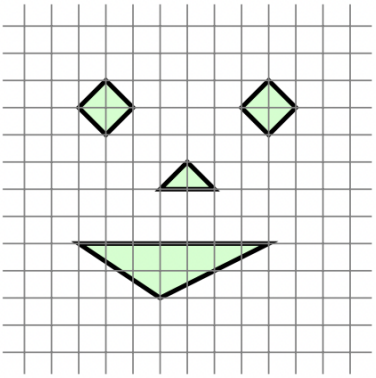
We next provide several geometric illustrations of functions $f : \mathbb{R}^2 \to \{\pm 1\}$ and show how to express them using a neural network with the sign activation function.

Let us start with a depth two network, namely a network with a single hidden layer. First, each neuron in the hidden layer implements a halfspace predictor. Then, the single neuron at the output layer applies a halfspace on top of the binary outputs of the neurons in the hidden layer. As we have shown before, a halfspace can implement the conjunction function. Therefore, such networks contain

all hypotheses, which are the intersection of $k-1$ halfspaces, where k is the number of neurons in the hidden layer. In other words, they can express all convex polytopes with $k-1$ faces.



We have shown that a neuron in layer V_2 can implement a function that indicates whether $\mathbf{x}$ is in some convex polytope. By adding one more layer and letting the neuron in the output layer implement the disjunction of its inputs, we get a network that computes the union of polytopes.



13.5 Sample complexity of neural networks

Next, we discuss the sample complexity of learning the class $\boldsymbol{\mathcal{H}}_{V,E,\sigma}$. Recall that the fundamental theorem of learning tells us that the sample complexity of learning a hypothesis class of binary classifiers depends on its VC dimension. Therefore, we focus on computing the VC dimension of hypothesis classes of the form $\boldsymbol{\mathcal{H}}_{V,E,\sigma}$ where the output layer of the graph contains a single neuron. We start with the sign activation function, namely $\boldsymbol{\mathcal{H}}_{V,E,\text{sign}}$. What is the VC dimension of this class? Intuitively, since we learn $|E|$ parameters, the VC dimension should be of order $|E|$. This is indeed the case, as formalized by the following theorem.

Theorem 13.5.1. *The VC dimension of $\boldsymbol{\mathcal{H}}_{V,E,sign}$ is $O(|E|\log(|E|))$.*

Next, we consider $\boldsymbol{\mathcal{H}}_{V,E,\sigma}$ where σ is the sigmoid function. Surprisingly, the VC dimension of $\boldsymbol{\mathcal{H}}_{V,E,\sigma}$ is the number of tunable parameters squared. It is also possible to upper bound the VC dimension as follows.

Theorem 13.5.2. *The VC dimension of $\boldsymbol{\mathcal{H}}_{V,E,\sigma}$, where σ is the sigmoid function, is $O(|V|^2\,|E|^2)$.*

Remember that the number of edges $|E|$ is proportional to the square of the number of neurons $|V|$, namely $|E| \propto |V|^2$. Clearly, to properly learn the hypothesis class, we need a massive amount of data.

13.6 Runtime of learning neural networks

In the previous sections, we have shown that the class of neural networks with an underlying graph of polynomial size can express all functions that can be implemented efficiently and that the sample

complexity has a favorable dependence on the network size. In this section, we turn to the analysis of the time complexity of training neural networks.

We first show that it is NP-hard to implement the ERM rule with respect to $\boldsymbol{\mathcal{H}}_{V,E,\text{sign}}$ even for networks with a single hidden layer that contains just four neurons in the hidden layer.

Theorem 13.6.1. *Let $k \geq 3$. For every n, let (V, E) be a layered graph with n input nodes and $k+1$ nodes at the (single) hidden layer, where one is the constant neuron and a single output node. Then, it is NP-hard to implement the ERM rule with respect to $\boldsymbol{\mathcal{H}}_{V,E,sign}$.*

One way around the preceding hardness result could be that for the purpose of learning, it may suffice to find a predictor $h \in \boldsymbol{\mathcal{H}}$ with low empirical error, not necessarily an exact ERM. However, it turns out that even the task of finding weights that result in close-to-minimal empirical error is computationally infeasible.

A widely used heuristic for training neural networks relies on the SGD framework. We have shown that SGD is a successful learner if the loss function is convex. Unfortunately, in neural networks, the loss function is highly non-convex. Nevertheless, we can still implement the SGD algorithm and hope it will find a reasonable solution to the problem (as happens to be the case in several practical tasks).

13.7 SGD and backpropagation

Finding a hypothesis in $\boldsymbol{\mathcal{H}}_{V,E,\sigma}$ with low-risk amounts to the problem of tuning the weights over the edges. This section shows how to apply a heuristic search for good weights using the SGD algorithm. Throughout this section, we assume that σ is the sigmoid function

$$\sigma(a) = \frac{1}{1+e^{-a}}$$

but the derivation holds for any differentiable scalar function.

Since E is a finite set, we can think of the weight function as a vector $\mathbf{w} \in \mathbb{R}^{|E|}$. Suppose the network has n input neurons and k output neurons, and denote by $h_{\mathbf{w}} : \mathbb{R}^n \to \mathbb{R}^k$ the function computed by the network if the weight function is defined by $\mathbf{w}$. Let us denote by $\Delta(h_{\mathbf{w}}(\mathbf{x}), y)$ the loss of predicting $h_{\mathbf{w}}(\mathbf{x})$ when the target is $y \in \boldsymbol{\mathcal{Y}}$. For concreteness, we will take Δ to be the squared loss

$$\Delta(h_{\mathbf{w}}(\mathbf{x}), y) = \frac{1}{2}||h_{\mathbf{w}}(\mathbf{x}) - y||^2$$

Note that a similar derivation can be obtained for every differentiable function. Finally, given a distribution $\boldsymbol{\mathcal{D}}$ over the examples domain $\mathbb{R}^n \times \mathbb{R}^k$, let $L_{\boldsymbol{\mathcal{D}}}(\mathbf{w})$ be the risk of the network, namely

$$L_{\boldsymbol{\mathcal{D}}}(\mathbf{w}) = \mathbb{E}_{(\mathbf{x},y)\sim\boldsymbol{\mathcal{D}}}[\Delta(h_{\mathbf{w}}(\mathbf{x}), y)]$$

Recall the SGD algorithm for minimizing the risk function $L_{\boldsymbol{\mathcal{D}}}(\mathbf{w})$. We repeat the pseudocode in Section 11.3 with a few modifications, which are relevant to the neural network application because of the non-convexity of the objective function. First, while in Section 11.3 we initialized $\mathbf{w}$ to be the zero vector, here we initialize $\mathbf{w}$ to be a randomly chosen vector with values close to zero. This is because an initialization with the zero vector will lead all hidden neurons to have the same weights. In addition, the hope is that if we repeat the SGD procedure several times, where each time we initialize the process with a new random vector, one of the runs will lead to a good local minimum. Second, while a fixed step size η is guaranteed to be good enough for convex problems, here we utilize a variable step size η_t. Because of the non-convexity of the loss function, the choice of the sequence η_t is more significant, and it is tuned in practice in a trial-and-error manner. Third, we output the best-performing vector on a validation set. In addition, it is sometimes helpful to add regularization on the weights with parameter λ. That is, we try to minimize $L_{\boldsymbol{\mathcal{D}}}(\mathbf{w}) + \frac{\lambda}{2}||\mathbf{w}||^2$. Finally, the gradient does not have a closed-form solution. Instead, it is implemented using the backpropagation algorithm. The idea is thus to propagate forward the training data and compute the loss function. Then we compute the loss gradient with respect to the weights and update the weights with SGD. The way we compute the gradient is through the backpropagation algorithm.

Algorithm: Stochastic gradient descent for neural networks

parameters: number of iterations τ
step size sequence $\eta_1, \eta_2, \ldots, \eta_\tau$
regularization parameter $\lambda > 0$
input: layered graph (V, E)
differentiable activation function $\sigma : \mathbb{R} \to \mathbb{R}$
initialize: choose $\mathbf{w}^{(1)} \in \mathbb{R}^{|E|}$ at random from a distribution such that $\mathbf{w}^{(1)}$ is close to $\vec{0}$
for $i = 1, \ldots, \tau$ **do**
  sample $(\mathbf{x}, y) \sim \boldsymbol{\mathcal{D}}$
  compute gradient $\mathbf{v}_i = \texttt{backpropagation}(\mathbf{x}, y, \mathbf{w}, (V, E), \sigma)$
  update $\mathbf{w}^{(i+1)} = \mathbf{w}^{(i)} - \eta_i \left(\mathbf{v}_i + \lambda \mathbf{w}^{(i)}\right)$
end
output: $\bar{\mathbf{w}}$ is the best performing $\mathbf{w}^{(i)}$ on a validation set

13.7.1 Backpropagation

At this point, we need to compute the gradient of the loss function with respect to every single weight in the network. However, we can only compute the loss on the network's outputs, i.e., after the last layer. Backpropagation overcomes this issue as follows.

Recall that each layer has the linear input $\mathbf{a}^{(t)}$, where the elements of the vector are the linear inputs of neurons in layer t, and the non-linear output $\mathbf{o}^{(t)}$ that is simply the application of the activation function to the linear input. Let us denote with

$$\delta^{(t)} = \frac{\partial L}{\partial \mathbf{a}^{(t)}} = \begin{pmatrix} \frac{\partial L}{\partial a_1^{(t)}} \\ \vdots \\ \frac{\partial L}{\partial a_{n_t}^{(t)}} \end{pmatrix}$$

the change in error with respect to the weighted average *before* the non-linear transformation. This notation will be useful as we will decompose the gradient with the chain rule.

The (linear) input to the j-th neuron in the t-th layer is

$$a_j^{(t)} = \sum_{k=0}^{n_{t-1}} w_{k,j}^{(t)} o_k^{(t-1)}$$

thus each weight $w_{i,j}^{(t)}$ impacts only on $a_j^{(t)}$. Hence, we need $\delta^{(t)} = \frac{\partial L}{\partial \mathbf{a}^{(t)}}$ to compute the gradient.

$$\frac{\partial L}{\partial w_{i,j}^{(t)}} = \frac{\partial L}{\partial a_j^{(t)}} \frac{\partial a_j^{(t)}}{\partial w_{i,j}^{(t)}} = \delta^{(t)_j} \frac{\partial}{\partial w_{i,j}^{(t)}} \left(\sum_{k=0}^{n_{t-1}} w_{k,j}^{(t)} o_k^{(t-1)} \right) = \delta_j^{(t)} \, o_i^{(t-1)}$$

We can also further decompose

$$\delta_j^{(t)} = \frac{\partial L}{\partial a_j^{(t)}} = \frac{\partial L}{\partial o_j^{(t)}} \frac{\partial o_j^{(t)}}{\partial a_j^{(t)}} = \frac{\partial L}{\partial o_j^{(t)}} \sigma'(a_j^{(t)})$$

where $\sigma'(a)$ is the derivative of the neuron's activation function. The challenging term to compute is indeed $\partial L / \partial o_j^{(t)}$ as we need to understand first how the loss function changes with respect to the neuron's output. While each weight $w_{i,j}^{(t)}$ impacts only on $a_j^{(t)}$, the output of neurons in layer t affects all neurons in layer $t+1$. The key point is that each neuron affects all neurons in the next layer. Thus, to compute $\partial L / \partial o_j^{(t)}$, we need to sum contributions coming from all neurons in layer $t+1$.

$$\frac{\partial L}{\partial o_j^{(t)}} = \sum_{k=1}^{n_{t+1}} \frac{\partial L}{\partial a_j^{(t+1)}} \frac{\partial a_j^{(t+1)}}{\partial o_j^{(t)}} = \sum_{k=1}^{n_{t+1}} \delta_k^{(t+1)} w_{jk}^{(t+1)}$$

Thus, we have found that

$$\delta_j^{(t)} = \frac{\partial L}{\partial o_j^{(t)}} \frac{\partial o_j^{(t)}}{\partial a_j^{(t)}} = \sum_{k=1}^{n_{t+1}} \delta_k^{(t+1)} w_{jk}^{(t+1)} \sigma'(a_j^{(t)}) \tag{13.2}$$

The critical result is that the term $\delta_j^{(t)}$ for each layer t needs the solution of the following layer $t+1$. Since we can only compute the loss on the network's outputs, i.e., after the last layer, we can start right from the last layer and compute $\delta^{(T)}$ using the output loss. Then we back-propagate the gradients through all the layers from last to first.

Algorithm: Backpropagation

input: a single example $(\mathbf{x}_i, y_i)$

the neural network with weights $w_{ij}^{(t)}$ $(t \in [1, T])$

compute: $\delta^{(T)} = \frac{\partial L}{\partial a^{(T)}}$

for $t = T - 1$ **downto** 1 **do**

  $\delta_j^{(t)} = \sum_{k=1}^{n_{t+1}} \delta_k^{(t+1)} w_{jk}^{(t+1)} \sigma'(a_j^{(t)})$ for all $j = 1, \ldots, n_t$

end

output: $\delta^{(1)}, \ldots, \delta^{(T)}$

We now have all the tools to write the complete neural network training algorithm

---

Algorithm: Neural network training algorithm

input: training sample $(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_m, y_m)$

initialize: choose $w_{ij}^{(t)}$ at random from a distribution such that $\mathbf{w}^{(t)}$ is close to $\vec{0}$ for all i, j, t

for $s = 0, 1, 2, \ldots$ **do**

  sample $(\mathbf{x}_k, y_k)$ at random from the training set

  compute the output of neurons $o_j^{(t)}$ for all j, t ; `/* forward propagation */`

  compute $\delta_j^{(t)}$ for all j, t ; `/* backward propagation */`

  update $w_{ij}^{(t)[s+1]} = w_{ij}^{(t)[s]} - \eta o_i^{(t-1)} \delta_j^{(t)}$ for all i, j, t

end

if *converged* **then**

  output: return $w_{ij}^{(t)[s]}$ for all i, j, t

end

13.8 Training details

13.8.1 Pre-processing and weights initialization

Typically, input datasets are normalized and centered around zero to speed up the training process. Note that both local and global normalization strategies can be employed.

On top of that, we have already discussed that initializing the weights to zero does not work. Thus, the simplest overcoming of this constraint is to pick random values around zero: the model is roughly linear in this regime. Usually, uniform or gaussian distributions are used to random sample the initial weights. It is common practice to test multiple weights initialization and training to choose the best result afterward. This is because there is always the possibility of falling into a local minimum of the loss function (with highly dimensional spaces, the probability is smaller, but there is no guarantee).

13.8.2 Stopping the training process

Usually, the perfect solution is never found. Thus we need to define a rule such that the algorithm stops at some point. The first possible solution is to define a training error threshold and stop the training process when the loss falls below the predefined value. Another possibility is to set an upper bound on the number of iterations or to stop when we have small marginal improvements in training error at each step. Note that all these three strategies can be implemented together to avoid wasting time training the network unnecessarily.

13.8.3 Regularization

Due to the incredibly large number of parameters, the risk of overfitting is clearly very high. As a matter of fact, overfitting is even more critical when dealing with neural networks than any other learning model we have introduced so far. Thus, regularization is necessary. Instead of minimizing just the loss function, we jointly minimize

$$L_{\boldsymbol{\mathcal{S}}}(h) + \frac{\lambda}{m} \sum_{i,\,j,\,t} |w_{ij}^{(t)}| \tag{13.3}$$

$$L_{\boldsymbol{\mathcal{S}}}(h) + \frac{\lambda}{m} \sum_{i,\,j,\,t} \left(w_{ij}^{(t)}\right)^2 \tag{13.4}$$

where we respectively used the $L1$ and $L2$ regularizations to avoid large weights and make the optimization more stable.

Chapter 14

Convolutional Neural Networks

In the last chapter, we have widely discussed fully connected, feed-forward neural networks. We highlight here two main issues of the neural network model we have seen:

1. Each neuron in layer $t-1$ is connected to all neurons in layer t
2. The domain structure is not taken into account

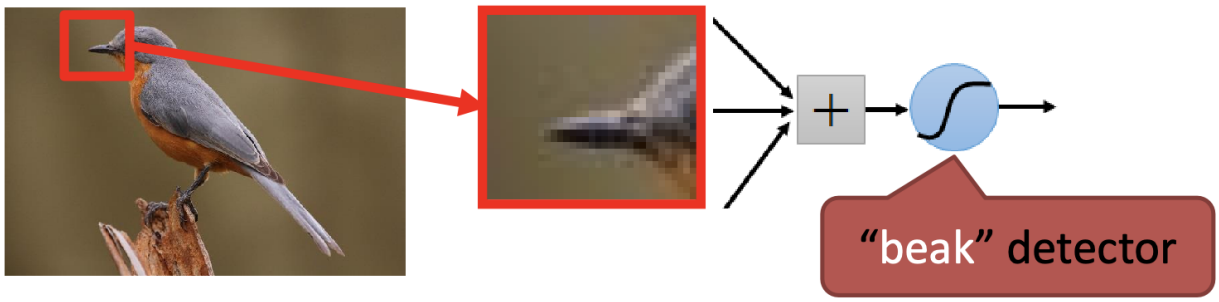
The first point leads to a massive number of weights. Recall that it is quadratic to the number of neurons. The second point shows that the model does not consider the possibility that a neuron can be "closer" (i.e., more related) to some neurons and less to others. On top of that, some domains have a relevant structure. Examples are the grid of pixels in an image, the sequence of samples in an audio signal, and the letters of a word in a text. To strengthen the model, we need to capture that a pixel in an image is more related to the close pixels than to the far apart ones or that a letter in a text is more related to letters of the same word than to the ones ten pages ahead. Instead, if we place data in a vector and feed it to the neural network, we lose the spatial structure of data.

In this chapter, then, we will discuss the most common upgraded neural network model, the convolutional one, and show how it overcomes the two main issues of fully connected feed-forward neural networks. In order to make the explanation clear and less theoretical, we will exploit a relevant example, namely, an image learning task.

14.1 Learning an image

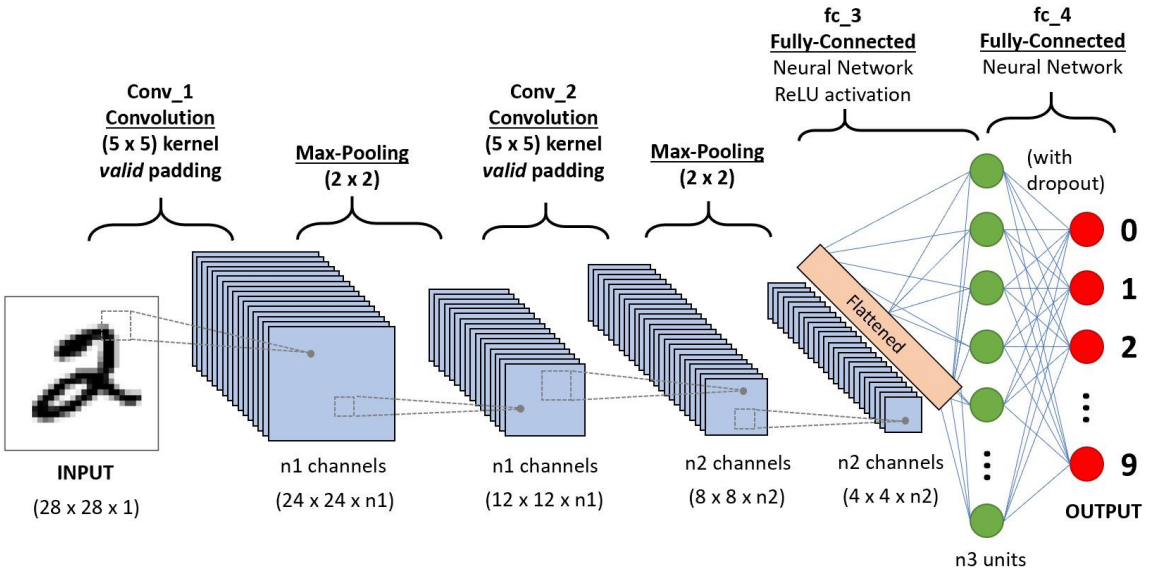
In many image recognition tasks, we usually focus on learning key features and details of the subject in the picture. For example, if we want to recognize birds, we will focus on finding a beak, wings, a tail and two legs. These details are far smaller than the whole image; thus, we can represent these smaller regions with fewer parameters.

The idea is to build a detector for each detail we want to recognize, namely a beak detector, a wing detector and so on. These features share two main characteristics: they are all *local* and *shift invariant*. In other words, they are all localized in a small portion of the image and can be found in different parts of the image.



To detect the birds' beaks in all possible locations in the image, we need to build similar detectors that work in different regions. However, the fact that these features, such as the beak, are shift-invariant means that similar detectors also share similar parameters. The idea is thus to have some small detectors and let them move around the image.

14.2 From neural networks to convolutional neural networks

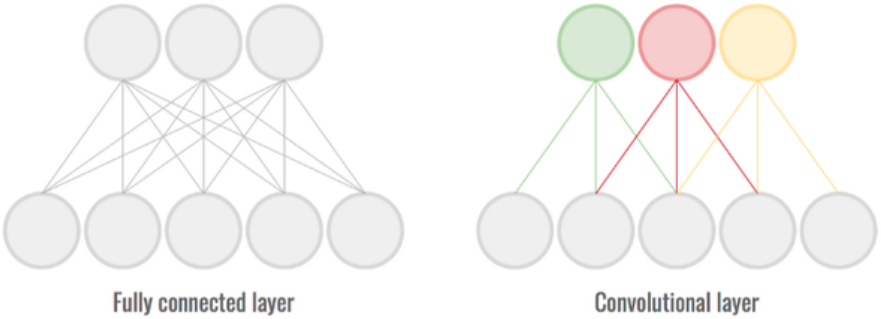


The convolutional neural network one is the learning model that allows us to implement such an idea and combines all detail detectors in just one algorithm. It has four main improvements:

1. **Local connectivity:** receptive field for each neuron
2. **Shared weights:** spatially invariant response
3. **Multiple feature maps**
4. **Pooling**

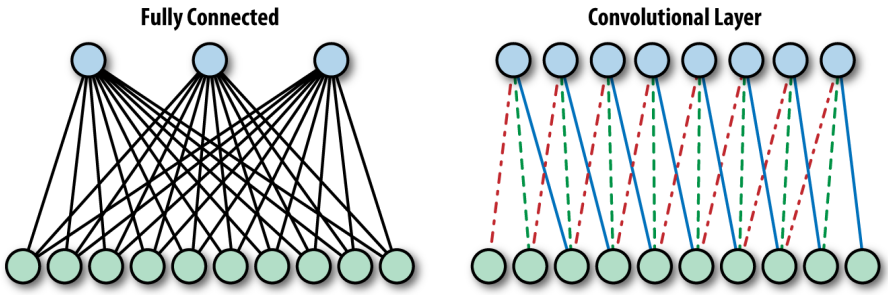
14.2.1 Local connectivity

Local connectivity means that each neuron in layer $t+1$ is connected to a restricted subset of neurons in layer t. The subset of neurons in layer t is called the *receptive field* of the connected neuron in layer $t+1$.



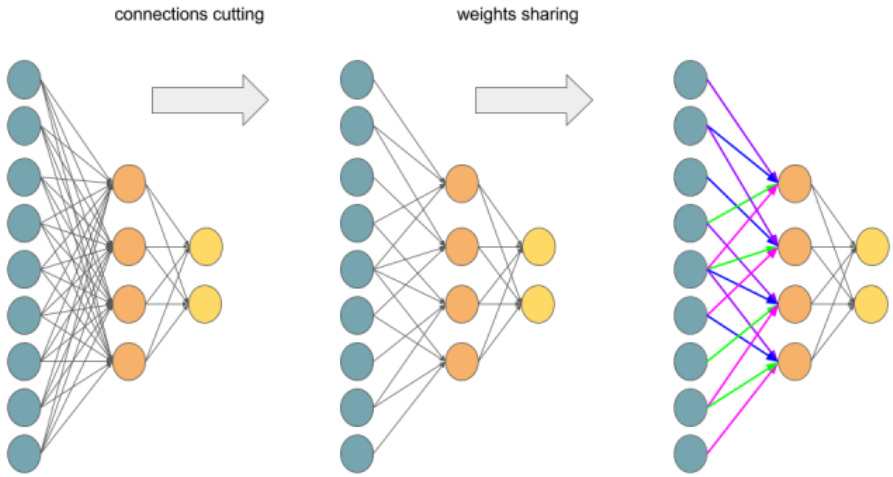
Consider, for example, having an image as input to the neural network. Then, local connectivity leads to connecting the neuron that represents a certain image pixel to only neighbor pixels instead of

connecting it to the whole image. This way, we consider that image pixels are more correlated with neighbors than those far from them. Moreover, we not only consider the spatial structure of data, but we drastically reduce the number of parameters of the model, thus making it easier to learn.



14.2.2 Shared weights

Shared, or tied, weights (*parameter sharing*) means that each neuron in layer $t+1$ has the same weights as the other in their corresponding receptive field. In other words, each neuron in layer $t+1$ computes the same function but with a different input window.



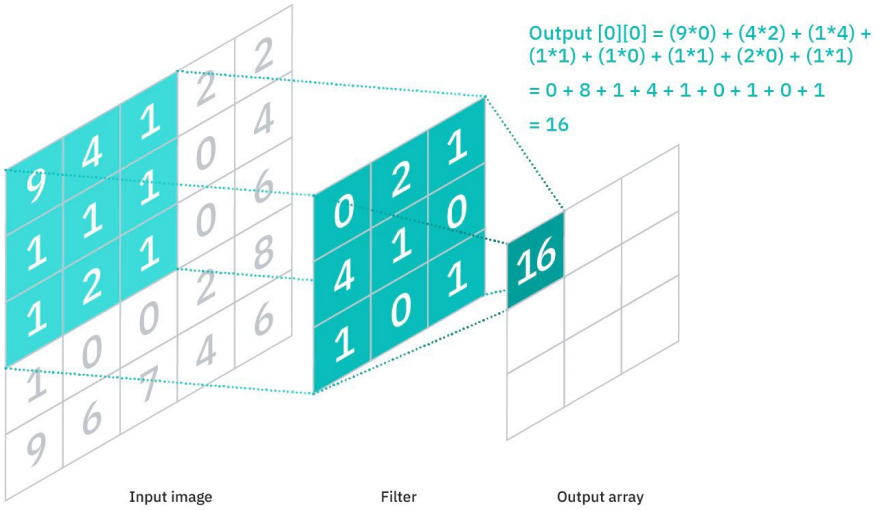
Parameter sharing reduces even more the overall number of parameters and acts as a one-dimensional filter. In this case, however, the filter parameters are learnable.

14.2.3 Multiple feature maps

For each layer, there exist multiple subsets of neurons to be tested. That is, there are multiple subsets of neurons (*maps*) in layer $t+1$ with the same receptive field but with different weights between maps. The idea is that each feature map tries to detect different features. For example, consider the bird image and the task of telling whether an input figure contains a bird or not. The first feature map could try to identify the peak, and the second could focus on the wings. As we said, this is possible by using different weights for each feature map.

14.3 The convolution operation

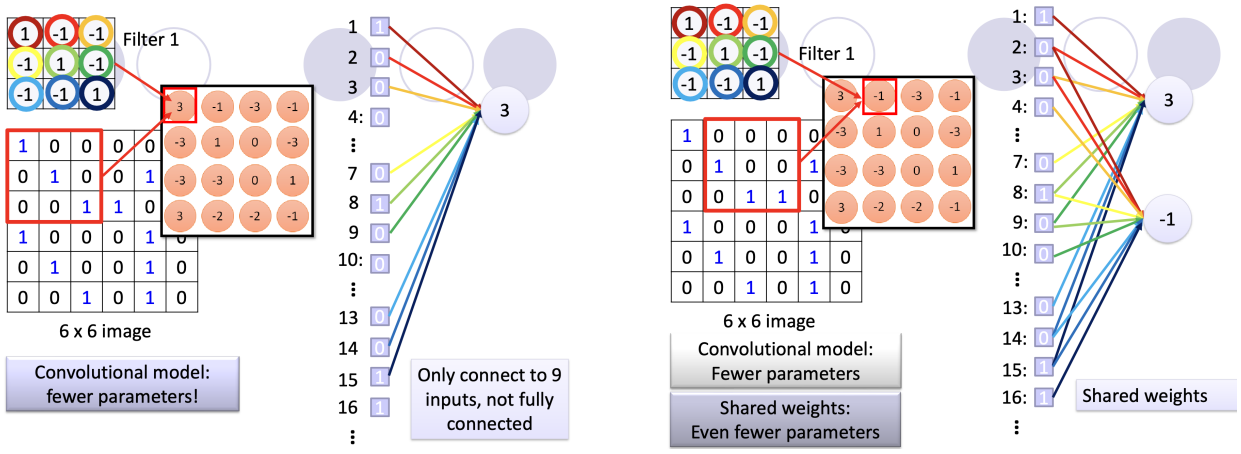
In this section, we dig deeper into the convolution operation, which is the main feature of convolutional neural networks. Let us assume we are trying to learn an image, namely a $n \times n$ matrix whose entries are related to the image pixels. We take as an example a 5×5 matrix. We then define filters (each filter leads to a different feature map) to be 3×3 matrices whose entries are the learnable parameters of the model.



The convolution operation is simply a matrix by the matrix dot product. Local connectivity and shared weights translate into the product between a 3×3 sub-matrix of the input image and the learnable filter. Usually, these filters are chosen to be squares of odd side length: this way, the convolution operation is *centered* in a specific input neuron. The filter is then applied to all possible input sub-matrices to cover the whole input image. Whenever the convolution is centered in a neuron located on the edge of the image, we can use padding to extend the input size or stop the convolution before reaching the edge to reduce the output size. Finally, we apply the same procedure using different filters to obtain k feature maps, an output matrix of shape $k \times n - 2 \times n - 2$.

To reduce the output image resolution, we can exploit *stride*. Stride allows the filter to move in steps of multiple samples. Namely, if the stride is set equals to 2, then the convolution operation is centered two pixels after the last convolution center instead of sliding to its immediate neighbor.

The following pictures show how the convolutional graphic representation relates to the usual neural network representation. Neurons are labeled from 1 to 36 following rows first. The 3×3 filter translates into a neuron's receptive field of 9 neurons in the previous layer. Sliding the filter matrix throughout the whole image translates directly into the shared weights feature of convolutional neural networks.



14.4 The pooling operation

A problem with the output feature maps is that they are sensitive to the location of features in the input. One approach to address this sensitivity is to down-sample the feature maps. This makes the resulting down-sampled feature maps more robust to changes in the position of the feature in the image, referred to by "local translation invariance."

Pooling layers provide an approach to down-sampling feature maps by summarizing the presence of features in patches of the feature map. Two common pooling methods are *average pooling* and

max pooling, which summarize the average presence of a feature and the most activated presence of a feature, respectively.

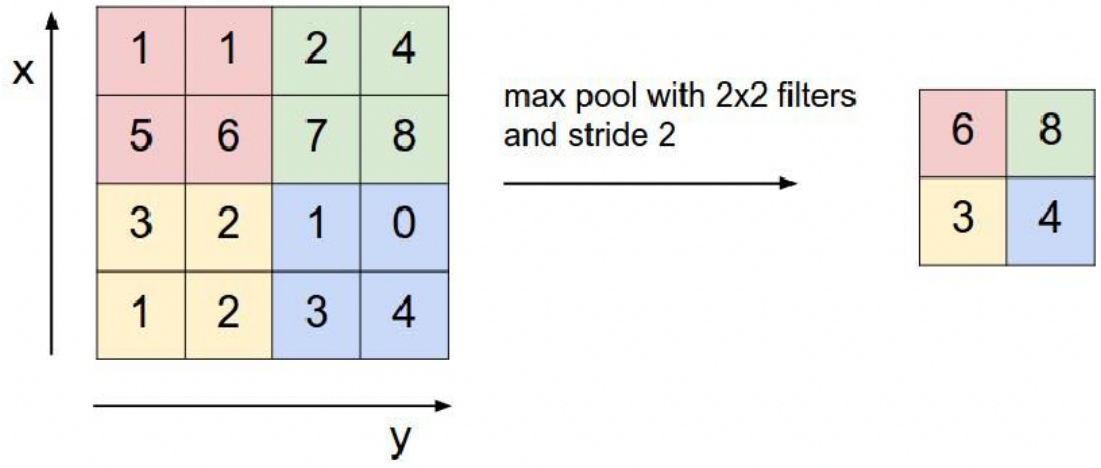
As we said earlier, a limitation of the feature map output of convolutional layers is that they record the precise position of features in the input. This means that small movements in the position of the feature in the input image will result in a different feature map. For example, this can happen with re-cropping, rotation, shifting, and other minor changes to the input image. Down sampling is a common approach to addressing this problem from signal processing. This is where a lower resolution version of an input signal is created that still contains the large or important structural elements without the fine detail that may not be as useful to the task.

A *pooling layer* is a new layer added after the convolutional layer. Specifically after a nonlinearity has been applied to the feature maps output by a convolutional layer. Adding a pooling layer after the convolutional layer is a common pattern used for ordering layers within a convolutional neural network that may be repeated one or more times in a given model. The pooling layer operates upon each feature map separately to create a new set of the same number of pooled feature maps.

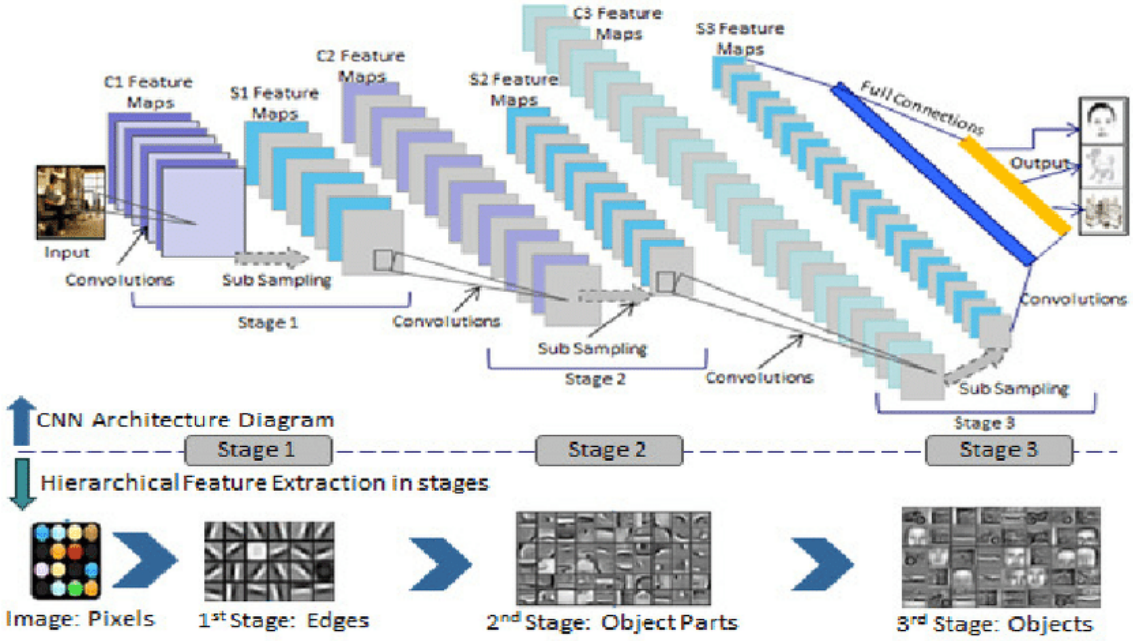
Pooling involves selecting a pooling operation to be applied to feature maps, much like a filter. The pooling operation or filter size is smaller than the feature map's size; specifically, it is almost always 2×2 pixels applied with a stride of 2 pixels. This means that the pooling layer will always reduce the size of each feature map by a factor of 2, e.g., each dimension is halved, reducing the number of pixels or values in each feature map to one-quarter the size. For example, a pooling layer applied to a feature map of 6×6 (36 pixels) will result in an output pooled feature map of 3×3 (9 pixels).

The pooling operation is specified rather than learned. Two common functions used in the pooling operation are:

- **Max pooling:** Calculate the maximum value for each patch of the feature map.
- **Average pooling:** Calculate the average value for each patch on the feature map.



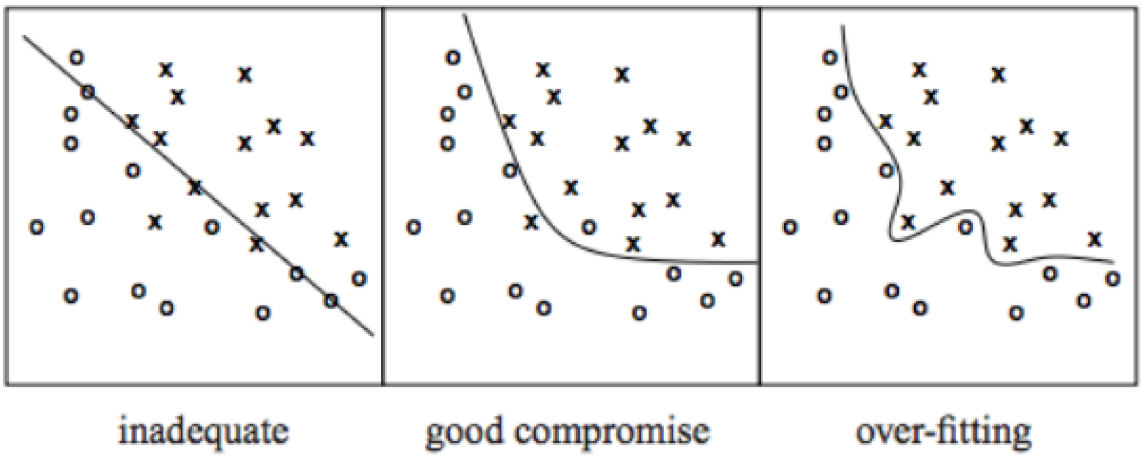
The result of using a pooling layer and creating down-sampled or pooled feature maps is a summarized version of the features detected in the input. They are useful as small changes in the feature's location in the input detected by the convolutional layer will result in a pooled feature map with the feature in the same location. This capability added by pooling is called the model's invariance to local translation.



14.5 Overfitting

Large neural networks trained on relatively small datasets can overfit the training data. This has the effect of the model learning the statistical noise in the training data, which results in poor performance when the model is evaluated on new data, e.g., a test dataset. In addition, generalization error increases due to overfitting.

The first and most efficient approach to deal with overfitting is to avoid using a complex model when the training dataset is limited in size.

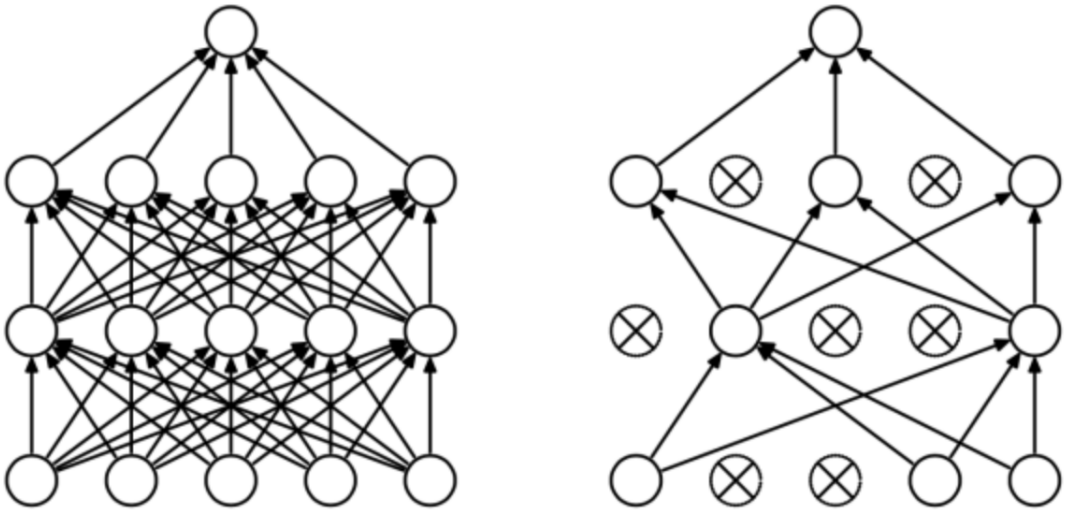


Though, there are a couple of tricks we can exploit to mitigate overfitting even when using a complex model.

14.5.1 Drop-out

Dropout is a regularization method that approximates training a large number of neural networks with different architectures in parallel. Some layer outputs are randomly ignored during training or "dropped out." This makes the layer look like and is treated like a layer with a different number of nodes and connectivity to the prior layer. Each update to a layer during training is performed with a different "view" of the configured layer. Dropout makes the training process noisy, forcing nodes within a layer to probabilistically take on more or less responsible for the inputs. This conceptualization suggests

that perhaps dropout breaks up situations where network layers co-adapt to correct mistakes from prior layers, making the model more robust.



(a) Standard Neural Net (b) After applying dropout.

14.5.2 Regularization

Another way to prevent overfitting is to use regularization. Regularization is a method that controls the model complexity. Each feature is assigned a certain weight. If there are many features, then there will be a large number of weights, which will make the model prone to overfitting. So regularization reduces the burden on these weights. The weights then end up less impacting the loss function, determining the error between the actual and predicted labels.

L1 regularization

For $L1$ regularization, we need to add the following penalty term to the loss function

$$L_{\text{total}} = L_{\mathbf{w}} + \lambda \sum_{i,j} |w_{i,j}| \tag{14.1}$$

However, one downside of L1 regularization is that you will end up with parameters with sparse values. This is because the values of the parameters will be close to zeros.

L2 regularization

For $L2$ regularization, instead, we take the square of the parameters

$$L_{\text{total}} = L_{\mathbf{w}} + \lambda \sum_{i,j} (w_{i,j})^2 \tag{14.2}$$

With $L2$ regularization, the weights will not be sparse, and we will get much better accuracy than $L1$.

14.5.3 Early stopping

A very intuitive and straightforward way to mitigate overfitting is to use the validation error to decide when to stop the training. For example, we can stop the training procedure when the monitored validation loss has not improved in the last n subsequent epochs. The parameter n is called *patience*.

14.5.4 Data augmentation

The strategy behind data augmentation is to virtually increase the number of training samples by adding a little variance to the data.

The artificial samples could be correlated to the real samples but could also be uncorrelated noise.

14.6 Optimization

A variety of optimization algorithms have already been proposed during the course in Chapter 11.

14.7 Loss function

Usually, when working with Deep Neural Networks, the *cross-entropy* loss function is preferred over the standard $0-1$ loss. For binary classification, the cross-entropy loss is

$$L(f(\mathbf{x}),\, y) = -y \log(f(\mathbf{x})) - (1-y)\log(1-f(\mathbf{x})) \tag{14.3}$$

Therefore, the optimal $f(\mathbf{x})$ minimizing the loss function is

$$f(\mathbf{x}) = P(y = 1 \,|\, \mathbf{x})$$

and, thus, we are training the neural network to output a conditional probability. Note that $f(\mathbf{x})$ must be strictly between 0 and 1, and to achieve this, it is common to employ the sigmoid as an activation function for the output layer. Finally, the cross-entropy loss function is convex, and this leads to better-working gradient descent algorithms such as SGD.

We can extend the cross-entropy loss to multi-class classification tasks using *one-hot encoding*. The labels become a vector $\mathbf{y}$ that has one entry for each class. The value of these entries is $y_i = 1$ if the sample is in class i and $y_i = 0$ otherwise.

The loss function extension to multi-class classification with labels that are one-hot encoded is

$$L(\mathbf{f}(\mathbf{x}),\, \mathbf{y}) = -\sum_i y_i \log(f_i(\mathbf{x})) \tag{14.4}$$

14.8 Generative Adversarial Networks

Generative Adversarial Networks, or GANs for short, are an approach to generative modeling using deep learning methods, such as convolutional neural networks. Generative modeling is an unsupervised learning task in machine learning that involves automatically discovering and learning the regularities or patterns in input data so that the model can be used to generate or output new examples that plausibly could have been drawn from the original dataset.

GANs are a clever way of training a generative model by framing the problem as a supervised learning problem with two sub-models: the generator model that we train to generate new examples and the discriminator model that tries to classify examples as either real (from the domain) or fake (generated). The two models are trained together in a zero-sum game, adversarial, until the discriminator model is fooled about half the time, meaning the generator model generates plausible examples.

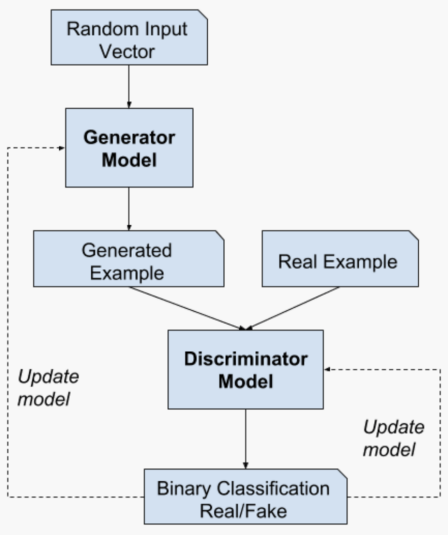
In supervised learning, we may be interested in developing a model to predict a class label given an example of input variables. This predictive modeling task is called classification. Classification is also traditionally referred to as discriminative modeling. This is because a model must discriminate examples of input variables across classes and choose or decide which class a given example belongs to. Alternately, unsupervised models that summarize the distribution of input variables may be able to create or generate new examples in the input distribution. As such, these types of models are referred to as generative models.

Generative Adversarial Networks, or GANs, are a deep-learning-based generative model. More generally, GANs are a model architecture for training a generative model, and it is most common to use deep learning models in this architecture.

The GAN model architecture involves two sub-models: a generator model for generating new examples and a discriminator model for classifying whether generated examples are real, from the domain, or fake, generated by the generator model.

- **Generator**: Model used to generate new plausible examples from the problem domain.
- **Discriminator**: Model used to classify examples as real (from the domain) or generated.

Generative modeling is an unsupervised learning problem, although a clever property of the GAN architecture is that the training of the generative model is framed as a supervised learning problem. The two models, the generator and discriminator, are trained together. The generator generates a batch of samples, and these, along with real examples from the domain, are provided to the discriminator and classified as real or fake. The discriminator is then updated better to discriminate real and fake samples in the next round. Importantly, the generator is updated based on how well or not the generated samples fooled the discriminator.



Generative Adversarial Network model

14.8.1 Loss function

The training of a Generative Adversarial Network is formulated as a *min-max* game, where

1. The Discriminator is trying to maximize its reward $V(D, G)$ (i.e. minimize its loss)
2. The Generator is trying to minimize the Discriminator's reward (i.e. maximize its loss)

In other words, $\min_G \max_D V(D, G)$, where the reward is

$$V(D, G) = \mathbb{E}_{x \sim p(x)}[\log D(x)] + \mathbb{E}_{z \sim q(z)}[\log(1 - D(G(z)))] \tag{14.5}$$

and the first term consists of true samples, while the second consists of generated samples.

The Nash (ideal) equilibrium of this game is achieved when $P_{\text{data}}(x) = P_{\text{gen}}(x)$ for all x and $D(x) = 1/2$ for all x.

Chapter 15

Unsupervised learning

This chapter will deal with a few basic ideas of *unsupervised learning*. So far, we have mainly dealt with supervised learning (e.g., the problem of learning a classifier from labeled training data). The goal of supervised learning is clear: we wish to learn a classifier that will predict the labels of future examples as accurately as possible. Furthermore, a supervised learner can estimate the success, or the risk, of its hypotheses using the labeled training data by computing the empirical loss.

In *unsupervised learning*, there are no labels we try to predict. We thus wish to organize the data in some meaningful way, but as a result, there is no straightforward success evaluation procedure.

15.1 Clustering

Clustering is one of the most widely used techniques for exploratory data analysis. Across all disciplines, people try to get a first intuition about their data by identifying meaningful groups among the data points. Intuitively, clustering is the task of grouping a set of objects such that similar objects end up in the same group and dissimilar objects are separated into different groups. Clearly, this description is quite imprecise and possibly ambiguous, but it is unclear how to come up with a more rigorous definition.

There are several sources for this difficulty. One basic problem is that the two objectives mentioned in the earlier statement may, in many cases, contradict each other. For example, mathematically speaking, similarity (or proximity) is not a transitive relation, while cluster sharing is an equivalence relation, and it is a transitive relation. More concretely, it may be the case that there is a long sequence of objects $x_1, \ldots, x_m$ such that every x_i is very similar to its two neighbors x_{i-1} and x_{i+1} but that x_1 and x_m are very dissimilar. If we wish to ensure that whenever two elements are similar, they share the same cluster, then we must put all the sequence elements in the same cluster. However, in that case, we end up with two dissimilar elements (x_1 and x_m) sharing a cluster, thus violating the second requirement.

Another basic problem is the lack of ground truth for clustering, a common issue in unsupervised learning. Even based on full knowledge of the underlying data distribution, it is unclear what is the *correct* clustering for that data or how to evaluate a proposed clustering.

To summarize, several very different conceivable clustering solutions may exist for a given dataset. As a result, there is a wide variety of clustering algorithms that, on some input data, will output very different clusterings.

15.1.1 A clustering model

Clustering tasks can vary in terms of the type of input they have and the outcome they are expected to compute. For concreteness, we shall focus on the following common setup:

Input: a set of elements $\boldsymbol{\mathcal{X}}$ and a distance function over it. That is, a function $d : \boldsymbol{\mathcal{X}} \times \boldsymbol{\mathcal{X}} \to \mathbb{R}_+$ that is symmetric, satisfies $d(x, x) = 0$ for all $x \in \boldsymbol{\mathcal{X}}$ and often also satisfies the triangle inequality. Alternately, the function could be a similarity function $s : \boldsymbol{\mathcal{X}} \times \boldsymbol{\mathcal{X}} \to [0, 1]$ that is symmetric and

satisfies $s(x, x) = 1$ for all $x \in \boldsymbol{\mathcal{X}}$. Some clustering algorithms also require an input parameter k to determine the number of required clusters.

Output: a partition of the domain set $\boldsymbol{\mathcal{X}}$ into subsets. That is, $\boldsymbol{\mathcal{C}} = (\boldsymbol{\mathcal{C}}_1, \ldots, \boldsymbol{\mathcal{C}}_k)$ where $\bigcup_{i=1}^{k} \boldsymbol{\mathcal{C}}_i = \boldsymbol{\mathcal{X}}$ and for all $i \neq j$, $\boldsymbol{\mathcal{C}}_i \cap \boldsymbol{\mathcal{C}}_j = \emptyset$. In some situations, the clustering is "soft," namely, the partition of $\boldsymbol{\mathcal{X}}$ into the different clusters is probabilistic where the output is a function assigning to each domain point $x \in \boldsymbol{\mathcal{X}}$ a vector $(p_1(x), \ldots, p_k(x))$, where $p_i(x) = \mathbb{P}[x \in \boldsymbol{\mathcal{C}}_i]$ is the probability that x belongs to cluster $\boldsymbol{\mathcal{C}}_i$.

15.1.2 K-Means clustering algorithm

A modern approach to clustering is called *cost-based clustering*. It starts by defining a cost function over a parametrized set of possible clusterings, and the algorithm aims to find a partitioning of minimal cost. Under this paradigm, the clustering task is turned into an optimization problem. The objective function is a function from pairs of inpit $(\boldsymbol{\mathcal{X}}, d)$ and a proposed clustering solution $\boldsymbol{\mathcal{C}} = (\boldsymbol{\mathcal{C}}_1, \ldots, \boldsymbol{\mathcal{C}}_k)$ to positive real numbers. Given such an objective function, which we denote by G, the goal of a clustering algorithm is defined as finding for a given input $(\boldsymbol{\mathcal{X}}, d)$, a clustering $\boldsymbol{\mathcal{C}}$ so that $G((\boldsymbol{\mathcal{X}}, d), \boldsymbol{\mathcal{C}})$ is minimized. In order to reach that goal, one has to apply some appropriate search algorithm. Note that most of the resulting optimization problems are NP-hard. Consequently, when people talk about k-means clustering (for example), they often refer to some particular common approximation algorithm rather than the cost function or the corresponding exact solution of the minimization problem.

The k-means objective function is one of the most popular clustering objectives. In k-means the data is partitioned into disjoint sets $\boldsymbol{\mathcal{C}}_1, \ldots, \boldsymbol{\mathcal{C}}_k$ where each $\boldsymbol{\mathcal{C}}_i$ is represented by a centroid μ_i. It is assumed that the input set $\boldsymbol{\mathcal{X}}$ is embedded in some larger metric space $(\boldsymbol{\mathcal{X}}', d)$ (such that $\boldsymbol{\mathcal{X}} \subseteq \boldsymbol{\mathcal{X}}'$) and centroids are members of $\boldsymbol{\mathcal{X}}'$. The k-means objective function measures the squared distance between each point in $\boldsymbol{\mathcal{X}}$ to the centroid of its cluster. The centroid of $\boldsymbol{\mathcal{C}}_i$ is defined to be

$$\mu_i(\boldsymbol{\mathcal{C}}_i) = \arg\min_{\mu \in \boldsymbol{\mathcal{X}}'} \sum_{x \in \boldsymbol{\mathcal{C}}_i} d(x, \mu)^2 \tag{15.1}$$

Then, the k-means objective is

$$G_{k-\text{means}}((\boldsymbol{\mathcal{X}}, d), \boldsymbol{\mathcal{C}}) = \sum_{i=1}^{k} \sum_{x \in \boldsymbol{\mathcal{C}}_i} d(x, \mu_i(\boldsymbol{\mathcal{C}}_i))^2 \tag{15.2}$$

that can also be rewritten as

$$G_{k-\text{means}}((\boldsymbol{\mathcal{X}}, d), \boldsymbol{\mathcal{C}}) = \min_{\mu_1, \ldots, \mu_k \in \boldsymbol{\mathcal{X}}'} \sum_{i=1}^{k} \sum_{x \in \boldsymbol{\mathcal{C}}_i} d(x, \mu_i)^2 \tag{15.3}$$

It turns out that finding the optimal k-means solution is often computationally infeasible. As an alternative, the following simple iterative algorithm is often used, so often that, in many cases, the term k-means clustering refers to the outcome of this algorithm rather than to the clustering that minimizes the k-means objective cost. We describe the algorithm with respect to the Euclidean distance function $d(\mathbf{x}, \mathbf{y}) = ||\mathbf{x} - \mathbf{y}||$.

Lemma 15.1.1. *Each iteration of the k-means algorithm does not increase the k-means objective function.*

While the lemma tells us that the k-means objective is monotonically non-increasing, there is no guarantee on the number of iterations the k-means algorithm needs to reach convergence. Furthermore, there is no non-trivial lower bound on the gap between the value of the k-means objective of the algorithm's output and the minimum possible value of that objective function. In fact, k-means might converge to the point that it is not even a local minimum. To improve the results of the k-means, it is often recommended to repeat the procedure several times with different randomly chosen initial centroids.

Algorithm: k-Means

input: The input dataset $\boldsymbol{\mathcal{X}} \subset \mathbb{R}^n$ and the required number of clusters k
initialize: Randomly choose initial centroids $\mu_1, \ldots, \mu_k$
repeat until convergence:
$\forall i \in [k]$ set $\boldsymbol{\mathcal{C}}_i = \{\mathbf{x} \in \boldsymbol{\mathcal{X}} : i = \arg\min_j ||\mathbf{x} - \mu_j||\}$
(break ties in some arbitrary manner)
$\forall i \in [k]$ update $\mu_i = \frac{1}{|\boldsymbol{\mathcal{C}}_i|} \sum_{\mathbf{x} \in \boldsymbol{\mathcal{C}}_i} \mathbf{x}$

15.1.3 Linkage-based clustering algorithms

Linkage-based clustering is probably the most straightforward paradigm of clustering. These algorithms proceed in a sequence of rounds. They start from the trivial clustering that has each data point as a single-point cluster. Then, repeatedly, these algorithms merge the closest clusters of the previous clustering. Consequently, the number of clusters decreases with each such round. If kept going, such algorithms would eventually result in trivial clustering in which all domain points share one large cluster. Two parameters, then, need to be determined to define such an algorithm clearly. First, we must decide how to measure the distance between clusters, and second, we must determine when to stop merging. Recall that the input to a clustering algorithm is a between-points distance function d. There were many ways of extending d to a measure of distance between domain subsets. The most common ways are

1. Single Linkage clustering, in which the between-clusters distance is defined by the minimum distance between members of the two clusters

$$D(A, B) = \min\{d(x, y) : x \in A, y \in B\} \tag{15.4}$$

2. Average Linkage clustering, in which the distance between two clusters is defined to be the average distance between a point in one cluster and a point in the other

$$D(A, B) = \frac{1}{|A||B|} \sum_{x \in A, y \in B} d(x, y) \tag{15.5}$$

3. Max Linkage clustering, in which the distance between two clusters is defined as the maximum distance between their elements

$$D(A, B) = \max\{d(x, y) : x \in A, y \in B\} \tag{15.6}$$

The linkage-based clustering algorithms are *agglomerative* in the sense that they start from completely fragmented data and keep building larger clusters as they proceed. Without employing a stopping rule, the outcome of such algorithms can be described by a clustering *dendrogram*: that is, a domain subsets tree with the singleton sets in its leaves and the full domain as its root. For example, if the input is the elements $\boldsymbol{\mathcal{X}} = \{a, b, c, d, e\} \subset \mathbb{R}^2$ with the Euclidean distance, then the resulting dendrogram is the one shown in Figure 15.1.

If one wishes to turn a dendrogram into a partition of the space, one needs to employ a stopping criterion. Common stopping criteria include

1. Fixed number of clusters, where we fix some parameter k and stop merging clusters as soon as the number of clusters is k.

2. Distance upper bound, where we fix some $r \in \mathbb{R}_+$ and stop merging as soon as all the between-clusters distances are larger than r.

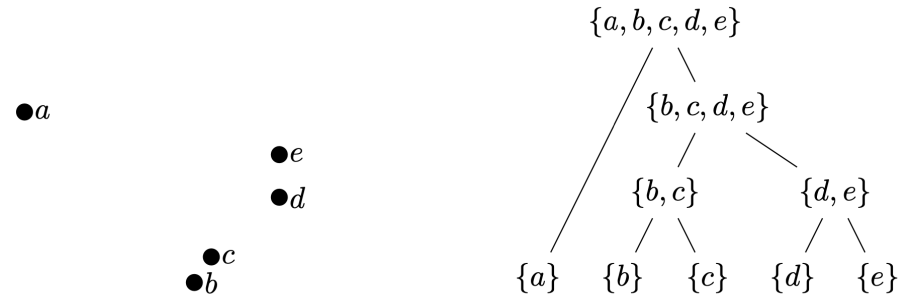


Figure 15.1: Left: data points in $\mathbb{R}^2$. Right: single linkage clustering dendrogram.

15.2 Principal Component Analysis (PCA)

Dimensionality reduction is taking data living in a high-dimensional space and mapping it into a new space with much smaller dimensionality. There are several reasons to reduce the dimensionality of the data. First, high dimensional data impose computational challenges. Moreover, in some situations, high dimensionality might lead to poor generalization abilities of the learning algorithm (for example, in Nearest Neighbor classifiers, the sample complexity increases exponentially with the dimension). Finally, dimensionality reduction can be used for the interpretability of the data, for finding the meaningful structure of the data, and for illustration purposes.

In this section, we present a method called *Principal Component Analysis* (PCA), in which the reduction is performed by applying a linear[1] transformation to the original data. Formally, if the original data is in $\mathbb{R}^d$ and we want to embed it into $\mathbb{R}^n$ with $n < d$, then we would like to find a matrix $W \in \mathbb{R}^{(n,d)}$ that induces the mapping $\mathbf{x} \mapsto W\mathbf{x}$. A natural criterion for choosing W is in a way that will enable a reasonable recovery of the original $\mathbf{x}$, though in general, the exact recovery of $\mathbf{x}$ from $W\mathbf{x}$ is impossible. In PCA, both the compression and the recovery are performed by linear transformations. The method finds the linear transformations for which the differences between the recovered and original vectors are minimal in the least squared sense.

15.2.1 Mathematical construction

Let $\mathbf{x}_1, \ldots, \mathbf{x}_m$ be m vectors in $\mathbb{R}^d$. We want to reduce the dimensionality of these vectors using a linear transformation. A matrix $W \in \mathbb{R}^{(n,d)}$, where $W\mathbf{x} \in \mathbb{R}^n$ is the lower dimensionality representation of $\mathbf{x}$. Then, a second matrix $U \in \mathbb{R}^{(d,n)}$ can be used to approximately recover each original vector $\mathbf{x}$ from its compressed version. That is, for a compressed vector $\mathbf{y} = W\mathbf{x}$, where $\mathbf{y}$ is in the low dimensional space $\mathbb{R}^n$, we can construct $\tilde{\mathbf{x}} = U\mathbf{y}$ so that $\tilde{\mathbf{x}}$ is the recovered version of $\mathbf{x}$ and resides in the original high dimensional space $\mathbb{R}^d$.

In PCA, we find the compression matrix W and the recovering matrix U so that the total squared distance between the original and recovered vectors is minimal; namely, we aim to solve the problem

$$\operatorname*{arg\,min}_{W \in \mathbb{R}^{(n,d)}, U \in \mathbb{R}^{(d,n)}} \sum_{i=1}^{m} ||\mathbf{x}_i - UW\mathbf{x}_i||^2 \tag{15.7}$$

Lemma 15.2.1. *Let (U, W) be the optimal solution to Equation 15.7. Then the columns of U are orthonormal (namely, U^TU is the identity matrix in $\mathbb{R}^n$) and $W = U^T$.*

Proof. Fix any U, W and consider the mapping $\mathbf{x} \mapsto UW\mathbf{x}$. The range of this mapping, $R = \{UW\mathbf{x} : \mathbf{x} \in \mathbb{R}^d\}$, is an n dimensional linear subspace of $\mathbb{R}^d$. Let $V \in \mathbb{R}^{(d,n)}$ be a matrix whose columns form an orthonormal basis of this subspace. Namely, the range of V is R, and V^TV is the identity matrix. Therefore, each vector in R can be written as $V\mathbf{y}$ where $\mathbf{y} \in \mathbb{R}^n$. For every $\mathbf{x} \in \mathbb{R}^d$ and $\mathbf{y} \in \mathbb{R}^n$ we have

$$||\mathbf{x} - V\mathbf{y}||^2 = ||\mathbf{x}||^2 + \mathbf{y}^TV^TV\mathbf{y} - 2\mathbf{y}^TV^T\mathbf{x} = ||\mathbf{x}||^2 + ||\mathbf{y}||^2 - 2\mathbf{y}^T(V^T\mathbf{x})$$

[1]Note: in more advanced courses, you will also study non-linear dimensionality reduction strategies.

where we used the fact that $V^T V$ is the identity matrix of $\mathbb{R}^n$. Minimizing the preceding expression with respect to $\mathbf{y}$ by comparing the gradient with respect to $\mathbf{y}$ to zero gives that $\mathbf{y} = V^T \mathbf{x}$. Therefore, for each $\mathbf{x}$ we have that

$$VV^T \mathbf{x} = \arg\min_{\tilde{\mathbf{x}} \in R} ||\mathbf{x} - \tilde{\mathbf{x}}||^2$$

In particular this holds for $\mathbf{x}_1, \ldots, \mathbf{x}_m$ and therefore we can replace U, W by V, V^T and by that we do not increase the objective

$$\sum_{i=1}^{m} ||\mathbf{x}_i - UW\mathbf{x}_i||^2 \geq \sum_{i=1}^{m} ||\mathbf{x}_i - VV^T \mathbf{x}_i||^2$$

and the proof of the lemma follows. □

Based on the preceding lemma, we can rewrite the optimization problem given in Equation 15.7 as follows:

$$\arg\min_{U \in \mathbb{R}^{(d,n)} : U^T U = \mathbb{I}} \sum_{i=1}^{m} ||\mathbf{x}_i - UU^T \mathbf{x}_i||^2 \tag{15.8}$$

We can further simplify the optimization problem using the following algebraic manipulations. For every $\mathbf{x} \in \mathbb{R}^d$ and a matrix $U \in \mathbb{R}^{(d,n)}$ such that $U^T U = \mathbb{I}$ we have

$$\begin{aligned} ||\mathbf{x} - UU^T \mathbf{x}||^2 &= ||\mathbf{x}||^2 - 2\mathbf{x}^T UU^T \mathbf{x} + \mathbf{x}^T UU^T UU^T \mathbf{x} \\ &= ||\mathbf{x}||^2 - \mathbf{x}^T UU^T \mathbf{x} \\ &= ||\mathbf{x}||^2 - \mathrm{Tr}(U^T \mathbf{x}\mathbf{x}^T U) \end{aligned}$$

where the trace of a matrix is the sum of its diagonal entries. Since the trace is a linear operator, this allows us to rewrite Equation 15.8 as follows:

$$\arg\max_{U \in \mathbb{R}^{(d,n)} : U^T U = \mathbb{I}} \mathrm{Tr}(U^T \sum_{i=1}^{m} \mathbf{x}\mathbf{x}^T U) \equiv \arg\max_{U \in \mathbb{R}^{(d,n)} : U^T U = \mathbb{I}} \mathrm{Tr}(U^T A U) \tag{15.9}$$

where $A = \sum_{i=1}^{m} \mathbf{x}\mathbf{x}^T$ is symmetric, and therefore it can be written using its spectral decomposition as $A = VDV^T$ where D is diagonal and $V^T V = VV^T = \mathbb{I}$. Here, the elements on the diagonal of D are the eigenvalues of A, and the columns of V are the corresponding eigenvectors. We assume, without loss of generality, that the eigenvalues are in increasing order. Therefore, since A is positive semidefinite, it also holds that the last eigenvalue $D_{(d,d} \geq 0$. We claim that the solution to Equation 15.9 is the matrix U whose columns are the n eigenvectors of A corresponding to the largest n eigenvalues.

Theorem 15.2.2 (Principal Component Analysis)**.** *Let $\mathbf{x}_1, \ldots, \mathbf{x}_m$ be arbitrary vectors in $\mathbb{R}^d$. Let $A = \sum_{i=1}^{m} \mathbf{x}\mathbf{x}^T$ and let $\mathbf{u}_1, \ldots, \mathbf{u}_n$ be n eigenvectors of the matrix A corresponding to the largest n eigenvalues of A. Then, the solution to the PCA optimization problem given in Equation 15.7 is to set U to be the matrix whose columns are $\mathbf{u}_1, \ldots, \mathbf{u}_n$ and to set $W = U^T$.*

Note that is is common practice to center the examples before applying PCA. That is, we first calculate the vector of means $\mu = \frac{1}{m} \sum_{i=1}^{m} \mathbf{x}_i$ and then apply PCA on the vectors $(\mathbf{x}_1 - \mu), \ldots, (\mathbf{x}_1 - \mu)$. This is also related to the interpretation of PCA as variance maximization.

The computation time is $O(d^3) + O(md^2)$, where the first term is for computing eigenvalues and the second for constructing the matrix A. In case $d \gg m$, there is a trick for a faster solution.

15.2.2 Implementation and demonstration

To illustrate how PCA works, let us generate vectors in $\mathbb{R}^2$ that approximately reside on a line, namely, on a one-dimensional subspace of $\mathbb{R}^2$. For example, suppose that each example is of the form $(x, x + y)$ where x is chosen uniformly at random from $[-1, 1]$ and y is sampled from a Gaussian distribution with mean 0 and standard deviation of 0.1. Suppose we apply PCA to this data. Then, the eigenvector corresponding to the largest eigenvalue will be close to the vector $(\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}})$. When

Algorithm: PCA

input: A matrix of m examples in d dimensions $X \in \mathbb{R}^{(m,d)}$
The number of components n
if $(m > d)$ **then**
  $A = X^T X$
  Let $\mathbf{u}_1, \ldots, \mathbf{u}_n$ be the eigenvectors of A with largest eigenvalues
end
else
  $B = XX^T$
  Let $\mathbf{v}_1, \ldots, \mathbf{v}_n$ be he eigenvectors of B with largest eigenvalues
  for $i = 1, \ldots, n$ **do**
    set $\mathbf{u}_i = \frac{1}{||X^T \mathbf{v}_i||} X^T \mathbf{v}_i$
  end
end
output: $\mathbf{u}_1, \ldots, \mathbf{u}_n$

projecting a point $(x, x + y)$ on this principal component we will obtain the scalar $\frac{2x+y}{\sqrt{2}}$. The reconstruction of the original vector will be $(\frac{x+y}{2}, \frac{x+y}{2})$. We depict the original versus reconstructed data in Figure 15.2.

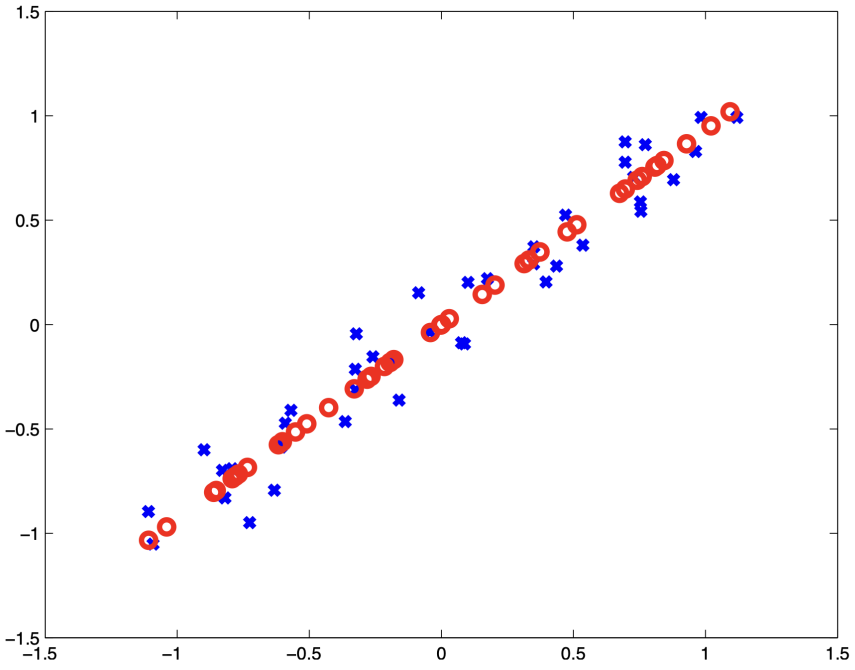


Figure 15.2: A set of vectors in $\mathbb{R}^2$ (the blue x's) and their reconstruction after dimensionality reduction to $\mathbb{R}^1$ using PCA (the red o's).